

**Gedanken: A tool for pondering the
tractability of correct program technology**

by

Lars Warren Ericson

A dissertation submitted in partial fulfillment

of the requirements for the degree of

Doctor of Philosophy

Computer Science Department

New York University

February, 1994

Approved

Prof. Bhubaneswar Mishra

© Lars Warren Ericson

All Rights Reserved 1994

Acknowledgements

I thank Lou Salkind for helping me get some urgently needed files from NYU in February 1988, when I began my visit to Europe.

For the completion of my degree, I thank Bud Mishra and Ed Schonberg of NYU, and Jim Cox and Gerson Levin of Brooklyn College. Discussions with Prof. Cox were especially helpful in working out the ideas of Chapter 5.

I have worked on several thesis topics between 1987 and 1993. Many people have contributed to my studies. I thank Jack Schwartz, Alberto Policriti and Chee Yap of NYU; Colm O'Dúnlaing, his family, and the Trinity College Math Dept; Tomas Recio and the University of Cantabria Math Dept; at INRIA, Phillipe Flajolet and Project ALGO, Gérard Huet and Project FORMEL, J. Michel Chasseriaux, the lady who was the director of external relations, secretaries Virginie and Sylvie of bâtiment 8, and Jacques Levy-Vehel and Hussein Yahia of bâtiment 21; Madames Germa, Molet, and Vignes of ENST; the French Government for a Bourse Chateaubriand; the Esprit ALCOM grant; and for inviting me to speak, Rene Schott of the U. of Nancy, Bruno Buchberger of the U. of Linz, Claude Puech of ENS, and Andre Galligo and Jacques Morgenstern of INRIA-Nice.

For friendship and support, I thank Elliot Ehrich; Jon Thomas; Lorenzo Savorelli; Abba Lahrech, and my family.

Contents

Chapter

1	Summary	1
2	Correct program technology	9
2.1	Specification	9
2.1.1	Function	9
2.1.2	Specification of a function	10
2.1.3	The goal of correct programming	11
2.2	Verification tool design factors	11
2.3	Comparison of three approaches	15
2.3.1	RAPTS	15
2.3.2	Calculus of Constructions	17
2.3.3	Correct Program Technology	19
3	Correct programmer's workbench	22
3.1	Components of a CPW	22
3.1.1	Deliverable CPW	22
3.1.2	Development Environment	23
3.2	Stack in six popular languages	26
3.2.1	Why Stack is a good litmus test	26
3.2.2	Stack in Scheme, an algorithm language	27

3.2.3	Stack in Maple, a computer algebra language	28
3.2.4	Stack in Setl2, an algorithm language	28
3.2.5	Stack in Mathematica, a computer algebra language	29
3.2.6	Stack in Axiom, a computer algebra language	29
3.2.7	Stack in SML, a logic and type theory language	30
3.2.8	Discussion	31
3.3	Factor review of six candidate development platforms	43
4	Correct program editor	45
4.1	Praa programming language	45
4.2	Editor structures	58
4.2.1	Placed propositions and satisfied computations	58
4.2.2	Claims	66
4.2.3	Programs with assumptions and assertions	77
4.2.4	Proof of a claim	81
4.2.5	Proof of a praa	84
4.3	Discussion	89
5	Heuristics	90
5.1	Some key intractable sublanguages	91
5.1.1	Set theory	92
5.1.2	Real algebra	92
5.1.3	Presburger arithmetic and the feasibility problem for integer linear programming	94
5.2	Heuristics for two key sublanguages	95
5.2.1	Real algebra	95
5.2.2	Presburger arithmetic and the feasibility problem for integer linear programming	96

5.2.3	Analysis of these heuristics	96
5.3	Stronger average case results	98
5.3.1	Goldberg for Davis-Putnam for SAT (1979)	99
5.3.2	INFREQ	100
5.3.3	DPE	102
5.3.4	Resolution is hard under a different sampling distribution	104
5.4	The theory of average case complexity	105
5.4.1	Rationale for the new theory	106
5.4.2	Average case complexity classes	109
5.4.3	The average case complexity of <i>SAT</i>	114
5.4.4	The average case complexity of <i>EMLS</i>	115
5.4.5	The average case complexity of <i>FPILP</i>	116
5.5	Relating expressiveness, proof size and verification time	117
6	Elementary relational languages	121
6.1	Finite structure descriptions	122
6.2	Elementary relational structure	125
6.3	Theorem proving paradigm we assume	126
6.4	EPCPLs	128
6.5	Realization, Denotation and Model	137
6.6	Patterns and match-counting	138
6.7	Syntactic simplification rule sets	143
6.7.1	Elimination simplifiers	144
6.7.2	Invalidating simplifiers	145
7	The match counting problem	146
7.1	Counting matches in $EMLS_n$	148
7.1.1	Reduction of $EMLS_n$ to $L_{4,2,3,n}$	149

7.1.2	Reduction of $L_{j,k,m,n}$ to $L_{k(n+1)+j,m-1,n+1;V}$	150
7.1.3	Reduction of $L_{k,m,n;V}$ to $L_{kn,m-1,n;V}$ for $m > 0$	153
7.1.4	Reduction of $L_{k,0,n;V}$ to PC_k	154
7.1.5	The match counting algorithm for PC_k	156
7.2	Conjoining and disjoining pattern sets	158
7.2.1	Conjoining	158
7.2.2	Disjoining	159
7.3	Match counting worst-case time cost	159
7.3.1	PC_k	159
7.3.2	$L_{k,m,n}$	160
7.4	Some optimizations	161
7.4.1	PC_k	161
7.4.2	$L_{k,m,n}$	163
7.5	Match counting for PC_k is in $\#P$	163
7.6	The generating function approach and approximability	166
8	Deciding EMLS Faster On Average	170
8.1	Elementary multi-level syllogistic	170
8.2	The Model Graph algorithm for <i>EMLS</i>	173
8.2.1	Overview	173
8.2.2	Set-Theoretic Preliminaries	174
8.2.3	The Model Graph Construction	175
8.2.4	The Mostowski Collapse of a Model Graph	175
8.2.5	The Skeletal Model Graph construction	176
8.2.6	Mostowski Collapse of a Skeletal Model Graph	178
8.2.7	Some simplification rules	179
8.3	Analysis of average case effect	180

8.3.1	Definition of average case complexity	181
8.3.2	Pragmatic considerations	182
8.3.3	Analysis for the simplifier model graph algorithm	185
9	Conclusions	190
9.1	Pragmatics	190
9.2	Outstanding Issues	191
 Appendix		
A	Gedanken	192
A.1	Correct Program Editor	192
A.1.1	Target programming language	192
A.1.2	Placed propositions	196
A.1.3	Claims	200
A.1.4	Praas	209
A.1.5	Claim proofs	210
A.1.6	Praa proofs	211
A.2	Rule Set Analyser	213
 Bibliography		
		231

Figures

Figure

3.1	Factor comparison of six candidate proof metalanguages	44
4.1	Context of operational semantics	47
4.2	Praa programming language	48
4.3	Placed propositions	59
4.4	Claims	67
4.5	Claim proof rules	67
4.6	Programs with assumptions and assertions	77
4.7	Praa transformation rules	78
4.8	Claim Proofs	82
4.9	Praa Proofs	84
5.1	Franco's CNF simplification rules	101
5.2	Average-Case Complexity of <i>NP</i> -Complete Languages	113
6.1	Abstract syntax of elementary languages in <i>Gedanken</i>	129
7.1	Match counting algorithm for patterns over PC_k	157
8.1	$\log_2$ speed of Model Graphs after elimination	187
8.2	$\log_2$ speed-up of Model Graphs after elimination	188
8.3	$\log_2$ speed of Model Graphs after invalidation	188

8.4	$\log_2$ speed-up of Model Graphs after invalidation	189
-----	--	-----

Chapter 1

Summary

One goal of computer science has been to develop a tool T to aid a programmer in building a program P that satisfies a specification S by helping the programmer build a proof in some logic of programs L that shows that P satisfies S . S typically is a pair of propositions (ϕ, ψ) such that, for an input x to P , $\phi(x) \Rightarrow \psi(P(x))$ when P is defined on x . ϕ is called the **precondition** or **assumption**, and ψ is called the **postcondition** or **assertion**. The problem of finding a suitable logic L of programs and specifications and verification tool T may be generically referred to as the “Floyd-Hoare problem”, formulated around 1967 [Flo67, Hoa69].

Around 1977, Davis and Schwartz proposed an extension of the Floyd-Hoare problem in which there are multiple assumptions and assertions, referring to the state of a program as execution passes through different **places** π in the program [DS77, Sch77]. A **placed proposition** is then a pair (ϕ, π) , where π is either a line of a program or the name of a function. A placed proposition (ϕ, π) **holds** when, if execution reaches π and the value of the variables X in P is V , then $\phi(V)$ is valid. A **program with assumptions and assertions** or **praa** is then a triple $R = (P, E, F)$ where the assumptions E and assertions F are sets of placed propositions. The pair (E, F) is the specification for P , and R is **correct** when, for every (ϕ, π) in F , if every assumption in E holds, then (ϕ, π) holds. They proposed a logic of programs L for establishing the correctness of praas. Deak in her 1980 PhD thesis showed how this logic could be

applied to derive several variants of searching algorithms from a common root algorithm [Dea77, Dea80]. The Davis-Schwartz vision of the ideal verification tool T was one in which the inference rules I of L could be extended by decision procedures for commonly occurring programming language constructs, subject to the restriction that the set $\mathcal{R}$ of correct praas verifiable in L would not change under any extension to I . That is, assuming that L is sound, any extension of L is **stable** and consequently also sound. This approach is called the “correct-program technology problem” [Sch78]. We examine this approach, and two others discussed below, in Chapter 2.

Davis and Schwartz sketched the outlines of a verification tool. As part of our work in this thesis, we:

- Discuss, in Chapter 3, the engineering requirements for a “Correct programmer’s workbench”, and examine several candidate programming environments for their suitability as development environments for this problem.
- Elaborate, in Chapter 4, a portion of their sketch into the data structures for a **verifying correct program editor**, that is, an editor whose steps take correct praas to correct praas. We code these structures in **Mathematica** as part of our **Gedanken** package. Our observations concerning the computational complexity obstacles to correct program technology will be pessimistic. But we may still hope that an accumulation of small advances in commonly-occurring tractable special cases in program verification will yield an aggregate set of techniques sufficient to produce a practical verification tool. When this becomes the case, our editor design will be applicable.

Davis and Schwartz proposed that a series of decision procedures be discovered for sublanguages of Zermelo-Frankel set theory corresponding to programming constructs commonly used in the high-level set-oriented programming language SETL [Sny90a, Sny90b]. Two key assumptions were that

- (1) By replacing long proofs with trivial steps by short proofs with complex steps, it would be cognitively easier for a programmer to assist the verification tool in finding a correctness proof for a praa, and the average time cost of verifying a praa would be reduced. However, if the proofs establish the correctness of sentences in a language for which the problem of recognizing correct sentences is decidable and NP-Complete, then in the worst case, any systematic choice in the spectrum of long proofs-easy steps/short proofs-long steps will yield the same worst case time complexity, and hence the fact that the problem is NP-Complete alone tells us all we need to know from a worst-case standpoint. It would nevertheless be interesting to precisely formulate this proof step verification time/proof length tradeoff. Accordingly, in Chapter 5 we frame this problem precisely using Kolmogorov complexity.
- (2) The cost $T_{2\circ 1}$ of verifying the correctness of the **composition** of two praas R_1 and R_2 would often be significantly less than the costs T_1, T_2 of verifying the correctness of R_1 and R_2 individually. But let P_1, P_2 be programs, with assumptions ϕ_1, ϕ_2 and assertions ψ_1, ψ_2 , so that our specification of P_1 (P_2) is $\phi_1(x) \Rightarrow \psi_1(P_1(x))$ ($\phi_2(x) \Rightarrow \psi_2(P_2(x))$). Suppose we wish to construct a program $P_3 = P_2 \circ P_1$, and that we expect P_3 to satisfy assumption ϕ_3 and assertion ψ_3 . That is, we want to prove that $\phi_3(x) \Rightarrow \psi_3(P_2(P_1(x)))$. The claim that $T_{2\circ 1} \ll T_1 + T_2$ then becomes the claim that the time cost of verifying the proposition

$$(\forall x) \phi_3(x) \Rightarrow \phi_1(x) \wedge \psi_1(x) \Rightarrow \phi_2(x) \wedge \psi_2(x) \Rightarrow \psi_3(x)$$

is of a lower order of magnitude than the time cost of verifying the proposition

$$(\forall x) \phi_1(x) \Rightarrow \psi_1(P_1(x)) \phi_2(x) \Rightarrow \psi_2(P_2(x))$$

But it is easy to construct cases in which this is not true. For example, let

x be an integer, and interpret x as the Gödel number $\langle T, y \rangle$ of the pair of some Turing machine M and an input y to M , let $\phi_1, \psi_1, \phi_2, \phi_3$ be True, let $P_1(x) = P_2(x) = x$, and let $\psi_3(\langle M, y \rangle) = \text{Halt}(M, y)$ be the halting predicate. Then $T_{2\circ 1}$ is undefined because $\text{Halt}(M, y)$ is undecidable in general [Man74]. Therefore the hope that composition is cheap must be accompanied by a far more detailed context than the one that says it is easier to write an application when a library of correct programs is available than when one is not. Depending on how well-designed the library is, it can be cheaper to start from scratch, because adherence to a specification and adequacy of the specification to express a given intent are two different things.

In the 80's, many approaches to the Floyd-Hoare problem and the correct program technology problem were explored, too many for us to categorize in this thesis. It suffices for our purposes to examine two additional approaches, the RAPTS system of Cai and Paige [CP89], and the Calculus of Constructions of Huet, Coquand and Paulin-Mohring (the literature includes [CH85],[Coq86],[Moh86], [PM88],[Hue88a],[Hue88b],[Hue89]). RAPTS proceeds by **compiling** a high-level specification set-valued fixed-point equation directly into a SETL program for computing the fixed point, and limits itself to fixed point equation classes for which a solution algorithm exists. The Calculus of Constructions **realizes** a typed λ -calculus program from the proof that a specification in a logic without the Law of the Excluded Middle is satisfiable. (That is, there is no axiom of the form $a \vee \neg a$.) The language of specifications may be limited to those constructs for which a general method of proving satisfiability is known. In all three cases, the Davis-Schwartz approach, RAPTS and the Calculus of Constructions, we see a reliance on decision procedures for establishing equivalence of values constructable by composition from some class of functions, relations and types. In all three cases, we observe that three strengths of specification language naturally arise:

- Specifications for primitive recursive functions.
- Specifications for some class of partial recursive functions for which verification (Davis-Schwartz), compilation (RAPTS) or realization (Constructions) is decidable. The decision problem for interesting such classes is typically NP-Complete [GJ79].
- Specifications expressing all partial recursive functions, for which verification/compilation/realization is undecidable.

And in every case, we are faced with a choice of whether to:

- (1) Use a tool which is logically strong enough to prove the correctness of only the simplest algorithms.
- (2) Use a tool which always verifies the correctness of some interesting class of algorithms, but which never does so in a practical amount of time.
- (3) Use a tool for which there is no decision procedure for the correctness for all algorithms specifiable in the specification language, but which can check the steps of any valid correctness proof, by verifying that the proof is well-formed, that each step is justified, and that the final step asserts the correctness of the program.

It is then natural to focus on the problem of improving on the average cost of deciding problems whose decision problem is in general intractable. In Chapter 5, we review work on the problem of improving the average cost of deciding sentences in languages whose decision problem is in general intractable. First, we examine three sublanguages, in set theory [FOS80, Pol87a], real algebra [CH91, Hon93] and integer linear programming [Dav57, Co071, Co072, Sho77, CZ93]. The average-case complexity analyses arising in this work tends to be somewhat anecdotal. Several approaches

have arisen which give optimistic or at least more definitive average-case complexity results. For example, we can search for tiny sublanguages whose decision problem has polynomial time complexity in the worst case [Pra77, Sho81]. Or we can show that the average case time complexity of some selection of the instances of an intractable decision problem is of a lower order of magnitude than the worst case for all instances. We briefly review such results for variants of the Davis-Putnam propositional decision procedure [DP60, Gol79, Fra86, Fra91, PT92]. Several approaches have arisen which give pessimistic results: One approach is to show that if the decision problem for a given L is NP-Complete, then it is also **average-case hard in the sense of Levin** [Lev86, BDCGL89, BG91, Gur91, VR92, SY92, RS93]. We define the general terms and some of the results of this approach, and prove that MLS is NP-average complete. Secondly, it is clear that if we tailor a logic to a certain NP-Complete problem, in terms of our choice of axioms and inference rules (for example, we can build a strong inference rule like “equivalence under associative-commutative laws”), then many proofs for instances of that problem will be shorter, but the verification cost for establishing the enabling conditions of such an inference rule for each step may be higher. Conversely, a proof with simpler rules may be much longer, but each step may be easier to check. This suggests a certain “incompressibility” of the product of proof size and verification cost. We make a conjecture that comparable short obtained by varying the logical system for NP-Complete problems are not checkable significantly faster than long proofs in unenhanced systems.

We define, in Chapter 6, a specific class of elementary relational languages L , elementary positive conjunct parametrised propositional languages (epcppl’s). The epcppl’s are purely existentially quantified conjuncts of non-negated literals. The literals are applications a collection of m relations R_i of arities β_i , over some number of variables and constants. We code this in **Mathematica** as part of our package **Gedanken**. We define the concept of a syntactic simplifier rule set R of this class of languages.

In Chapter 7, we showing how to reduce the problem of counting the matches of a pattern set in $EMLS_n$ to the problem of counting matches for a related pattern set in a much simpler language PC_k of conjuncts of propositional constants. The chapter gives methods of counting matches for conjunctions and disjunctions of pattern sets; worst-case time complexity analyses of the basic algorithms; some optimizations; and proof that match counting for PC_k is in $\#P$. We conjecture that match-counting is $\#P$ -Complete. The chapter concludes by exploring the use of generating functions to encode, for a given rule set Π , as n increases, the number of matches of Π with $EMLS_n$. We note that the obvious sequence of generating function constructions fails early on, at the stage of counting the elements of a power set. Our hope in exploring generating functions is to find a method of computing, in polynomial time, an approximation to the match count valid to within an order of magnitude. We conjecture that there is no solution to finding a closed-form generating function for the number of elements of a power set of a set in a family of sets indexed by n whose size is given by another generating function, and from that failure we conjecture that the problem of approximating the match count for PC_k is just as hard as computing the exact match count.

In Chapter 8, we do an algorithm-level analysis of the prospects for improving the average case complexity of deciding sentences in $EMLS$. We “fall back” to simpler average case analysis techniques than those used in Chapter 5. We sketch the constructions of the Model Graph Algorithm of Policriti for deciding $EMLS$ [Pol87a], and we give some simplification rule sets for $EMLS$ defined Policriti, who hoped to speed up the Model Graph Algorithm in practical use [Private communication, 1992]. We do a small practical analysis, using the rule set analyser component of *Gedanken* described in the previous chapter, of a subset of Policriti’s rule sets. We can show a speedup which is significant but which still does not affect the order of magnitude cost of deciding $EMLS$ for practical problem sizes.

We conclude, in Chapter 9, with a summary of our main conclusions and of the

key areas that remain to be clarified.

Chapter 2

Correct program technology

The “Correct Program Technology” initiative was proposed by Davis and Schwartz in 1977 [Sch77, DS77, Sch78] as an enhancement of the Floyd-Hoare approach [Flo67, Hoa69]. Subsequently, the most successful directly-related developments have been a thought-piece on transformational programming [Dea77, Dea80], and theoretical work on decidable sublanguages of set theory [FOS80, CFOS87, CFO89, COP90, Pol87a, Pol87b, PS92, PT92]. We first define factors to consider in reviewing a given approach towards building verification tools, and then we briefly consider three quite distinct approaches to building such tools.

2.1 Specification

We define the notion of a **function**, then we say what it means to **specify** a function, and then, in terms of those definitions, we give the goal of correct programming.

2.1.1 Function

The classic notion of a function is defined as follows.

- Let A and B be sets.
- Define the product of A and B as

$$A \times B = \{(a, b) : a \in A \wedge b \in B\}$$

- Define the power set of A as

$$\text{Pow}(A) = \{a : a \subseteq A\}$$

- Define the maps from A to B as

$$\text{Map}(A, B) = \text{Pow}(A \times B)$$

- Define the functions from A to B as

$$A \rightarrow B = \{m \in \text{Map}(A, B) : \forall (a, b), (a', b') \in m : a = a' \Rightarrow b = b'\}$$

2.1.2 Specification of a function

We define the concept of a specification of a function as follows:

Let A, B be sets, and $f : A \rightarrow B$ a function. Let $Q(x, y)$ be a predicate, in a first-order logic L of set theory, such that

- (1) If $x \notin A$ or $y \notin B$ then $Q(x, y)$ is false.
- (2) If $x \in A$ and $y \in B$ and $f(x) = y$ then $Q(x, y)$ is true, else false.
- (3) If $Q(x, y)$ and $Q(x', y')$ are true and $x = x'$ then $y = y'$.

Such a Q we call an **implicit specification** of f .

Let T be a Turing Machine [Man74, §1-2.1]. Suppose there is a program P for T such that if $x \in A$ is written on the tape of T and $Q(x, y)$, then T running P will halt in an accepting state with y , where Q is a specification. Then the function f specified by Q is called partial recursive.

Suppose Q specifies $f : A \rightarrow B$ and Q' specify $g : B \rightarrow C$. Define the **logical composition** $R \equiv (Q' \circ Q)$ as $R(x, z) \equiv (\exists y \in B) Q(x, y) \wedge Q'(y, z)$. Then $R(x, z)$ specifies $h : A \rightarrow C$ where $h(x) = (g \circ f)(x) = g(f(x))$.

Note that some specifications have no associated programs, for example $\text{Halt}(\langle P, x \rangle)$ where $\langle P, x \rangle$ is an encoding as an integer of the pair of program P and x is an input to P , is a specifiable function which is not computable.

2.1.3 The goal of correct programming

Finally, the goal of correct programming is as follows:

- (1) To define a “suitable first order logic of set theory” L .
- (2) Within L , to find a sublanguage S in which every specification Q describes a partial recursive function.
- (3) From a specification Q in S , to find a program P which computes the partial recursive function $f : A \rightarrow B$ that Q specifies.
- (4) From a program P for f specified by Q , to find a P' which also computes f , but which is more efficient than P , and ideally, is as efficient in the worst case as any P'' computing f .
- (5) From a worst-case optimal P , to find an average-case optimal P' .
- (6) Given two correct programs P_f and P_g computing the $f : A \rightarrow B$ and $g : B \rightarrow C$ specified by Q_f and Q_g , to efficiently perform steps 3-5 for the program computing $g \circ f$ as specified by $Q_g \circ Q_f$, the logical composition of Q_g and Q_f .

2.2 Verification tool design factors

A **program verification tool** allows a programmer to:

- (1) Specify and derive correct programs.
- (2) Compose correct programs.
- (3) Store, retrieve, execute, and inspect the execution state of correct programs.

We now describe some factors to when evaluating a given program verification tool design:

- **Semantic basis.** Is the meaning of a program viewed in an operational fashion or in a fixed-point semantics or denotational fashion? The distinction seen in the following example discussed by Manna [Man74, §5]. Let P be

$$P : F(x, y) \leftarrow \text{if } x = y \text{ then } y + 1 \text{ else } F(x, F(x - 1, y + 1))$$

then if $F(x, y)$ is replaced by any of the following expressions:

$$f_1 : \text{if } x = y \text{ then } y + 1 \text{ else } x + 1$$

$$f_2 : \text{if } x \geq y \text{ then } x + 1 \text{ else } y - 1$$

$$f_3 : \text{if } x > y \wedge x - y \text{ is even then } x + 1 \text{ else } \perp$$

and the $\leftarrow$ is replaced by $=$, the resulting expression is valid. It happens to be the case that whenever f_3 is defined, so are f_1 and f_2 , and they have the same value. It turns out that f_3 is the only such function, and so it is called the least fixed point of P (fixed point in the sense that the definition of P above can be viewed as an equation of the form $F = G(F)$). To choose fixed-point semantics means to say that the partial recursive function computed by P is f_3 . To choose operational semantics is to define a machine M capable of executing P on stored arguments x, y , and to say that the function computed by P is the final state of M on P and x, y . This is not quite the same in the sense that we can construct machines M whose computations are “overdetermined” in the sense that we can choose a set of default computation rules to apply that may lead us to compute, say, f_1 or f_2 in lieu of f_3 . This situation is undesirable because a single program then has multiple meanings, depending on our choice of evaluation mechanisms for P . The burden on systems that choose an operational semantics is to avoid being unjustifiably overdetermined. To avoid this burden it suffices to choose a specification language and proof system that always interpret P as its least

fixed point.

- In proving the correctness of a program, is the **inference method** one of axiomatic proof? Does the user develop a proof of correctness by applying a fixed number of axioms and rules of inference (axiomatic proof)? (When reasoning axiomatically from first principles, it is preferable to reason from principles that are as powerful as possible, for example using Vuillemin's fixed-point induction technique [Man74, §5-3.3].) Does the system maintain a proof structure, in the logician's sense of a proof, that records the justification for each step in a conclusion? Is the axiomatic system supplemented by inference rules applying decision procedures for decidable sublanguages (decision procedure proof step), for example, integer linear programming, boolean operations on sets, etc.? Or is the system used in a purely transformational mode, so that correct programs are derived from correct programs, and so only the sequence of derived programs is the proof of correctness?
- **Expressiveness** of the specification language. Can it specify only primitive recursive functions? Can it specify any partial recursive function? Is its expressiveness extended by some mix of decidable sublanguages?
- **Metamathematical extensibility.** Let L be the logic of our proof system S , and L' a certain subset of it. We assume S is sound for L , that is, it constructs only valid proofs of theorems in L . Let T be the set of all theorems provable in L' . Let F be the class of functions specifiable in L' , and let P be the class of programs implementing some specifiable function in L' . Let f be a specifiable function on a set-encoding of terms (or propositions) of L' . Write $\langle x \rangle$ for an encoding of x as an integer. Suppose we can prove in L that, for all x , for all propositions Q , $\vdash Q(f(\langle x \rangle)) \equiv Q(x)$. Then L is **metamathematically extensible** if

- (1) We can extend L with the inference rule “Replace x by $f(\langle x \rangle)$, where $f(\langle x \rangle)$ is obtained by executing the program P implementing f on $\langle x \rangle$ ”, giving a new system L^+ .
- (2) S remains sound for L^+ .
- (3) S is stable from L to L^+ , that is, the theorems of L^+ are exactly the theorems T of L .

The perceived benefit of metamathematical extensibility is that proof lengths (that is, space cost of a proof) may be greatly decreased by the adoption of such inference rules. However, the time cost (the cost of running the program which is the basis of the new inference rule) of the proof may be greatly increased (since, for interesting equivalences, the corresponding reduction procedures are frequently computationally intractable), and so we have the following rule of thumb:

Expressive = Short proof = Intractable inference rules
 Inexpressive = Long proof = Tractable inference rules

This rule of thumb summarizes pragmatic observations for particular decidable sublanguages. At the end of Chapter 5 we make a precise conjecture which attempts to formalize this rule of thumb.

- **Development mode** from the end-user’s point of view. There are two basic ways of proceeding:

- (1) In the **specification-driven** or **program transformation** approach we specify a partial recursive function to be computed, and transform that specification into find a program that computes that function.
- (2) The Floyd-Hoare or **program-driven** or **program verification** approach adds specifications to an existing program, to prove that it is correct. So

instead of starting with a specific partial recursive function to be computed, we start with a program P on machine M with input vector X , output vector Y , pre-condition $F(X)$ and post-condition $G(Y)$. P is partially correct if when $F(X)$ is true, P runs to completion on input X and produces output Y and $G(Y)$ is true. The program-driven approach has the benefit that we can work with pre-existing programs whose correctness we wish to verify by formulating appropriate F and g . But since an already-written program is, to a large extent, the result of a sequence of intentions, which intentions have been largely elided, the specification-driven approach is preferable, as we effectively have more information available about our task.

2.3 Comparison of three approaches

We briefly compare three distinct approaches to correct program technology:

- The RAPTS system of Cai and Paige 1988 [CP89].
- The Calculus of Constructions of Coquand, Huet and Paulin-Mohring 1985 [CH85, Coq86, Hue88b], and
- The Correct Program Technology initiative of Davis, Schwartz and Deak [DS77, Dea77, Dea80, Sch77, Sch78, Sch92].

2.3.1 RAPTS

We will now discuss the RAPTS system of Cai and Paige from 1988. First we give the basic idea of the system, then we give an example of its use. We discuss the complexity and expressiveness of its specification language, and then we review it in terms of the factors that we have previously defined.

The input to RAPTS is a set-theoretic equation of one of the two forms

Least fixed point: **the** $s : w \subseteq s \mid s = f(s)$ **minimizing** s
 Greatest fixed point: **the** $s : s \subseteq w \mid s = f(s)$ **maximizing** s

RAPTS solves the equation to get an inefficient program P computing the least (or greatest) fixed point of s . Then a technique called generalized finite differencing is applied to P . This introduces access points which may trigger various program transformations giving a more efficient P' . Automatic data structure selection is applied to select the most efficient set representation given the set operations occurring in P' .

For example, RAPTS takes the following specification for the nodes in a directed graph e reachable from w :

the $s : w \subseteq s \mid e[s] \subseteq s$

into:

```
proc reachable (e,q)
  u := w
  p := {}
  while exists z in u
    for x in e{z}
      if x notin p
        then if x notin u then u with:= x endif
      endif
    endfor
    u less:= z
    p with:= z
  endwhile
  return w
end proc
```

Now we will discuss the complexity and expressiveness of RAPTS. The specifica-

tion language is called SQ^+ with restriction SQ . SQ expresses the primitive recursive functions, and every equation in SQ is decidable. SQ^+ expresses all partial recursive functions, and the problem of finding a program to compute a finite fixed point is undecidable. The enabling conditions for key program transformation rules over SQ^+ are undecidable. Key conditions are, for example, **inflationary**: $f(x) \geq x$, and **monotone**: if $x \leq y$ then $f(x) \leq f(y)$. Sublanguages of SQ^+ for which these predicates are decidable can, however, be built up by composition of operations for which the properties are decidable, for example cardinality of $s \# s$, or the Cartesian product of s and $t \times s$, where s and t are in monotone or inflationary domains.

In terms of the factors discussed previously,

- RPTS has an operational semantics.
- It is transformational, not axiomatic.
- There is a range of choice of expressiveness.
- The “metamathematically extensibility” concept doesn’t apply because proofs are not kept. Beyond that, one can extend RPTS by adding decision procedures for transformation-enabling properties and for solution of specific classes of fixed point functionals.
- Beyond that, we can think of it as a specification-driven system.

2.3.2 Calculus of Constructions

Now we will consider the Calculus of Constructions of Huet, Coquand and Paulin-Mohring from 1988. We give the basic idea, which springs from the school of Intuitionism, the concept of **realization**, and a review of the system in terms of our factors discussed above.

Intuitionist developments spring from the heuristic that, to construct an object, it suffices in thinking about it to never rely on the assertion “ a or not a ”.

The following interpretations are made for the specification “ $a : A$ ”:

- a is an element of set A
- a is a term of type A
- a is a proof of proposition A
- a is a program for task A
- a is a solution to problem A

So, interpreting A as an element of a specification language, we may identify specifications with data structures and algorithms as follows:

- Given the specification $A \wedge B$, make the Cartesian product $A \times B$. That is, build a ; build b ; and form the pair (a, b) .
- $A \Rightarrow B$ corresponds to the function space $A \rightarrow B$, so we would build a function which given an a yields a b .
- $(\forall a \in A)B(a)$ corresponds to the Cartesian product of a family of types $B(a)$ for $x \in A$. In this case we would build a function which given an a yields a b in $B(a)$. For example, from the integer 3 to describe the type of identity 3-matrices.

A **realization** is a means of translating a proof that an object exists with a certain property into a program for constructing that object. For example:

- (1) Specify what it means to be sorted.
- (2) Prove that a sorted object exists.

- (3) Refine the proof so that it follows the contorsions of, say, BubbleSort.
- (4) Automatically extract BubbleSort as a typed λ -calculus program from the suitably hand-crafted proof.

Certain restrictions of the specification language are always realizable. The realization of objects describable in the general specification language is, however, undecidable.

Finally, for our factor review of the Calculus of Constructions proof system, we can say that it is:

- Denotational.
- Axiomatic.
- There is a choice of fully or restricted expressiveness.
- It is not intended specifically to be to be metamathematically extensible in the sense of Davis and Schwartz, and
- In the sense that we prove the realizability of the specification, this is a verification approach, but then realizing the program from the proof of realizability it is a transformational approach.

2.3.3 Correct Program Technology

Now we will consider the Correct Program Technology initiative of Davis, Schwartz and Deak of around 1977. We describe the central object of this technology, which is **praas**, or “programs with assumptions and assertions”. We then describe the program development process, the verification technology, and finish with a brief review of design factors.

The Correct Program Technology approach adds to the Floyd-Hoare approach of around 1967 of attaching a precondition to a program, and pushing it along towards a

postcondition. We carry pre- and post-conditions together with a program, and allow the conditions to be attached to any place in the program. The result is called a “praa” or “program with assumptions and assertions”. The program language is SETL. Use of an otherwise type-free language of sets is a characteristic of this approach, even though types **à la** SML can provide a very economical (in verification cost) source of assumptions and assertions. The language of assumptions and assertions is first order logic plus Zermelo-Frankel set theory, including transfinite constructs (i.e., unbounded quantification), including especially collections of predicates and operations for which decision procedures exist.

Program development proceeds by:

- (1) Starting with a (possibly transfinite) high-level assumption and a (possibly empty) program.
- (2) Slowly strengthening the program and deriving assumptions until the desired assumptions are obtained for an executable, if inefficient, program.
- (3) Applying correctness-preserving transformations to refine the inefficient program to the desired, more efficient algorithm.

In addition to establishing the correctness of praas **à la** Floyd-Hoare, the plan envisions:

- The construction of a family of correctness-preserving praa program transformations, and that
- That composition of correct praas (in the sense of logical composition defined above) would prove to be much more tractable than the proof of correctness of initial “root” praas.

For verification technology, an eclectic approach to theorem-proving is advocated:

- Decidable sublanguages of set theory.

- Complete term-rewriting rule sets.
- Unification.
- First order logic resolution proof search.

Tempering this, in theory, implementation of disparate methods in the verification engine should be obtained by metamathematical extension from first order logic and Zermelo-Frankel set theory in the strict sense defined by Davis and Schwartz in 1977. However, experimental work has proceeded by just directly programming the various techniques in SETL without verifying formally that stability is preserved in the sense defined by Davis and Schwartz.

In summary, our factor review of the Correct Program Technology initiative, is that

- It endorses an operational semantics.
- It is fully expressive.
- It is transformational but axiomatic.
- It advocates a mixture of verification and verified transformation.
- It is metamathematically extensible, and
- In terms of complexity of program derivation, almost all underlying decidable sublanguages that it relies on are intractable.

Chapter 3

Correct programmer's workbench

We define the idea of a Correct Programmer's Workbench (CPW), proposing requirements for

- The development environment and proof metalanguage needed to create the CPW, and
- The end-user language and programming environment for the deliverable CPW.

As a simple demonstration of the conceptual framework offered by a candidate proof metalanguage, we code a stack datatype in six candidate platforms. We conclude with a brief review of the features of six existing languages as development platforms for the CPW.

3.1 Components of a CPW

First we will describe the deliverable Correct Programmer's Workbench, and then the development environment.

3.1.1 Deliverable CPW

The end-user's version of the Correct Programmer's Workbench consists of a **target language**, a **program proof system**, and a **praa editor**.

The **target language** is typically a restriction of the proof metalanguage in the Developer's Version to be described.

The **proof system** consists of a

- A language of propositions.
- A set of inference rules.
- A set of axioms, and
- A data structure for proofs.

The **praa editor** consists of

- A display of programs, assumptions, assertions and proofs.
- A way of marking places in the display.
- Editing operations, including text editing, inference rule application, and program transformation application.

3.1.2 Development Environment

The development environment would contain, in addition to all of the components in the deliverable version,

- A proof metalanguage.
- A parser generator integrated with the user-defined datatype facility of the proof metalanguage.
- A rule set analyser as in this thesis for analysing the effects of heuristic simplification rules, and
- Scientific visualization methods (graphics) to aid in algorithm and heuristic analysis.

We now present a grab-bag of design factors for a proof metalanguage and its environment. We discuss:

- Basic features
- Execution environment
- Types and modules
- Algebraic environment
- Proof theoretic environment
- Numerical methods environment
- Scientific visualization environment

The basic features that we might expect to find in the proof metalanguage include:

- Fully automatic storage allocation
- Procedures as objects
- Lists, sets, and arrays
- Symbols
- Big integers
- The ability to coerce data to procedures
- Environments as objects and operations on environments, and
- Exceptions

The features of the execution environment of the proof metalanguage that we might expect include:

- Explicit access to execution state
- An interpreter
- An incremental compiler [RI82]
- Garbage collection elimination
- An optimizer
- An automatic data structure selector [SSS81], and
- An interactive data structure inspector

Many, many choices are possible for the type and module facilities of the proof metalanguage, including:

- Overloading
- Pattern-directed invocation as in CAML and Mathematica
- User-defined polymorphic types as in CAML or SML [AM90]
- Structures as in SML, Packages as in Ada or Categories as in Axiom [JS92], and
- Objects with methods as in T [RIM84]

We would expect the algebraic environment of the proof metalanguage to contain polynomials and primitives for mapping over polynomials and other structures, as in Mathematica, and various magic bullets, including:

- Gröbner bases [MY89]
- Symbolic integration
- Real algebraic number arithmetic

- Factoring
- Parameterisation of curves described by implicit equations
- Functional decomposition

and so on.

In the proof theoretic environment of the proof metalanguage, we expect to have available a parser generator as in CAML “mapping concrete syntax to abstract syntax” [WAL⁺87, §8]. In addition, we expect various magic bullets to be available, such as

- The unification algorithm
- Critical-pair completion algorithm
- Quantifier elimination, and
- Integer linear programming

In the proof theoretic environment of the proof metalanguage, we expect to have

- ODE and PDE system numerical solution methods
- Polynomial inequality minimum solution finder, and
- Polynomial equation system solvers

In the scientific visualization environment of the proof metalanguage, we would expect 2- and 3-D graphics to the level of Mathematica or PV-Wave or S+ and animation of displays.

3.2 Stack in six popular languages

3.2.1 Why Stack is a good litmus test

Consider the example E of a stack abstract datatype with stack type S , operations `push`, `pop`, `top` and `isEmpty`, constructor `emptyStack` and exceptions `cantPop`

and `cantTop`. This example, implemented in a given language, tells one what it has for modules, user-defined datatypes, exception handling and procedure declaration. Viewing a language as a potential proof metalanguage, the clarity and informativeness of a clean implementation of E in L is strongly indicative in an intuitive sense of the degree to which proof procedures and data structures implemented in L will be verifiable to the extent necessary to deploy a correct programmer's workbench in L as a production system for mission critical applications. We apply this simple test to six popular symbolic computation languages, and we leave the reader to decide which language most clearly expresses the intent of a stack and the computational outcome of a given stack operation. We assume that readers are familiar with the languages involved; if not, such familiarity is left as an exercise which ought to be undertaken by anybody wishing to produce a verifiable production system. We give the examples in order of progressive sophistication of the language in terms of power and clarity of expression of the module, type and exception content of the example. Three of the languages are computer algebra environments (Maple [ea85], Mathematica [Wol91] and Axiom [JS92]), two are typically used in compiler implementation and the study of algorithms (Scheme [CRea89] and Setl2 [Sny90a, Sny90b]), and one is in a family of languages explicitly intended as metalanguage for the implementation of proof checkers for axiomatic logics (SML [AM90], the example could equally have been done in CAML [WAL⁺87], which, however, has a less developed module facility).

3.2.2 Stack in Scheme, an algorithm language

Scheme [CRea89] doesn't have user-defined datatypes or exceptions, so the expression is purely functional:

```
(define push (lambda (x S) (cons x S)))
(define pop (lambda (S) (if (null? S) 'cantPop (cdr S))))
(define top (lambda (S) (if (null? S) 'cantTop (car S))))
```

```
(define isEmpty (lambda (S) (null? S)))
(define emptyStack '())
```

3.2.3 Stack in Maple, a computer algebra language

Maple [ea85] doesn't have user-defined datatypes or exceptions, so the expression is purely functional:

```
push := proc (x, S) [x] + S end;
pop := proc (S) if S=[] then ERROR("cantPop") else [S[2..]] fi end;
top := proc (S) if S=[] then ERROR("cantTop") else S[1] fi end;
isEmpty := proc (S) S=[] end;
emptyStack := [];
```

3.2.4 Stack in Setl2, an algorithm language

Setl2 doesn't have user-defined datatypes; it has a concept of classes, but prolonged use of the only compiler in existence revealed numerous semantic inconsistencies and bugs [Eri91], and the compiler itself was not available for maintenance or peer review, so we don't consider the use of Setl2 classes here. It doesn't have exceptions. It does have modules, adding a little additional structure to the previous two versions:

```
package Stack;
  const emptyStackpush := [];
  procedure push(x,S);
  procedure pop(S);
  procedure top(S);
  procedure isEmpty(S);
end Stack;

package body Stack;
  procedure push(x,S); return [x]+S; end push;
  procedure pop(S);
```

```

    if S=[] then print("cantPop"); stop;
    else return S[2..];
    endif;
end pop;
procedure top(S);
    if S=[] then print("cantTop"); stop;
    else return S[1];
    endif;
end top;
procedure isEmpty(S); return #S = 0; end top;
end Stack;

```

3.2.5 Stack in Mathematica, a computer algebra language

Mathematica [Wol91] has a way of effectively encoding user-defined types (by following a certain discipline, one attains the effect), and a way of effectively encoding exceptions (with non-local exit). To wit (we use the symbol **Stak** because **Stack** is predefined in Mathematica):

```

Stak/: push[x_,S_Stak] := Join[Stak[x],S]
Stak/: pop[Stak[]] := Throw[cantPop];
Stak/: pop[S:Stak[___]] := Rest[S];
Stak/: top[Stak[]] := Throw[cantTop];
Stak/: top[S:Stak[___]] := S[[1]];
Stak/: isEmpty[Stak[]] := True;
Stak/: isEmpty[Stak[___]] := False;
emptyStack := Stak[]

```

3.2.6 Stack in Axiom, a computer algebra language

Axiom [JS92] has a powerful language of user-defined types, but no exceptions. What is happening here is that we define the “category of all stacks”, that is, of any

datatype with certain operations and values. Then we define a particular datatype (or in Axiom parlance, domain), which is a member of the category of all stacks:

```
Stacks: Category == with
  {operations}
  push: (Expression,$) -> $
  pop: $ -> $
  top: $ -> Expression
  isEmpty: $ -> Boolean
  emptyStack: $

Stack: Stacks with == add
  {define}
  push(x:Expression,S) == listUnion([x],S)
  pop(S) == "cantPop" when #S = 0
  pop(S) == rest(S) when #S > 0
  top(S) == "cantTop" when #S = 0
  top(S) == S(1) when #S > 0
  isEmpty(S) == #S = 0
  emptyStack == []
```

3.2.7 Stack in SML, a logic and type theory language

SML has a strong language of user-defined types and exceptions, and is in some sense perfectly designed for this example, which Appel and MacQueen themselves present [AM90, ch. 11]. (similar code would work in CAML [WAL⁺87], except that it lacks modules). A **structure** in SML is like a Setl2 package, except that it is also, in a certain sense, a first-class object in the language: one can operate on structures (at compile-time only, alas) to produce other structures with functors, whereas Setl2 packages are effectively constants. A **signature** connotes a class of structures, those whose contents (which may be types, exceptions, and values) include the contents partially (but not totally, in the case of values and types) described in the signature:


```

signature STACK =
  sig
    exception cantPop and cantTop
    type 'a stack
    val push: 'a * stack -> stack
    val pop: stack -> stack
    val top: stack -> 'a
    val isEmpty: stack -> bool
    val emptyStack: 'a stack
  end

structure Stack: STACK =
  struct
    exception cantPop and cantTop
    datatype 'a stack = Empty | push of 'a * 'a stack
    val push = Push
    fun pop(Push(v,s)) = s | pop(Empty) = raise cantPop
    fun top(Push(v,s)) = v | v | top(Empty) = raise cantTop
    fun isEmpty(Empty) = true | isEmpty(Push(v,s)) = false
    val emptyStack = Empty
  end

```

3.2.8 Discussion

Let us now focus on the qualities of Setl, Axiom, SML and Mathematica as candidate proof metalanguages.

3.2.8.1 Setl as a proof metalanguage

A problem with Setl as a proof metalanguage is that it makes some assumptions for us which are not at all atomic, and it doesn't give us a linguistic mechanism for

really putting those assumptions “on the table”. Since the assumptions are left hidden, it becomes that much more difficult to reason directly about them.

In particular, consider the notion of set. What does it mean for one set to equal another? Every element of one set is an element of the other, and vice versa. How can we tell? We test each element for equality with another element. What does it mean for two elements to be equal? One approach, taken by Set1, is to define a single heterogeneous-element set representation, and to define equality as representation similarity. But under this definition, $x + 1$ and $1 + x$ may not be similar, and hence, not equal. But we have a notion of equality for polynomials which says that they are equal. What does all this lead to? It means that in constructing “Sets”, we really need to construct “Sets of X , where X is a type with equality”. Further more, if we wish to represent sets as, say, B-trees, then we need “Sets of X , where X is a type with equality and a partial order $\sqsubseteq$ ”, and so on. In other words, there are many kinds of sets, and how we construct them depends strongly on what they contain.

One may object that this precludes the use of sets with elements of heterogeneous types, and that we must also laboriously construct a new set type for every distinct usage. The former objection is not serious, as there are few cases where one really needs a “set of anything”. The latter objection is more serious. There will be many distinct set types in a program, and so the declaration (equivalent to the construction) of every distinct usage will make programming more laborious than in a “type-free” Set1, but the “tower of domains” language of Axiom shows us that this is not an insurmountable obstacle.

3.2.8.2 Axiom as a proof metalanguage

Axiom is a language that has an excellent facility for making just the kind of differentiation between set types that we are discussing. If one wants, say, to declare a variable to hold a matrix of univariate polynomials over Gaussian integers, one just says

“`m: M P G I`”. To get a square matrix of dimension 3 with bivariate polynomials, one says “`m: SM[3] P[x,y] G I`”. But to return to the Sets example, here is the Axiom version:

```
SetElement: Category == with
  {operations}
  "=": ($,$) -> Boolean
Set(E: SetElement): Category == with
  {operations}
  "=": ($,$) -> Boolean
```

ordered sets are then a refinement of sets with equality:

```
OrderedSetElement: Category == SetElement with
  {operations}
  "<": ($,$) -> Boolean
  {attributes}
  irreflexive("<") { not(x<x) }
  transitive("<") { x<y and y<z implies x<z }
  total("<") { x<y or x=y or x>y }
OrderedSet(E: OrderedSetElement): Category == Set(E)
```

and in any domain representing `OrderedSet`, we know we can use a representation which takes advantage of order, such as B-trees.

The only drawback to all of this, what keeps Axiom from being “the” language, is that the language is a little too rich, giving rise to problems:

- (1) It is not easy to formalize the notion of a category in a fashion which is, in a mechanical or even philosophical sense, verifiable. (See [Law66, FK69, Bla84]; for a general introduction to category theory for computer sciences, see [BR83, Hue87, Dyb87, Pie90]).

- (2) The attributes play a role in the semantics of the language. These attributes are unverified atomic propositions (everything in curly brackets `{}` is uninterpreted commentary), but still, the inclusion of even a restricted language of propositions at the level of the user-defined type language may pose problems in defining the formal semantics of the language.

3.2.8.3 SML as a proof metalanguage

Axiom is well suited to the definitional needs of computational algebraists [Lam92]. But the trend in programming language semantics, and for the needs of automated deduction, it is preferable to have a language with semantics which can be stated compactly. We have seen two responses to this need for a compact semantics: Setl posits pure sets as the most perspicuous basis for algorithmic discourse. The ML family of languages posits category-theoretic language as the best basis for computations in pure logic. Unfortunately, SML does not have the ability to:

- Parameterize category descriptions on values and properties of values. So there is no way to express “`SquareMatrix(n)`” as a signature, or to obtain the following effect [Dea84, §7.5]:

The category `Modulus(m)` defines the category associated with integers modulo the positive number m . `Modulus(m)` contains the operations of `FiniteField` only if m is prime:

```
Modulus(m: PositiveInteger): Category ==
  Join(Ring, Finite) with
    if isPrime m then FiniteField
```

- Parameterize category descriptions on category descriptions. That is, there is no way to express the signature “`OrderedSetElement`” as a function of the signature “`SetElement`”.
- Parameterize category descriptions on logical propositions about operations in parameter categories, for example

```
if D has commutative("+": ($,$) -> $) then...
```

A more complete example is the following definition of semigroups, which refines the default implementation of exponentiation variously depending on whether the domain implementing the semigroup belongs to one of several other categories already known to the system [Dea84, §7.6]:

```
SemiGroup: Category == Set with
{operations}  "*": ($,$) -> $
              "**": ($, PositiveInteger) -> $
{attributes}  associative("*") {(x*y)*z = x*(y*z)}
default
RS == RepeatedSquaring($)
x ** n: PositiveInteger == expt(x,n)$RS
if $ has Monoid then
  x ** n: NonNegativeInteger ==
    n = 0 => 1
    expt(x,n)$RS
if $ has Group then
  x ** n: Integer ==
    n = 0 => 1
    n < 0 => expt(inv(x),-n)$RS
    expt(x,n)$RS
if $ has Field then
  x ** n: Integer ==
    x = 0 => 0
    n = 0 -> 1
    n < 0 => expt( recip(x),-n)$RS
    expt(x,n)$RS
```

SML does have a functor concept which will map structures to structures. It does not have a “signature functor” concept to take signatures to signatures. SML objects

may be organized roughly into a hierarchy:

- **Level 0.** Types, exceptions, functions, variables.
- **Level 1.** Structures, signatures, functors.

The parameters of functors are structures with signatures; these are Level 1 values. SML functors do not have Level 0 parameters. “Signature functors” would create a Level 2. Logicians might say that mixing values of different levels could create circularities in the semantics which could lead to paradoxes, so the enterprise might be foundationally hopeless.

3.2.8.4 **Mathematica as a proof metalanguage**

Mathematica has two problems as a proof metalanguage: logical instability and conceptual complexity of the formal semantics.

Logical Instability The instability problem was raised elsewhere, namely that reliance on the logical correctness of symbolic computations can lead to divergent proof outcomes from one version of Mathematica the next. This is contrary to the requirement of proof system stability, and so it means that, to whatever extent we rely on the symbolic computation algorithms in Mathematica during development, all such uses must be replaced by verified algorithms for the same tasks in the production version. This is a real issue, and one which has most likely already been of some concern to others. The designers of Mathematica offer no assurances [Wol91, copyright page]:

The author, Wolfram Research and Addison-Wesley do not recommend the use of the software described in this book for applications in which errors or omissions could threaten life, injury or significant loss.

For applications where the cost of failure is very high, a program verification system used to develop such applications would need to be based on either a “Mil-Spec Mathematica” that came with a more positive recommendation, or on a platform with a simpler

semantics for which, at the minimum, extensive software testing had been performed to verify compliance with that semantics.

Conceptual complexity of the formal semantics Mathematica program execution is rule-based, in the sense that an element of a set of rules, not a single rule, must be selected from in order to determine the next step of execution. The selection of the correct rule is governed by numerous factors, for example:

- **Similarity and order of declaration.** If we declare the similar rules

```
f[x_] := 12;  
f[y_] := 13;
```

which gets selected, for example in:

```
f[1]
```

```
12
```

Evidently the older, not the newer declaration. Is the second forgotten?

```
??f
```

```
Global`f
```

```
f[x_] := 12
```

```
f[y_] := 13
```

No. Will it ever get used? No. But if we subsequently change `f[x_]`, will that get used?

```
f[x_] := 20;
```

```
f[1]
```

20

Yes. So if you change an argument name, but not the level of generality of the argument pattern, what must we do?

```
Clear[f];  
f[y_] := 12;  
f[1]
```

12

We must get rid of all the rules with head `f` and start over again.

- **Generality.** Of the two rules

```
f[x_] := x[[1]] + 5;  
f[{x_}] := x+1;
```

as applied to

```
f[{1}]
```

2

the one declared second will apply because it has a more specific pattern in some lattice of specificity.

- **Up- and down- values.** Normally [Wol91, §2.4.10] when Mathematica evaluates an expression such as `f[g[x]]`, it looks to select a rule for the meaning of `f` in the rule set for symbol `f`, as in (this is called defining a down-value for `f`):

```
f[g[x_]] := 12;  
f[g[a]]
```

12

However, one can also associate the value of `f[g[x]]` to the rule set for symbol `g`, as in (this is called defining an up-value for `g`):

```
g/: f[g[x_]] := 13;  
f[g[a]]
```

13

This introduces an ambiguity: there are two rules that might apply, so which gets chosen? As we can see, the up-value gets chosen.

- **Side conditions.** We can also make the execution of a rule conditional on an arbitrary procedure, for example:

```
h[x_] := 12 /; x > 3;  
h[x_] := 13;  
h[5]
```

12

In this case, the rule selected depends on the value of the arguments as well as the structure of the argument pattern. In addition, the semantics of variable bindings interacts with the side conditions to further complicate the outcome of rule selection [Wol91, §2.5.12]:

Some of the trickiest cases occur when one has rules that depend on complicated `/;` conditions. One particularly awkward case is when the condition involves a global variable. Mathematica may think that the evaluation is finished because the expression did not change. However, a side effect of some other operation could change the value of the global variable, and so should lead to a new result in the evaluation. The best way to avoid this kind of difficulty is not to use global variables in `/;` conditions. If all else fails, you can type `Update[s]` to tell Mathematica to update all expressions involving `s`.

In other words, a formal semantics of Mathematica, in its state model for the execution of Mathematica programs, would effectively need to include the current value of the program text (which changes as changes to the value of global variables are propagated into the program text with explicit invocations of the Update operation and any of its homologues automatically invoked by the evaluation mechanism).

Clearly, then, to select a rule to evaluate a given expression, one must examine the lattice of pattern specificity, up-and-down-ity and side conditions. This is a much more complex mechanism to describe formally than goto, straight procedure call or even method resolution in object-oriented languages.

Beyond the complex behavior of rule selection are the surprises offered by variable-binding rules in Mathematica. In particular, one might assume that constructs such as `Block[{x}, e]` create an unbound assignable local variable x and then evaluate e in that context. We give two examples of `Block` behavior that might surprise a Mathematica novice:

- (1) Bindings in a block become ordinary symbols outside of the block: That is,

```
x = 12;
{Block[{z}, z+1], Block[{x}, x+1]}
```

```
{1 + z, 13}
```

but

```
x = 12;
{Block[{x}, x = 20; x+1], x+1}
```

```
{21, 13}
```

In the first example, the expression “ $x + 1$ ” gets passed out of the block, which is evaluated in the enclosing context in which the symbol x is bound to 12. That

is, the block doesn't really create a distinct symbol which remains distinct even when passed out of the block. Instead, the symbol x which starts out referring to the store maintained within the block then becomes the symbol x referring to the store outside of the block.

- (2) Conversely, unbound variables outside a block become bound variables within the block: That is, suppose we have an expression in which symbol x is unbound, and then we enter into a block in which x is bound. Then the expression will change, even though we might naively expect the “ x ” in the block local variable declaration to refer to a different store abstractly. Consider:

```
g = x + 12;
{g, Block[{x}, x = 13; g]}

{12 + x, 25}
```

The fact that these binding conventions are surprising is not necessarily bad in and of itself. It creates a risk of confusion when it comes to developing correct program technology in Mathematica. In particular, it means that when writing a function which operates on a symbolic structure S , we have to make sure that the local block variables names that we choose do not collide with any symbol in S , otherwise the value of the function will diverge strongly from our expectations when there is such a collision. This means that we have to choose unlikely variable names, like `V111223311xxFoo`. For example, here is what we hope to be a squaring function:

```
Square[x_] := Block[{z}, z = x; z z]
```

This is what we expect `Square` to do:

```
Square[w + 1]
```

2

```
(1 + w)
```

This is what we don't expect **Square** to do:

```
Square[z + 1]
```

```
$RecursionLimit::reclim: Recursion depth of 256 exceeded.
```

```
$Aborted
```

This means that when we go to define the semantics of such a function, we will not be able to formally describe our intentions in a clean fashion, because every such description will need to be preceded with the condition “if the block variables do not collide with the symbolic variables in the argument, then the meaning of the function is...”.

Perhaps in response to these problems, there is a new construct in Version 2 and above of Mathematica, **Module**, which is like **Block** except that it creates true local variables in a unique way: every time a **Module** is entered, a global counter is incremented, and all occurrences of local variables **foo** in the **Module** are replaced with their names with the value of the global counter incremented in the form **foo\$1234**. However, careful reprogramming may be necessary to replace uses of **Block** with uses of **Module**. For example:

```
foo[x_] := Block[{E}, E = E/.{x}];  
foo[E -> 12]
```

```
12
```

is not the same as:

```
bar[x_] := Module[{E}, E = E/.{x}];  
bar[E -> 12]
```

```
E$703
```

Finally, then, the problem of semantic complexity is that if the intuitive description of the meaning of programs in a language is exceedingly complicated, then it is

unlikely that its formal semantics will be less so. So if the formal semantics of the proof metalanguage (in this case, Mathematica) is exceedingly complicated, then so will be the verification of any procedure written in it. If the metalanguage is unverifiable in this sense, then it is quite unclear how we could expect to deliver a mission-critical production correct programmer’s workbench using a Mathematica platform. This ultimately limits our ability to utilize Mathematica for long-term development (but this should not be an problem for near-term engineering experiments).

3.2.8.5 Conclusions

Mathematica is a strong, easy-to-use and economical tool, which however is semantically complex and unstable. Axiom allows us to say exactly what we mean, and say it in just the right way. But we are not confident that there is a clean and compact formal semantics for Axiom which would lend itself to some level of “self-verification” of the resulting correct programmer’s workbench. The ideal language would incorporate the features of Axiom in a sparse extension of SML in a foundationally sound fashion.

3.3 Factor review of six candidate development platforms

Now we are going to do a factor review of six candidate proof metalanguages discussed above. Our comparisons are made in terms of presently available facilities:

- In terms of types and modules, Scheme, Maple and Setl are fairly weak, Mathematica is good, and Axiom and SML are excellent.
- In terms of algebraic environment, Scheme, Setl and SML do not have a lot of available software, Maple of course is strong, and Mathematica and Axiom are very strong.
- In terms of proof theoretic algorithms, Scheme, Maple, Mathematica and Axiom do not have a lot of available software, and Setl and SML have a fair amount

Factor/Lang	Scheme	Maple	Setl	Math	Axiom	SML
Types and Modules	—	—	—	+	++	++
Algebraic Env	—	+	—	++	++	—
Proof Theoretic Env	—	—	+	—	—	+
Numeric Env	—	+	—	+	+	—
Graphics Env	—	+	—	++	+	—

Figure 3.1: Factor comparison of six candidate proof metalanguagæ

of code available.

- In terms of the numerical methods environment, Scheme, Setl and SML do not have a lot available; Maple, Mathematica and Axiom are stronger.
- Finally, in terms of scientific visualization methods, Scheme Setl and SML do not have anything to offer. Maple and Axiom have reasonable facilities, and Mathematica has excellent facilities.

These features are summarized in Figure 3.1.

Chapter 4

Correct program editor

This chapter presents the data structures that we have implemented in Mathematica for a correct program editor. We proceed as follows:

- We present the target programming language and its semantics.
- We define the data structures of the editor which allow the user to take correct praas to correct praas by a series of program verification steps. These steps apply **praa proof rules** and **praa transformation rules**.
- We demonstrate prototype correct program editor operations carried out in our package *Gedanken* written in Mathematica.

4.1 Praa programming language

We present an **example** programming for the correct program editor. It has certain minimum elements needed to carry out the Davis-Schwartz plan:

- The ability to mark places in a program.
- The presence of a nondeterministic choice operator for set-valued expressions.
- An escape to Mathematica to make available a wider universe of expressions.

This is because they stipulate that expressions should include [Sch77, §2a] “all sets whether finite or infinite, all mappings, plus integers and the set of all

integers in their ordinary set-theoretic sense”, and relations on these objects. For purposes of specification “We are even happy to admit transfinite ordinals and cardinals, etc...we will feel free to write *onetoone*(s, t) for the set of all one-to-one mappings from the set s into the set t , etc.” In a finished system this would obviously have to be formalised. For purposes of experimentation, we just allow the escape to Mathematica, after replacement under the current assignment.

Beyond that, we replace the language of discourse used in the Davis-Schwartz literature, a statement-oriented language with `goto` and assignment, with an ML-like language with more sophisticated binding constructs and `while`-loops. We give an operational semantics in Mathematica for this language which otherwise follows the specifications of Davis and Schwartz. Their `goto`-oriented rules no longer fit this language, however, so, while retaining the overall structure of placed propositions and claims that they define, we otherwise adopt the proof rules presented by de Bakker [dB80]. We have not established that the system of proof and transformation rules we present is complete; we define it only to show how a set of proof and transformation rules fits into the overall editor data structure design. Many systems of complete proof rules are available in the literature, and we anticipate that a good editor design will allow the eventual “correct programmer” to choose among several proof systems and programming languages. We summarize the types and operations surrounding the programming language and its operational semantics in Figure 4.1.

Let E be the type of programs. An **assignment** is a function $\sigma : Id \rightarrow Sets$ from identifiers to values (recall that we have defined our set of values rather broadly as sets constructable in Zermelo-Frankl set theory). We extend assignments by structural induction on E so that the effect of $\sigma(e)$ for $e \in E$ is to replace occurrences of free variables i in E by $\sigma(i)$, where by “free” we mean the usual notion from logic and

Object	Description
E	Type of programs
Id	Type of identifiers
$Sets$	Type of Zermelo-Frankel sets
$\perp$	The undefined set
$A = Id \rightarrow Sets$	Type of assignments
$A = Id \rightarrow E$	Assignments extended to operate on programs
$\emptyset$	The empty assignment
$S = E \times A \times Sets$	Type of program execution states
$S^* = \bigcup_{n=0}^{\infty} S^n$	Type of computations
$s_{\text{initial}} = \langle \{\emptyset\}, \perp \rangle$	The initial state
$M : E \rightarrow S \rightarrow 2^{S^*}$	The operational semantics

Figure 4.1: Context of operational semantics

programming language theory. Let A denote the set of all assignments. Write $\emptyset$ for the empty assignment. Define an **execution state** as a pair $\langle E, \sigma, \nu \rangle$ where $\sigma = \langle \sigma_i \rangle_{i=1}^n$ is a push-down stack of assignments σ_i and ν is the “current value”, that is, the value of the most recently evaluated expression, and E is the expression examined by the underlying machine in this state. Write $S = E \times A \times Sets$ for the type of states. Let the **initial state** s_{initial} be $\langle \text{bot}, \{\emptyset\}, \perp \rangle$, where $\perp$ is the **undefined value**.

Define a **computation** as a sequence of states that the program may pass through starting from the initial state. We write $S^* = \bigcup_{n=0}^{\infty} S^n$ for the set of all computations. Our language has a nondeterministic choice operator, so the **meaning** of any expression is a set of computations. Let 2^T denote the power set of set T , that is, the set of all choices of subsets of set T . We thus define the **operational semantics** of a program E starting in state s as a **meaning function** $M(E)(s)$, with type $M : E \rightarrow S \rightarrow 2^{S^*}$, by structural induction on programs. More generally, by a computation C for a program P , we mean some segment of an element C' of $M(P)(s_{\text{initial}})$, obtained by dropping 0 or more elements from the front and back of C' , such that the number of elements of the assignment stack is equal for the first and last elements of C . Let s be the first (last) element of C . Then we say that C is a computation from (to) s .

Notation	Gedanken	Semantics
ZF set expr	e [any Math expr]	L is strong
$\perp$	bot	Undefined value
$i : E$	place _{i} [E]	Label place E
if E_1 then E_2 else E_3	if [E_1, E_2, E_3]	Conditional
ηE	choose [E]	Nondeterministic choice
$i = E$	bind [i, E]	Variable binding
i	i	Variable
while E_1 do E_2	while [E_1, E_2]	Loop
let B in E	let [B, E]	Bind B in E
$E_1; E_2$	seq [$E_1, \dots$]	Evaluate E_1 then E_2
B_1 and $\dots$	par [$B_1, \dots$]	Parallel eval
$i_0(i_1, \dots) = E$	bind [fun [$i_0, \{i_1, \dots\}$], E]	Bind function
$i(E_1, \dots)$	call [$i, \{E_1, \dots\}$]	Function call

Figure 4.2: Praa programming language

Our example programming language is a language of expressions E . Figure 4.2 gives the concrete syntax, abstract syntax, and a hint of the semantics in English of each expression. The operational semantics for this language, together with examples of application of the meaning function in **Gedanken**, is as follows:

- $\mathbf{e}$ [any Mathematica expression x]. Let m be the value of x as the result of evaluation by Mathematica. Then

$$M(\mathbf{e}[x])(\langle E, \sigma, \nu \rangle) = \{ \langle \langle E, \sigma, \nu \rangle, \langle \mathbf{e}[x], \sigma, \sigma_1(x) \rangle \rangle \}$$

where $\sigma_1(x)$ denotes the substitution of e for v in x where $(v, e) \in \sigma_1$. For example, in **Gedanken**:

`M[e[12]][initial]`

`{{state[bot, {}], bot}, state[e[12], {}], 12}}`

- **bot**. This is the undefined value, with semantics

$$M(\mathbf{bot})(\langle E, \sigma, \nu \rangle) = \{ \langle \langle E, \sigma, \nu \rangle, \langle \perp, \sigma, \perp \rangle \rangle \}$$

For example, in **Gedanken**:

`M[bot][initial]`

`{{state[bot, {}], bot}, state[bot, {}], bot}}`

- $\mathbf{place}_i[E]$. This just labels an expression for verification purposes. The semantics is just

$$M(\mathbf{place}_i[E])(s) = \{ \langle s \rangle; y : y \in M(E)(s) \}$$

For example, in **Gedanken**:

`M[place[x, e[12]]][initial]`

```

{{state[bot, {}], bot},
 state[place[x, e[12]], {}], bot},
 state[e[12], {}], 12]]}

```

- **if** $[E_1, E_2, E_3]$. This is the usual conditional form. The semantics is

$$M(\mathbf{if}[E_1, E_2, E_3])(\langle E, \sigma, \nu \rangle) = \text{nexteval}(\text{cond}(E_2, E_3))(\langle \mathbf{if}[E_1, E_2, E_3], \sigma, \nu \rangle, E_1)$$

where

$$\text{nexteval}(f)(s, E) = \bigcup \{ \text{enchain}(f) : f \in M(E)(s) \}$$

and

$$\begin{cases} \text{enchain}(f)(\langle s_1 \rangle) &= f(s) \\ \text{enchain}(f)(\langle s_1, s_2, \dots, s_n \rangle) &= \{ \langle s_1, \dots, s_{n-1} \rangle; \xi : \xi \in f(s_n) \} \end{cases}$$

where $S;T$ denotes the concatenation of sequences S and T and

$$\text{cond}(E_2, E_3)(\langle E, \sigma, \nu \rangle) = \begin{cases} M(E_2)(\langle E, \sigma, \nu \rangle) & \text{if } \nu = \mathbf{True} \\ M(E_3)(\langle E, \sigma, \nu \rangle) & \text{if } \nu = \mathbf{False} \\ \{ \langle \perp, \sigma, \perp \rangle \} & \text{otherwise} \end{cases}$$

For example, in Gedanken:

```

M[if[e[True], e[3], e[4]]][initial]

```

```

{{state[if[e[True], e[3], e[4]], {}], bot},
 state[e[True], {}], True},
 state[e[3], {}], 3]]}

```

- **choose** $[E]$. This chooses one element nondeterministically from set-valued expression E . The semantics is

$$M(\mathbf{choose}[E])(s) = \text{nexteval}(\text{mchoose}(\mathbf{choose}[E]))(s, E)$$

where

$$\text{mchoose}(z)(\langle\sigma, \nu\rangle) = \{\langle\langle z, \sigma, x \rangle\rangle : x \in \nu\}$$

For example, in Gedanken:

`M[if[e[True], choose[e[{1,2}]], e[4]]][initial]`

```
{{state[if[e[True], choose[e[{1, 2}]], e[4]], {}}, bot],
  state[e[True], {}], True],
  state[choose[e[{1, 2}]], {}], 1}},
{state[if[e[True], choose[e[{1, 2}]], e[4]], {}}, bot],
  state[e[True], {}], True],
  state[choose[e[{1, 2}]], {}], 2}}}
```

- `bind[i, E]`. Binds variable i to the value of expression E in the current assignment. The semantics is

$$M(\text{bind}[i, E])(s) = \text{nexteval}(\text{mbind}(\text{bind}[i, E], i))(s, E)$$

where

$$\text{mbind}(z, i)(\langle\langle\sigma_j\rangle_{j=1}^n, \nu\rangle) = \langle\langle z, \sigma_1[\nu/i]; \langle\sigma_j\rangle_{j=2}^n, \nu\rangle$$

where “;” denotes concatenation of sequences and $a[e/i]$ denotes assignment a updated to assign value e to variable i . For example, in Gedanken:

`M[bind[x, e[12]]][initial]`

```
{{state[bot, {}], bot],
  state[bind[x, e[12]], {x -> 12}], 12}}}
```

- i a variable. This is the value of i under the current assignment, or $\perp$ otherwise.

The semantics is:

$$M(i)(\langle E, \sigma, \nu \rangle) = \{\langle\langle E, \sigma, \nu \rangle, \langle i, \sigma, \text{lookup}(\sigma_1, i) \rangle\rangle\}$$

where

$$\text{lookup}(a, i) = \begin{cases} e & \text{if } \exists (i, e) \in a \\ \perp & \text{if } \neg \exists (i, e) \in a \end{cases}$$

For example, in Gedanken:

`M[seq[bind[x, e[12]], e[x+1]]][initial]`

`{{state[bot, {}], bot},
state[bind[x, e[12]], {{x -> 12}}, 12],
state[e[1 + x], {{x -> 12}}, 13]}}`

- `let[B, E]`. Places a copy of the current assignment on the stack, evaluates bindings B , evaluates E in the assignment constructed by B to get value ν , then pops the stack. The precise semantics is:

$$\begin{aligned} M(\text{let}[B, E])(s) = \\ \bigcup \{ \text{enchain}(\\ \{ \langle \text{popAS}(x) \rangle \} : \\ x \in \text{nexteval}(M(\text{mlet}[E]))(\text{dupAS}(s), B) \} \} \end{aligned}$$

where

$$\text{popAS}(\langle E, \langle \sigma_j \rangle_{j=1}^n, \nu \rangle) = \langle E, \langle \sigma_j \rangle_{j=2}^n, \nu \rangle$$

and

$$\text{dupAS}(\langle E, \langle \sigma_j \rangle_{j=1}^n, \nu \rangle) = \langle E, \langle \sigma_1 \rangle; \langle \sigma_j \rangle_{j=1}^n, \nu \rangle$$

and

$$M(\text{mlet}[E])(\langle E, \sigma, \nu \rangle) = \{ \langle \langle \text{mlet}[E], \sigma, \nu \rangle \rangle; y : y \in M(E)(s) \}$$

For example, in Gedanken:

`M[seq[bind[y, e[3]],
let[seq[bind[y, e[4]], bind[x, y]], x]]][
initial]`

```

{{state[bot, {}], bot},
 state[bind[y, e[3]], {{y -> 3}, {y -> 3}}, 3],
 state[bind[y, e[4]], {{y -> 4}, {y -> 3}}, 4],
 state[bind[x, y], {{y -> 4, x -> 4}, {y -> 3}}, 4],
 state[mlet[x], {{y -> 4, x -> 4}, {y -> 3}}, 4],
 state[x, {{y -> 3}}, 4]}}

```

- **while** $[E_1, E_2]$. Evaluates E_1 and then E_2 in the current assignment until E_1 is **False** or $\perp$. The semantics is:

$$M(\text{while}[E_1, E_2])(s) = \text{nexteval}(\text{mwhile}(E_1, E_2))(s, E_1)$$

where $\text{mwhile}(E_1, E_2)(\langle F, \sigma, \nu \rangle)$ equals

$$\begin{cases} \text{nexteval}(M(\text{while}[E_1, E_2])(\langle F, \sigma, \nu \rangle, E_1)) & \text{if } \nu = \text{True} \\ \langle \langle F, \sigma, \nu \rangle \rangle & \text{if } \nu = \text{False} \\ \langle \langle \text{while}[E_1, E_2], \sigma, \perp \rangle \rangle & \text{otherwise} \end{cases}$$

For example, in Gedanken:

```

M[seq[bind[x, e[3]],
  while[e[x > 0],
    bind[x, e[x-1]]]]][
  initial]

```

```

{{state[bot, {}], bot},
 state[bind[x, e[3]], {{x -> 3}}, 3],
 state[e[x > 0], {{x -> 3}}, True],
 state[bind[x, e[-1 + x]], {{x -> 2}}, 2],
 state[e[x > 0], {{x -> 2}}, True],
 state[bind[x, e[-1 + x]], {{x -> 1}}, 1],
 state[e[x > 0], {{x -> 1}}, True],

```

```

state[bind[x, e[-1 + x]], {{x -> 0}}, 0],
state[e[x > 0], {{x -> 0}}, False]]}

```

- $\text{seq}[E_1, E_2, \dots]$. Evaluates the E_i in sequence, allowing the side-effects on the current assignment of E_i to propagate to the evaluation of E_{i+1} . The semantics

is:

$$\begin{cases} M(\text{seq}[E_1])(s) = M(E_1)(s) \\ M(\text{seq}[E_1, E_2, \dots])(s) = \text{nexteval}(M(\text{seq}[F]))(s, E) \end{cases}$$

For example, in Gedanken:

```

M[seq[bind[y, e[3]], let[bind[x,y], x]]][initial]

```

```

{{state[bot, {}], bot},
 state[bind[y, e[3]], {{y -> 3}, {y -> 3}}, 3],
 state[bind[x, y], {{y -> 3, x -> 3}, {y -> 3}}, 3],
 state[mlet[x], {{y -> 3, x -> 3}, {y -> 3}}, 3],
 state[x, {{y -> 3}}, 3]]}

```

- $\text{par}[E_1, E_2, \dots]$. Evaluates E_i in parallel with separate copies of the current assignment, and then merges the top assignment of each into one assignment, using the remaining stack of one of them as the final remaining stack for the final state. The execution sequence of all but the final state is the concatenation of the individual execution sequences. If new variables are defined in more than one E_i , some one of them takes precedence. The semantics is:

$$M(\text{par}[E_1, E_2, \dots, E_n])(s) = \{\text{parmerge}(\text{par}[E_1, E_2, \dots, E_n])(x) : x \in \times_{i=1}^n M(E_i)(s)\}$$

where

$$\text{parmerge}(z)(A) = (; A); \langle \text{merge}(z)(\{\text{last}(a) : a \in A\}) \rangle$$

where $\text{last}(\langle a_1, \dots, a_k \rangle) = a_k$ and $\text{merge}(z)(s) = s$ and

$$\begin{aligned} \text{merge}(z)(\langle \langle \sigma_{1,i} \rangle_{i=1}^{k_1}, \nu_1 \rangle, \langle z, \langle \sigma_{2,i} \rangle_{i=1}^{k_2}, \nu_2 \rangle) = \\ \langle \text{coalesce}(\sigma_{1,1}, \sigma_{2,1}); \langle \sigma_{1,i} \rangle_{i=2}^{k_1}, \nu \rangle \end{aligned}$$

and

$$\begin{aligned} \text{merge}(z)(s_1, s_2, \dots, s_n) = \\ \text{merge}(z)(\text{merge}(z)(s_1, s_2), s_3, \dots, s_n) \end{aligned}$$

and

$$\begin{aligned} \text{coalesce}(\emptyset, \sigma) &= \sigma \\ \text{coalesce}(\{i \rightarrow v\}, \sigma) &= \sigma[v/i] \\ \text{coalesce}(\{i \rightarrow v\} \cup \sigma', \sigma) &= \text{coalesce}(\sigma', \sigma[v/i]) \end{aligned}$$

For example, in Gedanken:

```
M[seq[bind[y, e[3]],
    let[par[bind[y, e[4]], bind[x, y]], x]] [
  initial]

{{state[bot, {}], bot},
 state[bind[y, e[3]], {{y -> 3}, {y -> 3}}, 3],
 state[bind[y, e[4]], {{y -> 4}, {y -> 3}}, 4],
 state[bind[y, e[3]], {{y -> 3}, {y -> 3}}, 3],
 state[bind[x, y], {{y -> 3, x -> 3}, {y -> 3}}, 3],
 state[par[bind[y, e[4]], bind[x, y]],
   {{y -> 4, x -> 3}, {y -> 3}},
  4],
 state[mlet[x], {{y -> 4, x -> 3}, {y -> 3}}, 4],
 state[x, {{y -> 3}}, 3]]
```

- **bind[fun[$i_0, \{i_1, \dots\}$], E].** Binds variable i_0 to function $f(i_1, \dots) = E$. Since this language is a simple one for purposes of exposition, the only free variables in E we allow are the parameters $(i_1, \dots)$. The semantics is, where $s = \langle \langle \sigma_j \rangle_{j=1}^n, \nu \rangle$,

$$M(\text{bind}[\text{fun}[i_0, \{i_1, i_2, \dots\}], E])(s) = \\ \{\langle s, \langle \text{bind}[\text{fun}[i_0, \{i_1, i_2, \dots\}], E], \langle \sigma_1[f/i]; \langle \sigma_j \rangle_{j=2}^n, f \rangle \rangle\}$$

For example, in Gedanken:

`M[bind[fun[Plus, {a, b}], e[a+b]]][initial]`

`{{state[bot, {}], bot},
state[bind[fun[Plus, {a, b}], e[a + b]],
{{Plus -> bind[fun[Plus, {a, b}], e[a + b]]}},
bind[fun[Plus, {a, b}], e[a + b]]}}`

- `call[i, {E1, E2, ..., En}]`. Calls function i bound in the current assignment with actual parameters $E_1, E_2, \dots, E_n$. If the formal parameters of i are $f_1, \dots, f_n$ and the body is E , we just form the parallel bind of f_i to E_i and then evaluate E in the resulting environment. The precise semantics is, where $z = \text{call}[i, \{E_1, E_2, \dots, E_n\}]$,

$$M(z)(\langle F, \sigma, \nu \rangle) = \text{funcall}(z, \sigma_1(i), \langle F, \sigma, \nu \rangle, \{E_1, E_2, \dots, E_n\})$$

where

$$\text{funcall}(z, \text{bind}[\text{fun}[i_0, \{i_1, \dots, i_n\}], E], \langle F, \sigma, \nu \rangle, \{E_1, \dots, E_n\}) = \\ \{\langle \langle F, \sigma, \nu \rangle \rangle; y : \\ y \in M(\text{let}[\text{par}_{j=1}^n i_j = E_j, E])(\langle z, \sigma, \nu \rangle)\}$$

For example, in Gedanken:

`M[seq[bind[fun[Add, {a, b}], e[a+b]],
call[Add, {e[3], e[4]}]]][initial]`

`{{state[bot, {}], bot},`

```

state[bind[fun[Add, {a, b}], e[a + b]],
  {{Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  bind[fun[Add, {a, b}], e[a + b]]],
state[call[Add, {e[3], e[4]}],
  {{Add -> bind[fun[Add, {a, b}], e[a + b]]},
   {Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  bind[fun[Add, {a, b}], e[a + b]]],
state[bind[a, e[3]],
  {{Add -> bind[fun[Add, {a, b}], e[a + b]], a -> 3},
   {Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  3],
state[call[Add, {e[3], e[4]}],
  {{Add -> bind[fun[Add, {a, b}], e[a + b]]},
   {Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  bind[fun[Add, {a, b}], e[a + b]]],
state[bind[b, e[4]],
  {{Add -> bind[fun[Add, {a, b}], e[a + b]], b -> 4},
   {Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  4],
state[par[bind[a, e[3]], bind[b, e[4]]],
  {{Add -> bind[fun[Add, {a, b}],
    e[a + b]], b -> 4, a -> 3},
   {Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  3],
state[mlet[e[a + b]],
  {{Add -> bind[fun[Add, {a, b}],
    e[a + b]], b -> 4, a -> 3},
   {Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  3],
state[e[a + b],

```

```

    {{Add -> bind[fun[Add, {a, b}], e[a + b]]}},
  7]}}

```

4.2 Editor structures

We now define the data structures needed to represent a correct program which is evolving under a sequence of “editing” operations, which apply program proof and transformation rules to develop a default, trivial program into a particular, more complex program which fulfills the specification for a particular problem. In brief,

- (1) A praa proof of correctness takes a praa and takes **claim proofs** to claim proofs.
- (2) A claim proof is comprised of **claim proof steps**. A claim proof step records the application of a **claim proof rule** to claims earlier within a claim proof.
- (3) A **program with assumptions and assertions** is a program with assumptions and assertions, which are placed propositions. Praa **transformations** take correct praas to correct praas.
- (4) A **claim** is a theorem about a praa **computation**, saying that given some **propositions** (assumptions) holding at some **places** in the program, if execution reaches some other places in the program, then some other propositions (assertions) will hold at those places.
- (5) A **placed proposition** is just a proposition at a place.

4.2.1 Placed propositions and satisfied computations

A **placed proposition** is a proposition at a place. In this section we define the Gedanken notations with respect to placed propositions shown in Figure 4.3.

Let a **proposition** ϕ on a program E be a sentence of the form:

- $x \in L$, where x is a variable and L describes a set. In **Gedanken**, we write this as $\text{InSet}[x, L]$. For example,

Notation	Gedanken	Semantics
ϕ	Any Mathematica expr	Proposition
$x \in L$	<code>InSet</code> [x, L]	Set membership
$s \subseteq t$	<code>SubsetEq</code> [s, t]	Set containment
$\phi \Rightarrow \psi$	<code>Imply</code> [ϕ, ψ]	Implication
$(\exists x) \phi$	<code>Exists</code> [x, ϕ]	Existential quantifier
ϕ^i	<code>At</code> [ϕ, i]	Proposition ϕ at place i
ϕ^f	<code>At</code> [$\phi, \text{fun}[f]$]	Proposition ϕ at function f
E_π	<code>GetPlace</code> [E][π]	The expr at place π
$E[s/\pi]$	<code>ReplacePlace</code> [E][π, s]	Replace expr
$E[\epsilon/\pi]$	<code>DeletePlace</code> [E][π]	Delete expr
$\text{occurs}(E)$	<code>Occurs</code> [E]	Free variables
$\vdash \phi$	<code>VerifyQ</code> [ϕ]	Verify theorem
$\models_s \phi^\pi$	<code>SatisfiesQ</code> [<code>At</code> [ϕ, π]][S]	Satisfiable at state
$\models_C \phi^pi$	<code>SatisfiesQ</code> [ϕ^π][C]	Satisfiable at computation
$\models_C E$	<code>SatisfiesQ</code> [E][S]	Satisfiable proposition set

Figure 4.3: Placed propositions

`InSet[x, L]`

`x in L`

- $s \subseteq t$, where s and t are set-valued. In **Gedanken**, we write this as `SubsetEq[s, t]`.
- An expression in mathematics of the form loosely circumscribed by Davis and Schwartz as cited above. For purposes of experimentation, we allow this to be any Mathematica expression. For example,

`x == 12`

- $\phi \Rightarrow \psi$, where ϕ and ψ are propositions. This is written in **Gedanken** as `Imply[ $\phi$ ,  $\psi$ ]`. The following simplifications happen without the recording of a separate proof step:

`True  $\Rightarrow$   $x \rightarrow x$`

`False  $\Rightarrow$   $x \rightarrow$  True`

`$x \Rightarrow$  True  $\rightarrow$  True`

- $(\exists x) \phi$, where ϕ is a proposition. If x is not free in ϕ , this automatically simplifies to ϕ alone. We write this in **Gedanken** as `Exists[x,  $\phi$ ]`. For example,

`Exists[y, x+y == 12]`

`[Exists y] (x + y == 12)`

As a result of our experimenting within Mathematica, certain forms will be simplified without our having proved the simplification formally through the application of a series of syntactic proof rules. This should be seen as a convenience, not a hindrance; pure proof behavior can easily be implemented by changing the definitions of the underlying propositional connectives and quantifiers. The following additional simplifications happen without the recording of a separate proof step:

$\mathbf{True} \wedge x \rightarrow \mathbf{True}$	$\mathbf{False} \wedge x \rightarrow \mathbf{False}$
$\mathbf{True} \vee x \rightarrow \mathbf{True}$	$\mathbf{False} \vee x \rightarrow x$
$a \wedge a \rightarrow a$	$a \vee a \rightarrow a$

A **place** π in program E is either:

- An identifier labelling a **place** expression. For example,

`A`

is a place in

`let[bind[y, e[12]], place[A, e[x + y]]]`

- The name f of a function declared in the outermost scope. For example,

`fun[Add]`

is a place in

`let[bind[fun[Add, {a, b}], e[a + b]], Add[e[1], e[2]]]`

We may operate syntactically on programs with the following operations, which are defined by structural induction on programs:

- The notation E_π , or in Gedanken `GetPlace[E][ $\pi$ ]`, which returns the expression at place π in E , or **Fail** if there is no place marked π in E .
- The notation $E[s/\pi]$, or in Gedanken `ReplacePlace[E][ $\pi$ ,  $s$ ]`, means “the program which results from replacing the at place π with expression e in program E ”. We only define this for places which are identified by **place** expressions. For example,

`ReplacePlace[let[bind[y, e[12]], place[A, e[x+y]]][A, z]`

`let[bind[y, e[12]], z]`

- The notation $E[\perp/\pi]$, or in **Gedanken**

DeletePlace $[E][\pi]$

means “delete the expression at place π in program E ”. This is just replacement of the expression at the place by the undefined expression $\perp$. Because expressions in our language may contain side-effects (assignments to the current assignment), we cannot be more aggressive than this, for example, we cannot (at this stage, without verification), replace **let** $[B, \perp]$ with $\perp$, since an expression in B could contain a side effect. For example, B could be **bind** $[\mathbf{x}, \mathbf{bind}[\mathbf{y}, \mathbf{e}[13]]]$, which has the effect of modifying y in the outer scope while binding x in $\perp$.

Let E be a program as above. Let the **Gedanken** form **Occurs** $[E]$ be the occurrences of free variables in E ; non-free or bound variables are those function parameters that occur in the scope of a function body or as bindings in a **let**. We define occurs on programs by structural induction on expressions. For example,

Occurs $[\mathbf{let}[\mathbf{bind}[\mathbf{y}, \mathbf{e}[12]], \mathbf{place}[\mathbf{A}, \mathbf{e}[\mathbf{x}+\mathbf{y}]]]]$

$\{\mathbf{x}\}$

A placed proposition ϕ^π , written in **Gedanken** as **At** $[\phi, \pi]$, is a proposition ϕ such that

- If π is a label for a **place** expression **place** $[i, E]$, then ϕ has as free variables only free variables of E . For example,

At $[\mathbf{x} > 0, \mathbf{B}]$

is a well-formed placed proposition for program

place $[\mathbf{B}, \mathbf{let}[\mathbf{bind}[\mathbf{y}, \mathbf{e}[12]], \mathbf{place}[\mathbf{A}, \mathbf{e}[\mathbf{x}+\mathbf{y}]]]]$

- If $\pi = \text{fun}[f]$, a function named declared in P , then ϕ has as free variables only the free variables of P plus the parameters $y_1, \dots, y_m$ of f , plus a special variable $\text{Ret}[f]$ to denote the return value of f . For example,

```
ImPLY[At[NumberQ[a] && NumberQ[b], Ret[Add] == a + b, Add]
```

is a well-formed placed proposition for program

```
let[bind[fun[Add, {a,b}], e[a+b]], Add[e[1], e[2]]]
```

Let us suppose that we have a theorem prover $\text{VerifyQ}[\phi]$ which returns **True** if there exists a proof in first order logic plus set theory of proposition ϕ , **False** otherwise. For purposes of experimentation, we just define this in **Gedanken** to be “simplifies to **True** using Mathematica’s **Simplify** operation”. For example,

```
VerifyQ[Cos[Pi] < 0]
```

```
True
```

Let E be a program. Let $s = \langle F, \sigma, \nu \rangle$ be a state of a computation C of E , and let ϕ^π be a placed proposition. Then:

- If π is an identifier, then
 - (1) If $F = \text{place}[\pi, G]$ then s satisfies ϕ^π if ϕ is true under σ_1 , that is, if $\text{VerifyQ}[\sigma_1(\phi)] = \text{True}$, and s trivially satisfies ϕ^π otherwise. We write this in **Gedanken** as $\text{SatisfiesQ}[\text{At}[\phi, \pi]][S]$. For example,

```
C1 =
```

```
M[seq[bind[x, e[1]],
      while[place[A, e[x > 0]],
            bind[x, e[x-1]]]]][initial][[1]]
```

```
{state[bot, {{}}, bot],
 state[bind[x, e[1]], {{x -> 1}}, 1],
```

```

state[place[A, e[x > 0]], {{x -> 1}}, 1],
state[e[x > 0], {{x -> 1}}, True],
state[bind[x, e[-1 + x]], {{x -> 0}}, 0],
state[place[A, e[x > 0]], {{x -> 0}}, 0],
state[e[x > 0], {{x -> 0}}, False]]

```

```
SatisfiesQ[At[x > 0,A]] [C1[[2]]]
```

True

```
SatisfiesQ[At[x < 0,A]] [C1[[3]]]
```

False

- (2) The computation C satisfies ϕ^π if every s in C satisfies ϕ^π . We write this in Gedanken as `SatisfiesQ[ $\phi^\pi$ ][ $C$ ]`. For example,

```
SatisfiesQ[At[x >= 0,A]] [C1]
```

True

- If $\pi = \text{fun}[f]$ and

- (1) f is a function bound in σ_1 with parameters $y_1, \dots, y_m$, and

- (2) $\phi = \phi(\text{Ret}[f], a_1, \dots, a_m)$

then C satisfies ϕ^π if

- (1) For each first state

$$s = \langle F, \sigma, \nu \rangle$$

of C such that F is

```
call[f, {e1, ..., em}]
```

and

- (2) The depth of σ is m
- (3) There exists a first state s'' following s with assignment stack binding depth $< m$, and
- (4) There exists a first state $s' = \langle F', \sigma', \nu' \rangle$ following s with $F' = \text{mlet}[F'']$, and
- (5) It is the case that

$$\text{VerifyQ}[\phi(\nu', \sigma'_1(e_1), \dots, \sigma'_1(e_m))] = \text{True}$$

We write this in Gedanken as $\text{SatisfiesQ}[\phi^\pi][s]$. For example,

```

C2 = M[let[bind[fun[Inc, {a}], e[a+1]],
        call[Inc, {e[3]}]]] [
    initial]

{{state[bot, {}, {}], bot],
 state[bind[fun[Inc, {a}], e[1 + a]],
  {{Inc -> bind[fun[Inc, {a}], e[1 + a]]}, {}},
  bind[fun[Inc, {a}], e[1 + a]]],
 state[mlet[call[Inc, {e[3]}]],
  {{Inc -> bind[fun[Inc, {a}], e[1 + a]]}, {}},
  bind[fun[Inc, {a}], e[1 + a]]],
 state[call[Inc, {e[3]}],
  {{Inc -> bind[fun[Inc, {a}], e[1 + a]]},
   {Inc -> bind[fun[Inc, {a}], e[1 + a]]}, {}},
  bind[fun[Inc, {a}], e[1 + a]]],
 state[bind[a, e[3]],
  {{Inc -> bind[fun[Inc, {a}], e[1 + a]], a -> 3},
   {Inc -> bind[fun[Inc, {a}], e[1 + a]]}, {}},
  3],
 state[bind[a, e[3]],

```

```

    {{Inc -> bind[fun[Inc, {a}], e[1 + a]], a -> 3},
     {Inc -> bind[fun[Inc, {a}], e[1 + a]]}, {}},
    3],
state[mlet[e[1 + a]],
    {{Inc -> bind[fun[Inc, {a}], e[1 + a]], a -> 3},
     {Inc -> bind[fun[Inc, {a}], e[1 + a]]}, {}},
    3],
state[e[1 + a], {{}}, 4]]}

```

```

SatisfiesQ[At[ImPLY[Hold[NumberQ[a]], Ret[Inc] < a],
    fun[Inc]]][C2][1]]

```

False

- Let E be a set of placed propositions of P . Then we say C satisfies E if C satisfies every member of E . We write in Gedanken as $\text{SatisfiesQ}[E][S]$. For example,

```

SatisfiesQ[{At[x >= 0, A], At[x+1 > 0, A]}][C1]

```

True

4.2.2 Claims

A valid claim is a theorem about a program. In this section we define the Gedanken notations with respect to claims shown in Figures 4.4 and the claim proof rules listed in Figure 4.5.

A **claim** c written

$$[\rho] E \vdash_P \phi^\pi$$

is a theorem about a program P , saying that every computation of P from ρ which

Gedanken object	Description
K	Type of claims
$\text{claim}[\rho, E, P, \text{At}[\phi, \pi]]$	Claim $[\rho] E \vdash_P \phi^\pi$
$\text{claim}[E, P, \text{At}[\phi, \pi]]$	Claim $E \vdash_P \phi^\pi$
$\text{Assumptions}[[\rho] E \vdash_P \phi^\pi]$	E
$\text{Assertion}[[\rho] E \vdash_P \phi^\pi]$	ϕ^π
$\text{ValidQ}[c]$	Claim is valid in model
Fail	A proof rule doesn't apply
$I : K^n \rightarrow E \rightarrow (K \cup \{\text{Fail}\})$	Proof rule on claims

Figure 4.4: Claims

Proof rule
$\text{DeductionTheorem}[[\pi_1]E \vdash_P \phi_1^\pi, [\pi_1]F \vdash_P \phi_2^{\pi_2}]$ $\text{ModusPonens}[[\pi_1]E \vdash_P \phi_1^{\pi_1}, [\pi_1]E \vdash_P (\phi_1 \Rightarrow \phi_2)^{\pi_2}]$ $\text{Resolution}[[\pi_1]E \vdash_P (\neg \phi_1)^\pi, [\pi_1]E \vdash_P (\phi_1 \vee \phi_2)^{\pi_2}]$ $\text{WidenAssumptions}[F][[\pi_1]E \vdash_P \phi^\pi]$ $\text{Tautology}[\pi_1, E, P, \phi^\pi]$ $\text{IntroduceTheorem}[\pi_1, E, P, \phi^\pi]$ $\text{PushPlace}[\beta][[\pi]E \vdash_P \phi^\beta]$ $\text{AssignmentRule}[\beta_1][[\pi_1]E \vdash_P \phi^{\beta_0}]$ $\text{SelectionRule}[\beta_1][[\pi_1]E \vdash_P \phi^{\beta_0}]$ $\text{IfTrue}[\beta_1][[\pi]E \vdash_P \phi^{\beta_0}]$ $\text{IfFalse}[\beta_1 3][[\pi]E \vdash_P \phi^{\beta_0}]$ $\text{AfterIf}[\beta_+][[\pi]E \vdash_P \phi_1^{\beta_2}, [\pi]E \vdash_P \phi_2^{\beta_4}]$ $\text{CallRule}[\alpha][[\pi_1]E \vdash_P \phi_1^\beta, [\pi_1]E \vdash_P \phi_2^\gamma]$

Figure 4.5: Claim proof rules

reaches π and satisfies E also satisfies ϕ^π . We define claims in **Gedanken** with the form

$$\text{claim}[\rho, E, P, \text{At}[\phi, \pi]]$$

Let K be the type of claims. We say that c is a claim **about** P . For convenience, we also define

$$E \vdash_P \phi^\pi$$

to be equivalent to

$$[\text{init}]E \vdash_{\text{place}[\text{init}, P]} \phi^\pi$$

or in **Gedanken**

$$\text{Claim}[E, P, \text{At}[\phi, \pi]]$$

For example, if P is

```
P = seq[bind[a, e[1]],
        while[e[a > 0], bind[a, e[a-1]]],
        place[A, a]]
```

then a claim about P is

```
c = claim[{}, P, At[a == 1, A]]
```

A

```
[init] {} |- (a == 1)
```

P

The accessor **Assumptions** $[[\rho] E \vdash_P \phi^\pi]$ just gives E . The accessor **Assertion** $[[\rho] E \vdash_P \phi^\pi]$ just gives ϕ^π .

A claim is **valid** when the assertion that it represents is true, that is, if **ValidQ** $[c]$ is **True**. **ValidQ** is a procedure which:

- (1) Computes all computations C of $M(P)(s_{\text{initial}})$.

- (2) Restricts them to their final segments, the computations C' from r of P , in case r is a symbol. If $r = \mathbf{fun}[f]$, no restriction is necessary.
- (3) Returns **True** when every such C' that satisfies E also satisfies ϕ^π , otherwise **False**.

For example, where c is as above

`ValidQ[c]`

`False`

A procedure $I(c_1, \dots, c_n)(P)$ which takes a sequence of claims and yields either the symbol **Fail** or a valid claim c about P is called a **claim proof rule**. For example, here is an application of the **DeductionTheorem** rule defined below:

`c1 = claim[A, {pp1, pp2}, P, At[phi1, B]]`

B

`[A] {pp1, pp2} |- phi1`

P

`c2 = claim[A, {pp2, pp1, At[phi1, B]}, P, At[phi2, pi2]]`

$B \quad \quad \pi2$

`[A] {pp2, pp1, phi1 } |- phi2`

P

`DeductionTheorem[c1,c2]`

$\pi2$

`[A] {pp1, pp2} |- phi1`

P

We now define a number of Davis and Schwartz' proof rules [Sch77, §2(b)] for programs in our basic language. We give an example but not necessarily complete sampling of the rules that they discuss.¹ We present each rule as a “natural deduction style” tableau. The tableaux are read as “if the claims above the line are valid, then so is the claim below the line”. The names f, g when they occur are presumed to be functions declared in P . The symbols π, π_1, π_2 are places. An notation of the form $\beta_-; expression; \beta_+$, denotes a expression at place β_- with immediately following place β_+ . A specification like $x; \mathbf{v}$ means “we have a variable x together with some variables $\mathbf{v}$ in an expression or argument pattern”, and e are expressions. For any symbolic place π , let the propositional variable $\text{From}[\pi]$ denote that we have arrived from a function call immediately after place π . The rules are:

(1) **Deduction Theorem.**

$$\frac{[\pi_1]E \vdash_P \phi_1^\pi \quad [\pi_1](E \cup \phi_1^\pi) \vdash_P \phi_2^{\pi_2}}{[\pi_1]E \vdash_P \phi_1^{\pi_2}}$$

An example application of this rule was given above.

(2) **Modus Ponens.**

$$\frac{[\pi_1]E \vdash_P \phi_1^\pi \quad [\pi_1]E \vdash_P (\phi_1 \Rightarrow \phi_2)^\pi}{[\pi_1]E \vdash_P \phi_2^\pi}$$

For example,

```
c1 = claim[A, {pp1, pp2}, P, At[phi1, B]]
```

B

```
[A] {pp1, pp2} |- phi1
```

P

¹ Our purpose in re-presenting their rules is to show how they are implemented in *Gedanken*, as a practical demonstration, since when they designed their rule set they did not actually implement it as a computer program.

c2 = claim[A, {pp1, pp2}, P, At[ImPLY[phi1, phi2], B]]

B

[A] {pp1, pp2} |- (phi1 => phi2)

P

ModusPonens[c1,c2]

B

[A] {pp1, pp2} |- phi2

P

(3) **Resolution.**

$$\frac{[\pi_1]E \vdash_P \neg \phi_1^\pi \quad [\pi_1]E \vdash_P (\phi_1 \vee \phi_2)\pi}{[\pi_1]E \vdash_P \phi_2^\pi}$$

For example,

c1 = claim[A, {pp1, pp2}, P, At[!phi1, B]]

B

[A] {pp1, pp2} |- (!phi1)

P

c2 = claim[A, {pp1, pp2}, P, At[phi1 || phi2, B]]

B

[A] {pp1, pp2} |- (phi1 || phi2)

P

Resolution[c1,c2]

B

$$\frac{[A] \{pp1, pp2\} \vdash \phi^2}{P}$$

(4) **Widen Assumptions.**

$$\frac{[\pi_1]E \vdash_P \phi^\pi \quad E \subseteq F}{[\pi_1]F \vdash_P \phi^\pi}$$

For example,

`c1 = claim[A, {pp1}, P, At[phi, B]]`

$$\frac{[A] \{pp1\} \vdash \phi}{P}$$

`WidenAssumptions[{pp1,pp2}][c1]`

$$\frac{[A] \{pp1, pp2\} \vdash \phi}{P}$$

(5) **Tautology.**

$$\frac{\phi^\pi \in E}{[\pi_1]E \vdash_P \phi^\pi}$$

For example,

`Tautology[A, {pp1,At[phi,B]}, P, At[phi,B]] []`

$$\frac{[A] \{pp1, \phi\} \vdash \phi}{P}$$

(6) **Introduce Theorem.**

$$\frac{\vdash \phi \quad (\phi \text{ is universally valid})}{[\pi_1]E \vdash_P \phi^\pi}$$

For example,

```
IntroduceTheorem[A, {pp1,pp2}, P,
  At[(x + 1)^2 - x^2 == 1 + 2 x, B]] []
```

$$\frac{[A] \{pp1, pp2\} \vdash (-x^2 + (1 + x)^2 == 1 + 2x) \quad B}{P}$$

(7) **Push Place.**

$$\frac{[\pi]E \vdash_P \phi^\alpha \quad \alpha : (\beta : e \dots)}{[\pi]E \vdash_P \phi^\beta}$$

(8) **Assignment Rule.**

$$\frac{[\pi_1]E \vdash_P f(x; \mathbf{v})\beta_0 \quad \beta_0; x := e(x; \mathbf{v}); \beta_1 \quad \pi_1 \neq \beta_1}{[\pi_1]E \vdash_P (\exists y) (f(y; v) \wedge x = e(y; v))_1^\beta}$$

For example,

```
P = seq[place[A, bind[x, e]], place[B, f]];
c1 = claim[C, E, P, At[x < 12, A]]
```

$$\frac{[C] E \vdash (x < 12) \quad A}{P}$$

```
AssignmentRule[B] [c1]
```

$$\frac{[C] E \vdash [\text{Exists } y\$9] \quad (y\$9 == 12 \ \&\& \ x == \text{choose}[e]) \quad B}{P}$$

(9) **Selection Rule.**

$$\frac{[\pi_1]E \vdash_P f(x; \mathbf{v})_{-}^{\beta} \quad \beta_{-}; x := \text{choose}[e(x; \mathbf{v})]; \beta_{+} \quad \pi_1 \neq \beta_{+}}{[\pi_1]E \vdash_P (\exists y) (f(y; v) \wedge x \in e(y; v))_{+}^{\beta}}$$

For example,

P = seq[place[A, bind[x, choose[e]]], place[B, f]];

c1 = claim[C, E, P, At[x == 12, A]]

A

[C] E |- (x == 12)

P

SelectionRule[B] [c1]

B

[C] E |- [Exists y\$10] (y\$10 == 12 && x in e)

P

(10) **If True.**

$$\frac{[\pi]E \vdash_P \phi^{\beta_0} \quad \pi \neq \beta_1 \quad \beta_0 : \text{if}[C, \beta_1 : E, E']}{[\pi]E \vdash_P (\phi \wedge C)^{\beta_1}}$$

For example,

P = place[A, if[E0, place[B, E1], E2]]

c1 = claim[C, E, P, At[phi, A]]

A

[C] E |- phi

P

IfTrue[B] [c1]

B

[C] E |- (phi && E0)

P

(11) **If False.**

$$\frac{[\pi]E \vdash_P \phi^{\beta_0} \quad \pi \neq \beta_3 \quad \beta_0 : \text{if}[C, E, \beta_3 : E']}{[\pi]E \vdash_P (\phi \wedge \neg C)^{\beta_3}}$$

For example,

```
P = place[A, if[E0, E1, place[B, E2]]]
c1 = claim[C, E, P, At[phi, A]]
```

A

```
[C] E |- phi
P
```

```
IfFalse[B][c1]
```

B

```
[C] E |- (phi && !E0)
P
```

(12) **After If.**

$$\frac{[\pi]E \vdash_P \phi_1^{\beta_2} [\pi]E \vdash_P \phi_2^{\beta_4} \quad \pi \neq \beta_+ \quad \text{if}[C, E_1; \beta_2 : E_2, E'_1; \beta_4 : E'_2]; \beta_+ : E_3}{[\pi]E \vdash_P (\phi_1 \vee \phi_2)^{\beta_+}}$$

For example,

```
c1 = claim[pi, E, P, At[phi1, A]]
```

A

```
[pi] E |- phi1
P
```

```
c2 = claim[pi, E, P, At[phi2, B]]
```

B

```
[pi] E |- phi2
P
```

AfterIf[C][c1,c2]

C

[pi] E |- (phi1 || phi2)

P

(13) **Call Rule.**

$$\frac{\beta: x := \text{call } f(e_1(x; \mathbf{v}), \dots, e_m(x; \mathbf{v})); \pi: e_0 \quad \pi_1 \neq \pi \quad a = a_1, \dots, a_m}{\frac{[\pi_1]E \vdash_P \phi_1(x; \mathbf{v})^\beta \quad [\pi_1]E \vdash_P \phi_2(y, a_1, \dots, a_m)^\phi}{[\pi_1]E \vdash_P (\exists y) (\phi_1(y; v) \wedge \phi_2(x, e_1(y; v), \dots, e_m(y; v)))^\pi}}$$

For example,

P = seq[place[B, bind[x, call[f, {e[12], e[13]}]]],

place[C, E0]];

c1 = claim[A, E, P, At[phi1, B]]

B

[A] E |- phi1

P

c2 = claim[A, E, P, At[phi2, fun[f]]]

fun[f]

[A] E |- phi2

P

CallRule[{a,b}][c1,c2]

C

[A] E |- (phi1 && phi2)

P

Gedanken form	Meaning
Praa[P, E, F]	Praa $\langle P, E, F \rangle$
VacuousPraa[P, E]	The vacuously correct praa $\langle P, E, \emptyset \rangle$
Program[$\langle P, E, F \rangle$]	P
Assumptions[$\langle P, E, F \rangle$]	E
Assertions[$\langle P, E, F \rangle$]	F
Assumption[i][$\langle P, E, F \rangle$]	E_i
Assertion[i][$\langle P, E, F \rangle$]	F_i
Claim[i][$\langle P, E, F \rangle$]	$E \vdash_P F_i$

Figure 4.6: Programs with assumptions and assertions

4.2.3 Programs with assumptions and assertions

A **program with assumptions and assertions** (praa) $R = \langle P, E, F \rangle$ is a program P with a set E of assumptions and F of assertions, where the elements of E and F are placed propositions. A praa $\langle P, E, F \rangle$ is **correct** if

$$(\forall \phi^\pi \in F) E \vdash_P \phi^\pi$$

A **praa transformation** T is a mapping $R' = T(R)$ from correct praas to correct praas. In this subsection we define the **Gedanken** notations with respect to praas shown in Figure 4.6, and we define the praa transformations shown in Figure 4.7.

The vacuously correct praa, for any P and E , is $\langle P, E, \emptyset \rangle$. In **Gedanken**, this is written **VacuousPraa**[P, E]. For example,

VacuousPraa[e[12], {}]

Praa

====

program: e[12]

assumptions: {}

assertions: {}

We may transform correct praas into correct praas in a number of ways [Sch77, §2(c)]. We give a few, but not all, of the rules defined in [Sch77, §2(c)]. (Our goal is just

Transformation rule
EqReduceLeft[F, e_1, e_2, π][$\langle P, E, G \rangle$]
EqReduceRight[F, e_1, e_2, π][$\langle P, E, G \rangle$]
Choice[F, e_1, e_2, π][$\langle P, E, G \rangle$]
Subset[F, e_1, e_2, π][$\langle P, E, G \rangle$]
DeleteDead[F, e_1, e_2, π][$\langle P, E, G \rangle$]
ReplaceDead[F, e_1, e_2, π, s][$\langle P, E, G \rangle$]
AddAssumption[ϕ^π][$\langle P, E, F \rangle$]

Figure 4.7: Praa transformation rules

to define the interface and intent of this kind of rule.) In our notation below, $P[e/\pi]$ means to replace `place[ $\pi, f$ ]` in program P with `place[ $\pi, e$ ]`. The rules are in tableau format, so that if we have what is described on top of the line, then that is equivalent to what is below the line. The rules are:

(1) **Equality Reduce Left.**

$$\frac{(P, E, G) \quad G = F \cup \{(e_1 = e_2)^\pi\} \quad \pi : e_1}{(P[e_2/\pi, E, F])}$$

For example,

```
EqReduceLeft[{}, e1, e2, pi][
  Praa[place[pi, e1], {}, {At[e1 == e2, pi]}]]
```

Praa

====

```
program: place[pi, e2]
```

```
assumptions: {}
```

```
        pi
```

```
assertions: {(e1 == e2) }
```

(2) **Equality Reduce Right.**

$$\frac{(P, E, G) \quad G = F \cup \{(e_1 = e_2)^\pi\} \quad \pi : e_2}{(P[e_1/\pi, E, F \cup \{(e_1 = e_2)^\pi\})}$$

For example,


```
EqReduceRight[{}], e1, e2, pi][
  Praa[place[pi, e2], {}, {At[e1 == e2, pi]}]]
```

Praa

====

program: place[pi, e1]

assumptions: {}

pi

assertions: {(e1 == e2) }

(3) **Choice.**

$$\frac{(P, E, G) \quad G = F \cup \{(e_1 \in e_2)^\pi\} \quad \pi : \mathbf{choose}[e_2]}{(P[e_1/\pi], E, F \cup \{(e_1 \in e_2)^\pi\})}$$

For example,

```
Choice[{}], e1, e2, pi][
```

```
  Praa[place[pi, choose[e2]], {}, {At[InSet[e1, e2], pi]}]]
```

Praa

====

program: place[pi, e1]

assumptions: {}

pi

assertions: {e1 in e2 }

(4) **Subset.**

$$\frac{(P, E, G) \quad G = F \cup \{(e_1 \subseteq e_2)^\pi\} \quad \pi : \mathbf{choose}[e_2]}{(P[\mathbf{choose}[e_1]/\pi], E, F \cup \{(e_1 \subseteq e_2)^\pi\})}$$

For example,

```
Subset[{}], e1, e2, pi][
```

```
  Praa[place[pi, choose[e2]], {}, {At[SubsetEq[e1, e2], pi]}]]
```

```

Praa
====

program: place[pi, choose[e1]]
assumptions: {}

                                pi
assertions: {SubsetEq[e1, e2] }

```

(5) **Delete Dead Code.**

$$\frac{(P, E, G) \quad G = F \cup \mathbf{False}^\pi}{(P[e/\perp @ \pi, E, F])}$$

For example,

```

DeleteDead[{}, e1, e2, pi][
  Praa[seq[place[pi,e1], e[12]], {}, {At[False, pi]}]]

```

```

Praa
====

program: seq[place[pi, bot], e[12]]
assumptions: {}
assertions: {}

```

(6) **Replace Dead Code.**

$$\frac{(P, E, G) \quad G = F \cup \mathbf{False}^\pi \quad s \text{ any expression}}{(\text{ReplacePlace}[P, \pi, s], E, F)}$$

For example,

```

ReplaceDead[{}, e1, e2, pi, call[f, {a,b}]] [
  Praa[seq[place[pi,e1], e[12]], {}, {At[False, pi]}]]

```

```

Praa
====

program: seq[place[pi, call[f, {a, b}]], e[12]]

```

```

assumptions: {}

                                pi

assertions: {False }

```

(7) **Add Assumption.**

$$\frac{(P, E, F) \quad \phi^\pi \text{ an assumption}}{(P, E \cup \{\phi^\pi\}, F)}$$

For example,

```

AddAssumption[At[phi,pi]] [
  Praa[seq[place[pi,e1], e[12]], {}, {}]]

Praa
====

program: seq[place[pi, e1], e[12]]

                                pi

assumptions: {phi }

assertions: {}

```

4.2.4 Proof of a claim

A **claim proof** gives the logical justification for a claim. It is comprised of a sequence of **claim proof steps** $\Xi = \langle \xi_1, \dots, \xi_n \rangle$ where $\xi_i = \langle c_i, r_i, A_i \rangle$, where

- c_i is a claim which equals $r_i(\langle c_{A_{i,j}} \rangle_{j=1}^{\#A_i})$
- r_i is a claim proof rule, and
- A_i is tuple of integers.

In this subsection we define the **Gedanken** notations with respect to claim proofs shown in Figure 4.8.

Concrete syntax	Gedanken form	Semantics
$\langle c, R, A \rangle$	<code>cproofStep</code> $[c, r, A]$	Claim proof step
$\xi_1, \dots, \xi_n$	<code>cproof</code> $[\xi_1, \dots, \xi_n]$	Claim proof
	<code>Assertion</code> $[\langle c, R, A \rangle]$	<code>Assertion</code> $[c]$
	<code>Claim</code> $[\langle c, R, A \rangle]$	c
	<code>Claim</code> $[\xi_1, \dots, \xi_n][i]$	<code>Claim</code> $[\xi_i]$
	<code>Attempt</code> $[\Xi, R, A]$	Proof rule on claims

Figure 4.8: Claim Proofs

In *Gedanken*, for a proof we write `cproof` $[\xi_1, \dots, \xi_i]$, where the steps ξ_i are written `cproofStep` $[c_i, r_i, A_i]$. If $\langle c_i, r_i, A_i \rangle$ is the last step of a proof C , then we say C is a proof of claim c_i .

We define the procedure `Attempt` $[\Xi, R, A]$ on proofs of claims which takes proof Ξ , rule R and integer argument tuple A . If $c = R(\langle c_{A_i} \rangle_{i=1}^{\#A})$ is not `Fail`, then the `Attempt` returns $\Xi; \langle \langle c, R, A \rangle \rangle$, otherwise `Fail`.

Here is an example of the proof of a claim, that the value of variable `max` is greater or equal to the value of `a`. Note that we define `VerifyQ` $[\phi]$ to `True` here to make the example go through (we do so to insert a theorem of pure arithmetic without building a decision procedure for pure arithmetic, just to make the demonstration):

```

P =
  seq[
    place[F,
      if[e[a > b],
        place[S1,
          seq[place[A, bind[max, a]],
            place[B, max]]],
        place[S2,
          seq[place[C, bind[max, b]],
            place[D, max]]]]],
    place[E, max]];
X1 = cproof[];

```

```

X2 = Attempt[X1,
              IntroduceTheorem[
                F, {}, P, At[True, F]], {}];
X3 = Attempt[X2, IfTrue[S1], {1}];
X4 = Attempt[X3, IfFalse[S2], {1}];
X5 = Attempt[X4, PushPlace[A], {-2}];
X6 = Attempt[X5, PushPlace[C], {-2}];
X7 = Attempt[X6, AssignmentRule[B], {-2}];
X8 = Attempt[X7, AssignmentRule[D], {-2}];
X9 = Attempt[X8, AfterIf[E], {-2, -1}];
VerifyQ[x_] := True;
X10 = Attempt[X9, IntroduceTheorem[F, {}, P,
  At[ImPLY[(a > b && max == a) ||
          (a <= b && max == b),
          max >= a], E]], {}];
X11 = Attempt[X10, ModusPonens[], {-2, -1}]

```

proof of claim

```

      F
1. [F] {} |- True    (IntroduceTheorem)
      P
      S1
2. [F] {} |- (a > b)  (IfTrue)
      P
      S2
3. [F] {} |- (a <= b) (IfFalse)
      P
      A
4. [F] {} |- (a > b)  (PushPlace)
      P

```

Gedanken form	Semantics
$\text{Ppc}[R, \Xi_E, \Xi_F]$	Praa proof
$\text{VacuousRule}[P, E][\]$	Praa proof rule
$\text{Attempt}[D, A, i][\Pi]$	Praa proof rule
$\text{Promote}[i][\Pi]$	Praa proof rule

Figure 4.9: Praa Proofs

C

5. [F] {} |- (a <= b) (PushPlace)

P

B

6. [F] {} |- (a > b && max == a) (AssignmentRule)

P

D

7. [F] {} |- (a <= b && max == b) (AssignmentRule)

P

E

8. [F] {} |- (a > b && max == a ||
a <= b && max == b) (AfterIf)

P

E

9. [F] {} |- (a > b && max == a ||
a <= b && max == b => max >= a)

P (IntroduceTheorem)

E

10. [F] {} |- (max >= a) (ModusPonens)

P

4.2.5 Proof of a praa

A **praa proof** proves that a praa is correct. In this subsection we define the Gedanken notations with respect to praa proofs shown in Figure 4.9.

Let $E = (e_1, \dots, e_m)$ be the assumptions of a correct praa $R = \langle P, E, F \rangle$. For $1 \leq j \leq m$, let $\Xi_{E,i}$ be a claim proof, which may range from being vacuous to proving e_i . ($\Xi_{E,i}$ is a **developing proof**, which if it is not empty is intended to be an eventual proof of claim $E \setminus \{e_i\} \vdash_P e_i$.) Let $\Xi_E = \langle \Xi_{E,1}, \dots, \Xi_{E,m} \rangle$. Let $F = (f_1, \dots, f_n)$ be the assertions of R . For $1 \leq i \leq n$, let Ξ_i be a proof of claim $E \vdash_P f_i$. Let $\Xi_F = \langle \Xi_{F,1}, \dots, \Xi_{F,n} \rangle$. Then we say that $\Pi = (R, \Xi_E, \Xi_F)$ is a **praa proof** (of correctness) of R . In Gedanken, write this as $\text{Ppc}[R, \Xi_E, \Xi_F]$.

A **proof rule** on proofs of praas is an operation $O(\Pi_1, \dots)$ from praa proofs to praa proofs. The rule may either not apply, returning **Fail**, or apply, returning a correct praa proof. Π may be modified in any way which preserves correctness. We define the following example proof rules:

- Let the vacuous praa proof of a program P with only assumptions E be obtained by the $\text{VacuousRule}[P, E] = \langle \langle P, E, \emptyset \rangle, \{ \langle \rangle : e \in E \}, \emptyset \rangle$. This gives a way of getting initial versions of praa proofs. For example,

```
P = seq[
  place[F,
    if[e[a > b],
      place[S1, seq[place[A, bind[max, a]],
        place[B, max]]],
      place[S2, seq[place[C, bind[max, b]],
        place[D, max]]]]],
  place[E, max]];
```

```
PP1 = VacuousRule[P, {At[max >= a, E]}] []
```

```
Praa proof of correctness
```

```
Praa
```

```
====
```

```
program: seq[place[F, if[e[a > b],
```

```

place[S1, seq[place[A, bind[max, a]], place[B, max]]],
place[S2, seq[place[C, bind[max, b]], place[D, max]]]]],
place[E, max]]

```

E

assumptions: {(max >= a) }

assertions: {}

Assumptions: Claim proofs:

proof of claim

Assertions: Claim proofs:

- Let D be any claim proof rule, A a list of integers referring to claims in the developing proof of assumption i in the praa of praa proof $\Pi = \langle \langle P, E, F \rangle, C_E, C_F \rangle$. $\text{Attempt}[\Pi, D, A, i]$ is a proof rule on proofs which does an $\text{Attempt}[\Xi_{E,i}, D, A]$. If this returns **Fail**, so does the **Attempt**, otherwise the **Attempt** extends proof $\Xi_{E,i}$ by one step and returns the modified Π . Continuing the previous example (“...” elides output which is the same as in the previous part of the example):

```
PP2=Attempt[IntroduceTheorem[F,{},P,At[True,F]], {}, 1][PP1];
```

```
PP3=Attempt[IfTrue[S1], {1}, 1][PP2];
```

```
PP4=Attempt[IfFalse[S2], {1}, 1][PP3];
```

```
PP5=Attempt[PushPlace[A], {-2}, 1][PP4];
```

```
PP6=Attempt[PushPlace[C], {-2}, 1][PP5];
```

```
PP7=Attempt[AssignmentRule[B], {-2}, 1][PP6];
```

```
PP8=Attempt[AssignmentRule[D], {-2}, 1][PP7];
```

```
PP9=Attempt[AfterIf[E], {-2, -1}, 1][PP8];
```

```
PP10=Attempt[IntroduceTheorem[F,{},P,
```

```
At[ImPLY[(a > b && max == a) ||
```

```
(a <= b && max == b), max >= a],
```

```
E]], {}, 1][PP9];
```

```
PP11=Attempt[ModusPonens[], {-2, -1}, 1][PP10]
```


Praa proof of correctness

Praa

====

program: ...

E

assumptions: {(max >= a) }

assertions: {}

Assumptions:

Claim proofs:

proof of claim

F

1. [F] {} |- True (IntroduceTheorem)

P

S1

2. [F] {} |- (a > b) (IfTrue)

P

S2

3. [F] {} |- (a <= b) (IfFalse)

P

A

4. [F] {} |- (a > b) (PushPlace)

P

C

5. [F] {} |- (a <= b) (PushPlace)

P

B

6. [F] {} |- (a > b && max == a) (AssignmentRule)

P

D

7. [F] {} |- (a <= b && max == b) (AssignmentRule)

```

8. [F] {} |- (a > b && max == a ||
               P
               E
               a <= b && max == b) (AfterIf)
9. [F] {} |- (a > b && max == a ||
               P
               E
               a <= b && max == b => max >= a) (IntroduceTheorem)
               E
10. [F] {} |- (max >= a) (ModusPonens)
               P

```

Assertions: Claim proofs:

- We can make assumption i become an assertion, with `Promote` $[\Pi, i]$, Π , i as above, if the developing proof $\Xi_{E,i}$ of e_i has as its final claim $E \setminus \{\psi_i\} \vdash_P \psi_i$. Completing the above example (“...” elides output which is the same as in the previous part of the example):

```

PP12 = Promote[1] [PP11]

Praa proof of correctness

Praa

====

program: ...

assumptions: {}

               E

assertions: {(max >= a) }

Assumptions: Claim proofs:

Assertions: Claim proofs:

               proof of claim

               F

1. [F] {} |- True (IntroduceTheorem)

```

$$\begin{array}{c}
P \\
\vdots \\
E \\
10. [F] \{ \} \vdash (\max \geq a) \quad (\text{ModusPonens}) \\
P
\end{array}$$

4.3 Discussion

We have been working interactively in a line-oriented fashion in Mathematica, marking every place by hand. A “production” version of the correct program editor would presumably have a two-dimensional interface in which the obvious chores of marking places as “anchors” for inference rules would be automated, so that the user need only point to the rule context points, select the rule to be applied, input any optional arguments to the rule, and the system would do the rest.

That said, we must still remark that our experience with the Davis-Schwartz framework, as evident above in the task of proving a simple fact about a one-line program to compute the maximum of two numbers, is that program verification in the Davis-Schwartz framework will remain an extremely tedious task, well beyond the reach of the average programmer. This is because, even with the automation suggested in the previous paragraph, the editor user will still need to have a firm grasp of the concept of **proof**, not in the “working mathematician’s” intuitive sense, but in the strict, fundamental sense employed by logicians.

We conjecture that a full automated exposition of the searching praa derivations simulated by hand by Deak in 10 pages [Dea77], would take 30 or 40 pages of tedious derivation if we were to undertake them here using the line-oriented technique we have been able to achieve. Development of a two-dimensional automated interface might take several months of full-time programming, and would not contribute significant additional insight.

Chapter 5

Heuristics

In this chapter, we review work on the problem of improving the average cost of deciding sentences in languages whose decision problem is in general intractable.

First, we examine three sublanguages, in set theory ([FOS80], [Pol87a]), real algebra ([CH91], [Hon93]) and integer linear programming ([Dav57], [Coo71], [Coo72], [Sho77], [CZ93]). In reviewing this work we will see that the analysis of the effectiveness of heuristics in real algebra and integer linear programming tends toward the anecdotal.

Several approaches have arisen which give optimistic results:

- (1) Search for tiny sublanguages whose decision problem has polynomial time complexity in the worst case. For example, the problem of deciding the satisfiability of purely existentially quantified conjuncts ϕ of literals of the form $x < y - c$, $x < c$ or $x = y$ is solvable in time $O(n^3)$ where n is the number of distinct integer variables occurring in ϕ [Pra77, Sho81].
- (2) Show that the average case time complexity of some selection of the instances of an intractable decision problem is of a lower order of magnitude than the worst case for all instances. We briefly review such results for variants of the Davis-Putnam propositional decision procedure [DP60, Gol79, Fra86, Fra91, PT92].

We explore item 2 more fully in this chapter; the utility of item 1 is self-evident.

Several approaches have arisen which give pessimistic results:

- (1) One approach is to prove that a given NP-Complete decision problem is **average-case hard** in the sense of Levin ([Lev86], [BDCGL89], [BG91], [Gur91], [VR92], [SY92], [RS93]). So far, it has fairly difficult to prove average-case NP-hardness under the various definitions. We should note that the definitions in the literature haven't yet been standardized. We define the general terms and some of the results of this approach, and we prove that *EMLS* is NP-average complete.
- (2) By “tailoring a logic to a problem”, we mean selecting inference rules which, in one step, decide entire theories (for example, Presburger arithmetic). It is obvious that if we tailor a logic to a certain NP-Complete problem, in terms of our choice of axioms and inference rules (for example, we can build a strong inference rule like “equivalence under associative-commutative laws”), then many proofs for instances of that problem will be shorter, but the verification cost for establishing the enabling conditions of such an inference rule for each step may be higher. Conversely, a proof with simpler rules may be much longer, but each step may be easier to check. This suggests a certain “incompressibility” of the product of proof size and verification cost. Indeed, the worst-case intractability of the given problem tells us that **no** choice of axioms and inference rules will change the overall difficulty of the problem of **finding** a proof of a valid relationship. (On the other hand, different approaches to finding a proof may yield proofs which are **checkable** more or less quickly.) But it is not satisfying to stop with this observation. In the last section of this chapter we expand on this question.

5.1 Some key intractable sublanguages

We review some of the key sublanguages relied upon by the decidable sublanguages of set theory approach [FOS80, CFOS87]. The three languages are in the do-

mains of set theory, real algebra, and integer linear programming.

5.1.1 Set theory

A key intractable decidable sublanguage in set theory is multi-level syllogistic (MLS), or sets with $=, \neq, \in, \notin, \cup$ and $\setminus$. First we define the grammar of the language, and then we give the worst case complexity of a decision procedure for it.

The grammar of **elementary multilevel syllogistic** (EMLS) is that of a conjunct of positive atoms:

$$\begin{aligned} \mathbf{Atom} ::= & \mathbf{id} = \mathbf{id} \cup \mathbf{id} \mid \mathbf{id} = \mathbf{id} \setminus \mathbf{id} \mid \\ & \mathbf{id} \in \mathbf{id} \mid \mathbf{id} \notin \mathbf{id} \mid \mathbf{id} = \mathbf{id} \mid \mathbf{id} \neq \mathbf{id} \\ \mathbf{EMLS} ::= & \mathbf{Atom} \mid \mathbf{Atom} \wedge \mathbf{EMLS} \end{aligned}$$

The problem of finding a satisfying assignment for sentences in EMLS is NP-Complete, that is, it can be computed in polynomial time by a nondeterministic Turing Machine, and any NP-complete problem such as 3-SAT can be reduced to it [GJ79]. While it has not yet been proven, the pragmatic experience to date is that this implies an exponential deterministic time lower bound in the worst case.

In the worst case, an upper bound on the number of model graphs that need to be checked is 2^{4n^3} where n is the number of variables in the sentence. Coincidentally, the number of distinct EMLS conjuncts is $2^{2n^3+4n^2}$, whose exponent is of the same order of magnitude.

5.1.2 Real algebra

A key intractable sublanguage within real algebra is the existential theory of sentences over real numbers with addition, multiplication and general inequality. We

give the definition of this language, and then comment on the tractability of the decision procedures available for the language.

The grammar of this language is as follows. Let **int** denote any integer. The rational numbers **rat** are fractions (pairs of integers, numerator and denominator). Terms **t** are either identifiers **id**, rational numbers **rat**, or binary operations **t op t** on terms. The operations are addition, subtraction and multiplication. Literals **lit** are the result of evaluating relations **rel** on terms. The relations are equality, inequality and magnitude comparison. Finally, the sentences of the language are the purely existentially quantified conjunctions of literals. All of this is summarized in the following grammar:

rat ::= **int/int**

t ::= **id** | **rat** | **t op t**

op ::= + | - | ×

rel ::= = | ≠ | < | ≥

lit ::= **t rel t**

R ::= **lit** | **lit** ∧ **R**

The complexity of finding satisfying assignments for sentences in **R** is bounded above by the complexity of the alternating quantifier problem, also known as Tarski sentences. Let $\phi \equiv f_1 \wedge f_2 \wedge \dots \wedge f_m$ be a quantifier-free conjunction of a set of polynomials $\mathcal{F} = \{f_i : 1 \leq i \leq m\}$ over n variables. Consider the sentence $\psi \equiv Q_1 \dots Q_n \phi$, where each Q_i is alternately an existential or universal quantifier over n_i variables. The worst-case time complexity of the best known sequential algorithm is [Mis93, p. 382]:

$$\left(|\mathcal{F}| \deg(\mathcal{F}) \prod^{O(n_i)}\right)$$

where $|\mathcal{F}|$ is some measure of the size of $\mathcal{F}$.

Strong restrictions are similarly complex:

Fischer and Rabin [FR74] prove a very strong lower bound. They show that any nondeterministic Turing machine which decide $(R, <, +)$ requires 2^{cn} time in the

worst case, for almost all n . Ferrante and Rackoff [FR75] give for the same problem a deterministic upper bound on the worst-case time complexity of $2^{2^{cn}}$ where c is a constant. The nondeterministic lower bound of Fischer and Rabin strongly suggests that the deterministic upper bound of Ferrante and Rackoff cannot be improved significantly.

5.1.3 Presburger arithmetic and the feasibility problem for integer linear programming

A key intractable decidable sublanguage within number theory is **Presburger arithmetic** and its purely existentially quantified subcase, the feasibility problem for integer linear programming (call this FPILP). Let the base language of literals be called **ILP**. Presburger arithmetic is **ILP** with unrestricted use of existential and universal quantifiers over integer-valued variables. FPILP is **ILP** with just universal or just existential quantifiers. The grammar of **ILP** is as follows:

$$\begin{aligned} \mathbf{ILP} ::= & \mathbf{id} = 0 \mid \mathbf{id} = 1 \mid 0 \leq \mathbf{id} \mid \\ & \mathbf{id} = \mathbf{id} + \mathbf{id} \mid \mathbf{id} = \mathbf{id} - \mathbf{id} \mid \\ & \mathbf{ILP} \wedge \mathbf{ILP} \end{aligned}$$

In the worst case, the satisfiability of Presburger arithmetic is, by algorithms of Hilbert-Bernays [HB68, pp. 366-375] and Cooper [Coo71, Coo72], according to Oppen [Opp75]

$$2^{2^{2^n}}$$

time steps for inputs of size n for mixed quantifiers, but Cooper's algorithm rarely exhibits worst case, according to Shostak [Sho77]. Shostak notes that Oppen has observed that the satisfiability problem for FPILP is NP-Complete. The problem of deciding the satisfiability of purely existentially quantified conjuncts ϕ of literals of the form $x < y - c$, $x < c$ or $x = y$ is, on the other hand, solvable in time $O(n^3)$ where n is the number of

distinct integer variables occurring in ϕ [Pra77, Sho81].

5.2 Heuristics for two key sublanguages

Now we review some heuristics in the literature for improving the average case complexity of two key decidable sublanguages, real algebra and integer linear programming.

5.2.1 Real algebra

Some heuristics for improving the average case tractability of computing satisfying assignments to sentences in real algebra were given by Collins and Hong (1991) [CH91], who provide a method of partial CAD (cylindrical algebraic decomposition). CAD, for a sentence ϕ in n variables, decomposes $\mathbf{R}^n$ into a set of cells over which the polynomials of ϕ do not vary in sign. The truth of a sentence is then determined by checking the sign of each polynomial in each cell against its constraint ($< 0, = 0, \leq 0, \geq 0, > 0$).

Partial CAD, using heuristics reminiscent of the Intuitionist paradigm, constructs only as much of the full CAD as needed. The main heuristics are:

- (1) If $\phi(x, y) \equiv \phi_1(x) \vee \phi_2(x, y)$, and $\phi_1(x)$ is true in cell C , then $\phi(x, y)$ is true, so construct no cylinder over C for $\phi_2(x, y)$.
- (2) If $\phi(x, y) \equiv \exists y. \phi_1(x, y)$, and $\phi_1(x, y)$ is true in one cell, then $\phi(x, y)$ is true, so construct no more cells.
- (3) If $\phi(x, y) \equiv \forall y. \phi_1(x, y)$ and $\phi_1(x, y)$ is false in one cell, then $\phi(x, y)$ is false, so construct no more cells.

5.2.2 Presburger arithmetic and the feasibility problem for integer linear programming

The complexity of computing satisfying assignments for Presburger sentences depends on whether the variables are quantified purely existentially or universally, or with mixed existential and universal quantifiers:

- **Mixed quantifiers.** This is the hardest case, in which we apply Cooper’s algorithm.
- **Not mixed, i.e. FPILP.** If we have an FPILP problem, we apply Shostak’s SUP-INF method, which lifts the problem into the rationals, triangularizes the system, and recursively computes upper and lower bounds for each variable, thus narrowing the search. This method may fail; if so, we apply Cooper’s algorithm.

Cooper’s algorithm is a method of elimination which uses a rule for eliminating the innermost quantifier. The eliminated variable is replaced by a **bounded parameter**, the interesting values of which are obtained by applying Euclid’s algorithm to find a finite solution set to a small system of linear Diophantine equations. This always works.

The above pair of methods constitutes a “heuristic” in the sense that we do not have a precise characterization of when Shostak’s method will fail and when Cooper’s method will blow up.

5.2.3 Analysis of these heuristics

The analysis used to judge the utility of these heuristics is somewhat anecdotal. We will present such analyses for heuristics for real algebra and integer linear programming.

5.2.3.1 Real algebra

Of Partial CAD, Collins and Hong note [CH91]:

- “Preliminary observations show that the new method is always more efficient than the original, and often significantly more efficient”.
- Strategies are written for particular problem classes to choose the next ϕ_i of the ϕ_i of minimum degree for which to construct a CAD cell.
- Performance comparisons are given for full versus partial CAD for single elements of those problem classes.

There is no reason to doubt that the improvements are dramatic, since it is clear that the heuristics presented will reduce the problem size when they are applicable. However, no evidence was presented to give conclusive arguments that an order of magnitude gain in the average case complexity has been attained in general.

5.2.3.2 Integer linear programming

There is a large body of literature in operations research on mixed integer-linear programming problems. With respect to deciding FPILP, Cooper [Coo71] notes that:

- If we have all $\forall$ or all $\exists$ quantifiers, the algorithm shows a significant improvement.
- If x appears only with a coefficient of 1 in all inequalities, the algorithm shows a significant improvement.

Shostak notes that:

- Incompleteness of SUP-INF algorithm is rare.

- Program verification rarely constructs sentences with the difficult Diophantine behavior with respect to SUP-INF of the invalid sentence

$$(\forall x, y) 2x + 3y \neq 1$$

- Most often, the algorithm produces sets of linear inequalities that have integral solutions iff they have rational solutions. (“Most often” is by experience, it is not given an analytic definition.)
- There is no simple syntactic categorization of the formulas that lead to incompleteness.
- In the incomplete case, Cooper’s algorithm can be used; no cost analysis is given, since this case is “largely a matter of theoretical interest”, and because such analysis would entail a definition of “most often” above.
- However, it is unlikely that these algorithms have good average case behavior on some very simple distributions. As we will show below, the feasibility problem for integer linear programming is NP-average complete.

5.3 Stronger average case results

Now we present some average case complexity results with stronger implications. A paradigm for average case complexity analysis is as follows:

- (1) Define the language L .
- (2) Assume a certain distribution D of frequency of occurrence in practice of elements of the language.
- (3) Define an algorithm A for deciding the language.
- (4) Compute the average case complexity of A on L weighted by D .

We review four papers giving results following this paradigm:

- First, Goldberg’s analysis of the Davis-Putnam algorithm for the satisfiability of propositions of 1979 [Gol79].
- Franco’s (1991) algorithm INFREQ [Fra86, Fra91] for the same problem.
- Policriti and Tetali’s algorithm of 1992 to decide the satisfiability of propositions in which the proposition letters are ground terms in a polynomial-time decidable sublanguage, obtained as a generalization of the Davis-Putnam algorithm [PS92, PT92].
- Chvatal and Szemerédi’s (1987) results on the distribution of “hard” instances of SAT for resolution [CS87].

5.3.1 Goldberg for Davis-Putnam for SAT (1979)

We briefly consider Goldberg’s analysis of the average case of the Davis-Putnam procedure for deciding sentences in propositional logic [Gol79]. We define the language, the probability distribution on inputs in the language, the algorithm, and the result of the analysis.

The language consists of r identifiers, literals which are either an identifier or the negation of an identifier, clauses, which are disjuncts of literals, and propositions, which are conjuncts of disjuncts:

$$\mathbf{Id}(r) ::= x_1 \mid \dots \mid x_r$$

$$\mathbf{L}(r) := \mathbf{Id}(r) \mid \neg \mathbf{Id}(r)$$

$$\mathbf{C}(r) := \{\mathbf{L}(r)^+\} \quad \text{Disjunct of literals}$$

$$\mathbf{P}(r) := \{\mathbf{C}(r)^+\} \quad \text{Conjunct of disjuncts}$$

A probability distribution of sentences gives our opinion of the likelihood of a given sentence occurring in practice. Let $\mathbf{C}'(r)$ be the subset of clauses $\mathbf{C}(r)$ in which ℓ and $\neg\ell$ do not appear simultaneously. There are 3^r elements of $\mathbf{C}(r)$ in this subset. Let $\mathbf{K}(n, r)$ be the indexed Cartesian product $\times_{1 \leq i \leq n} \mathbf{C}'(r)$. This is the set of conjuncts of n clauses over r variables, with repetitions. Clearly the size of $\mathbf{K}(n, r)$, $\#\mathbf{K}(n, r) = 3^{rn}$. We assume each element of $\mathbf{K}(n, r)$ to be equally likely to occur in practice.

Goldberg analyzes the following simplified version of Davis-Putnam. S is an element of $\mathbf{K}(n, r)$, that is, a sentence in conjunctive normal form expressed as an n -tuple of clauses in $\mathbf{C}(r)$. Recall that a clause in $\mathbf{C}(r)$ is a set of zero or more negated or non-negated atoms chosen from a set of r propositional letters. The algorithm is then:

```

proc  $DPP'(S) :=$ 
  if  $S = \emptyset$  then SATISFIABLE
  elseif  $\emptyset \in S$  then UNSATISFIABLE
  else  $L :=$  some literal in  $S$ 
     $S_1 := \{C \setminus \{\neg L\} \mid C \in S \wedge L \notin C\}$ 
     $S_2 := \{C \setminus \{L\} \mid C \in S \wedge \neg L \notin C\}$ 
     $\neg(DPP'(S_1) = \text{UNSATISFIABLE} \wedge$ 
       $DPP'(S_2) = \text{UNSATISFIABLE})$ 

```

Under these conditions, the average case complexity of DPP' over $\mathbf{K}(n, r)$ with uniform distribution is of order (where c is a positive constant)

$$T(n, r) = \begin{cases} cnr + \sum_{i=0}^{n-1} \binom{n}{i} (2/3)^i (1/3)^{n-i} T(i, r-1) & \text{for } n, r > 0 \\ 0 & \text{otherwise} \end{cases}$$

5.3.2 INFREQ

Now we look at Franco's analysis of the INFREQ algorithm for SAT [Fra91]. The language is the same as for Goldberg [Gol79]. Franco has a different input distribution

Var Type	Substitution Name	Occurrence	Replacement
unit	unit elimination	$(v, x, \dots)$	true
		$(\neg v, x, \dots)$	true
double	double elimination	$(v, v, x, \dots)$	true
		$(\neg v, \neg v, x, \dots)$	true
	trivial elimination	$(v, \neg v, x, \dots)$	true
	pure literal rule	$(v, x, \dots), (v, y, \dots)$	true
		$(\neg v, x, \dots), (\neg v, y, \dots)$	true
	resolution	$(v, x, \dots), (\neg v, y, \dots)$	$(x, \dots, y, \dots)$

Figure 5.1: Franco’s CNF simplification rules

assumption, different algorithm, and different result.

Franco’s distribution allows repetitions of variables in clauses. Let p be a probability. Suppose there are r variables. Let $M(n, r, p)$ be the set of conjuncts of n clauses C_i such that for all literals ℓ in $\mathbf{L}(r)$, ℓ is put in C_i with probability p .

Let S be a problem instance in $M(n, r, p)$. A **double variable** is one that appears exactly twice in S . A **unit variable** is one that appears exactly once in S . Then Franco’s algorithm $INFREQ(S)$ is as follows (where the “unit elimination” and “double variable” rules are as shown in Figure 5.1):

- (1) If $\emptyset \in S$ then return **Unsatisfiable**.
- (2) Otherwise,
 - (a) Repeatedly apply double variable rules in order until all opportunities vanish.
 - (b) Eliminate all clauses containing double variables that appear once in S .
 - (c) Repeatedly apply unit variable rules until all opportunities vanish.
 - (d) For all truth assignments t to variables which occur more than twice in S , if t satisfies S then return **Satisfiable**.
- (3) Return **Unsatisfiable**.

Double elimination deletes a clause containing a literal ℓ that occurs exactly twice in the whole sentence. **Single elimination** deletes a clause containing a literal ℓ occurring exactly once in the whole sentence.

Franco's results are as follows: If

$$\left\{ \begin{array}{l} n = r^e \\ 2/3 > e > 0 \\ pr < (\ln n/4)^{1/3} r^{2/3-e} \end{array} \right.$$

or

$$\left\{ \begin{array}{l} n = r^e \\ 1 > e \geq 2/3 \\ pr \leq (1 - e - d) \ln n / e \\ d > 0 \end{array} \right.$$

or

$$\left\{ \begin{array}{l} pn \rightarrow 0 \\ pr < g \ln \ln n \\ g > 0 \end{array} \right.$$

then INFREQ has polynomial average case complexity.

5.3.3 DPE

We mention Policriti and Tetali's result for extended Davis-Putnam. We give the language, distribution, algorithm and result.

The language that they consider is a language of propositions whose proposition letters are ground terms in a decidable sublanguage. Let there be r such terms t_1 through t_r . An **atom** is a relation from the sublanguage applied to some number of terms, and we form disjuncts of atoms and conjuncts of disjuncts as follows:

$$\mathbf{t}(r) ::= t_1 \mid \dots \mid t_r$$

$$\mathbf{A}(r) ::= op(\mathbf{t}(r), \dots) \mid \neg op(\mathbf{t}(r), \dots)$$

$\mathbf{C}(r) ::= \{\mathbf{A}(r)^*\}$ Disjunct

$\mathbf{S}(r) ::= \{\mathbf{C}(r)^*\}$ Conjunct

Let $\#A(r) = R$ ($\#s$ denotes the size of s). We exclude **a priori** occurrences of $\neg A$ and A from a single clause. Write $C(r)$ for the set of clauses, then $\#C(r) = 3^R$.

The distribution they choose is as follows: Let S be a conjunct of n clauses chosen uniformly from $\mathbf{C}(r)$. Each clause has probability 3^{-R} of occurring in S . Clauses may be repeated in S . The algorithm they employ is as follows:

proc $T_{SAT}(K)$:= a decision procedure for a conjunct K of

literals in a decidable sublanguage

proc $T_{DPE}(S) := T_{DPP}(S, \text{True})$

proc $T_{DPP}(S, K) :=$

if $\neg T_{SAT}(K)$ **then** UNSATISFIABLE

elseif $S = \emptyset$ **then** SATISFIABLE

elseif $\emptyset \in S$ **then** UNSATISFIABLE

else $\ell :=$ some literal in S

$(S_1, K_1) := (\{C - \{\neg\ell\} \mid C \in S \wedge \ell \notin C\}, K \wedge \ell)$

$(S_2, K_2) := (\{C - \{\ell\} \mid C \in S \wedge \neg\ell \notin C\}, K \wedge \neg\ell)$

$\neg(T_{DPP}(S_1, K_1) = \text{UNSATISFIABLE} \wedge$

$T_{DPP}(S_2, K_2) = \text{UNSATISFIABLE})$

Let S be a conjunct of n clauses. Recall that clauses are disjuncts chosen over R literals and their negations, without repetitions, but clauses may be repeated in S . Suppose T_{SAT} runs in polynomial time $f(m)$ for a conjunct of literals of size m in the worst case. Let $T(n, R)$ be the average case complexity of $T_{DPE}(S)$ where S is of size n . Then $T(n, R) \leq 4n^2$ for $n > 7$.

5.3.4 Resolution is hard under a different sampling distribution

Chvatal and Szemerédi's paper [CS87] suggests why these algorithms have good performance under the above sampling distributions. In their language, the sampling distributions above produce **dense** (in a sense we will define) formulæ; apparently, **sparse** instances of SAT are hard for resolution-based methods. In this subsection, we will cast their results in the language of the previous results discussed in this section.

Following the above, let $\mathbf{C}(r, k)$ be the set of clauses (disjuncts of literals) containing k literals over r variables. Let $\mathbf{C}'(r, k)$ be the set of clauses in which ℓ and $\neg\ell$ never appear simultaneously in a clause. Let $\mathbf{K}(n, r, k)$ be the n -fold Cartesian product of $\mathbf{C}'(r, k)$. The size of $\mathbf{C}'(r, k)$ is $\binom{r}{k}2^k$. We build a random sentence in $\mathbf{K}(n, r, k)$ by choosing n times from $\mathbf{C}'(r, k)$ with equal likelihood.

Suppose we have a set of n positive integers. If more than $n/2$ of these positive integers are greater than c , then the average of the set is greater than $c/2$. An immediate corollary of a theorem of Chvatal and Szemerédi is that:

Theorem 1 The **average** resolution complexity (and hence, the **average** complexity of the Davis-Putnam procedure) on random sentences in

$$\mathbf{K}(cr, r, k)$$

with

$$\begin{cases} k \geq 3 \\ \frac{c}{2^k} \geq 0.7 \end{cases}$$

is $(1 + \epsilon)^r$ where $\epsilon > 0$.

This contrasts with Goldberg's result, that the average case of Davis-Putnam on randomly generated formulas is easy. The difference is in the way we are constructing a randomly generated formula. In the Chvatal-Szemerédi result, we are constructing sentences consisting of a large set of small fixed-size clauses (e.g. 3-SAT), where the number

of variables appearing in the sentence is proportional to the length of the sentence. This we call a **sparse** sentence. On the other hand, Goldberg constructs sentences with large clauses, each variable having a $2/3$ probability of appearing in any given clause. The main difference is that Chvatal and Szemerédi are fixing the size of the clauses as the length of the sentences and the number of variables grow proportionally, so the proportion of the set of variables occurring in the entire sentence occurring in a random clause of that sentence declines as the number of variables increases. In other words, sparse sentences are those sentences where a large number of distinct variables appear in the sentence (relative to the length of the sentence), yet each clause contains only a small proportion of the total number of variables.

In conclusion, the way Goldberg, Franco and Policriti-Tetali do their sampling essentially avoids the infinite set of hard instances constructed by Chvatal and Szemerédi. Sparse formulas with bounded-length clauses are hard, dense formulas seem easy. This brings us to the problem of understanding of the average case complexity of SAT.

5.4 The theory of average case complexity

In complexity theory, the problem of the average-case complexity of deciding the satisfiability of propositional sentences is generally referred to as *SAT*, and it is the canonical NP-Complete problem [GJ79]. It also happens to be a problem around which a theory of average case complexity of considerable subtlety has grown ([Lev86], [BDCGL89], [BG91], [Gur91], [VR92], [SY92], [RS93]). This theory, originated by the Russian mathematician Leonid Levin around 1984, arises from the study of complexity classes. While the work of Goldberg, Franco, Policriti and Tetali, and Chvatal and Szemerédi on the Davis-Putnam procedure concentrated on the particular algorithm and the effect of a given distribution on that algorithm, the Levin-inspired work focuses on the problem, not the algorithm, and on the class of problems of which SAT is a member, and in so doing makes deeper observations about the complexity of any

member of this class.

In this section, we:

- Review the rationale for the somewhat non-intuitive definitions of the new theory of average-case complexity.
- Give the structure of the theory, sufficient to state its results concerning the average case complexity of *SAT*.
- Define the average case complexity class of *SAT*, and show how *SAT* is distinguished among NP-Complete problems with respect to average case complexity. This involves the notion of a distributional problem.
- This approach gives the conditions on reductions from a class of distributional problems to *SAT* sufficient to characterize *SAT*'s average-case complexity. We give a distributional reduction of *SAT* to *EMLS* from which we may conclude that *EMLS* has at best the average-case complexity of *SAT*.

5.4.1 Rationale for the new theory

The traditional notion of the average-case complexity of a machine M is the average of the time cost of the procedure for all instances of problems of a given size n , weighted by the probability $\mu_n(x)$ of a given instance x , and divided by the number of instances of that size. We write this as

$$Time_M^\mu(n) = \sum_{|x|=n} \mu_n(x) \, time_M(x)$$

where $time_M(x)$ is the actual number of steps needed for M to halt on input x . It turns out that this definition of average-case complexity is inadequate. Ben-David **et. al.** [BDCGL89] note:

...naive formulations of these definitions suffer from serious problems such as not being closed under functional composition of algorithms, being model dependent, encoding dependent etc.

Regarding the functional composition and model-dependence criticism, Gurevich [Gur91] claims that it is easy to find M such that $Time_M^\mu(n)$ is polynomial in n , but where, if M computes $f(x)$, and M^2 computes $f(f(x))$, $Time_{M^2}^\mu(n)$ is no longer polynomial in n . Reischuk **et.al.** also have a problem with the possibility that, since μ is subscripted by n , it could be piecewise-defined so that qualitatively different measures are used for inputs of different sizes.

Such considerations have resulted in a more evolved definition of average-case complexity of a machine. Let $\mu(x)$ be a distribution function defined over the space of all inputs x of any size. Let $T_M(n)$ be such that $Time_M^\mu(n) \leq T_M(n)$ for all $n \geq 0$. Then M is called Levin- μ -average T_M -time bounded if

$$\sum_x \mu(x) \frac{T_M^{-1}(time_M(x))}{|x|} \leq 1$$

Example 5.1 Reischuk **et.al.** find that there are still problems with this definition, namely that the functional growth of $\mu(x)$ influences the time bound T [RS93]:

If, for example, one takes the “standard” **uniform probability distribution**, which assigns probability $\mu_{\text{uniform}}(x) := 6/\pi^2 \cdot |x|^{-2} \cdot 2^{-|x|}$ to a string $x \in \{0, 1\}^*$ a machine using n^2 steps on every input of length n would already be average $O(n^{1+\epsilon})$ -time bounded for arbitrary $\epsilon > 0$.

So instead of Levin- μ -average T_M -time boundedness, they propose a hierarchy of average-case complexity classes whose members are **distributional problems** (L, μ) where L is a language and μ is a distribution upon which is imposed, in addition, a restriction on monotone transformations of μ . We will give the more restricted definition and define some of these classes as we proceed. However, the price for this more accurate theory will be a considerable loss in “intuitiveness” over the original, simple definitions.

The idea of the results of Reischuk **et.al.**’s theory of average case complexity is as follows

- We can ask for the kinds of inputs we see in practice, what is the average case complexity for those kinds of inputs?

- For the space of all inputs, we attach a much higher weight to the inputs we expect to see. This gives us the notion of a distribution function on the set of inputs. We can now define a **distributional problem** to be a decision problem together with the probability distribution function on the instances of that problem, and seek algorithms that have good average-case behavior with respect to this distribution function.
- The initial work in average case complexity sought to bound the computational complexity of the distribution function in various ways because distribution functions that were computed using a lot of computational resources could artificially select the hard instances of a problem with respect to a given algorithm as being most likely, thereby frustrating a given algorithm. So the idea was to use distribution functions that weren't too complex, either in terms of computing the distribution function or computationally sampling from it. However, restricting an average case analysis to simple distributions may yield results of little practical value. According to Mitchell, Selman and Levesque, this is because the distributions they describe do not reflect the range of inputs occurring in practice in Artificial Intelligence applications [DML92].
- The functional growth of the distribution function still affected and led to **poor** estimates of the average case running time of a given algorithm (see Example 5.1 above). By **poor** we mean that our average case complexity determination is not of the same order as the actual performance of our algorithm on the given distribution of inputs.
- In order that the functional growth of the distribution function not influence the upper bound estimate on the average time complexity of a given Turing machine, Reischuk and Schindelbauer bound the complexity of the machine with respect to all monotone transformations on the distribution function. This

is shown to be equivalent to the notion of the **rankability** of the distribution function.

- Let the **rank** of an input be the number of elements in the input space of higher probability than a given input, according to the distribution function. We will characterize distribution functions based on the amount of resources needed to compute the rank of any given input.

5.4.2 Average case complexity classes

Let $\Sigma = \{0, 1\}$. Let $\Sigma^* = \cup_{n \geq 0} \Sigma^n$ be the set of all strings. A **language** is a subset of Σ^* . Let $\text{bin} : \mathcal{N} \rightarrow \Sigma^*$ be a mapping of natural numbers to strings, so that $\text{bin}^{-1}(s)$ is the number x of string $s = \text{bin}(x)$ given by the usual correspondence between the binary encoding of a string and the binary number of the same representation.

Let $\mu : \Sigma^* \rightarrow [0, 1]$ be a probability distribution on strings. Then:

- The **rank** of a problem instance x is the number of problem instances with probability greater than x according to probability measure μ , i.e.

$$\text{rank}_\mu(x) = |\{z \in \Sigma^* : \mu(z) \geq \mu(x)\}|$$

This number is always finite, if $\mu(x) > 0$, since if a given instance has probability $p > 0$, then the number of instances that have probability $q \geq p$ is bounded by $\lfloor (1 - p)/p \rfloor$.

- A **complexity bound** is a function $T : \mathcal{N} \rightarrow \mathcal{N}$. We assume that any such T is monotone increasing and time-constructible. Time-constructible means that there is a Turing machine M that on input x runs exactly $T(|x|)$ steps for all $x \in \mathcal{N}$. T is not necessarily injective. We define T^{-1} as it is used below to mean

$$T^{-1}(m) = \min\{n : T(n) \geq m\}$$

- The **rankability** of a problem is the amount of effort needed to compute the rank of an input x . Let T -rankable be the set of all distributions μ for which there exists a deterministic Turing machine M that on input x computes $\text{bin}(\text{rank}_\mu(x))$ in time $T(|x|)$.

Let POL be the set of complexity bounds

$$POL = \bigcup_{k \in \mathcal{N}} O(N^k)$$

and

$$POL\text{-rankable} = \bigcup_{T \in POL} T\text{-rankable}$$

Hence $POL\text{-rankable}$ is the class of distributions rankable in polynomial time.

A **distributional problem** is a pair (L, μ) with $L \subseteq \Sigma^*$ a language and $\mu : \Sigma^* \rightarrow [0, 1]$ a probability distribution.

An injective function $f : \Sigma^* \rightarrow \Sigma^*$ is a **polynomial time reduction from language L_1 to language L_2** if f can be computed in deterministic polynomial time and $x \in L_1 \equiv f(x) \in L_2$.

An injective function $f : \Sigma^* \rightarrow \Sigma^*$ is a **distributional reduction** from the distributional problem (L_1, ρ_1) to the distributional problem (L_2, ρ_2) if:

(1) f is a polynomial time reduction from L_1 to L_2 .

(2) **Domination Condition:**

$$(\exists c_0, c_1 > 0) (\forall x \in \Sigma^*) \rho_2(f(x)) \leq c_0 |x|^{c_1} \rho_1(x)$$

The class of languages NP are those languages for which membership is decidable in polynomial time on a nondeterministic Turing Machine.

$$\text{Let } NP^{dist} = \{(L, \mu) : L \in NP \wedge \mu \in POL\text{-rankable}\}.$$

A distributional problem (L, ρ) is **NP-distributional complete** if $(L, \rho) \in NP^{dist}$ and if for all distributional problems in NP^{dist} there exists a distributional reduction to (L, ρ) .

A language L is **NP-average complete** if $L \in NP$ and for all $L' \in NP$ and $\rho' \in POL\text{-rankable}$ there exists a distribution $\rho \in POL\text{-rankable}$ such that (L', ρ') has a distributional reduction to (L, ρ) . An important result of [RS93] is the following:

Theorem 2 If (L, ρ) is NP-distributional complete then L is NP-average complete.

A real-valued function $m : [0, 1] \rightarrow [0, 1]$ is called **monotone** if $x \leq y$ implies that $m(x) \leq m(y)$. A real-valued monotone function $m : [0, 1] \rightarrow [0, 1]$ is called a **monotone transformation** of distribution μ if $\sum_x m(\mu(x)) \leq 1$.

Define $Av(T)$ to be the set of pairs (f, μ) such that

$$\begin{aligned} f & : \Sigma^* \rightarrow \mathcal{N} \\ \mu & : \Sigma^* \rightarrow [0, 1] \end{aligned}$$

and for all monotone transformations m ,

$$\sum_x m(\mu(x)) \frac{T^{-1}(f(x))}{|x|} \leq 1$$

This definition makes reference to all monotone transformations of a distribution function, and is therefore unnecessarily complex. However, by a result of [RS93], a simple alternative definition is that $Av(T)$ consists of all (f, μ) which have the property that if we choose any integer ℓ ,

$$\sum_{rank_\mu(x) \leq \ell} \frac{T^{-1}(f(x))}{|x|} \leq \ell$$

In this way, each rank function represents a whole set of distributions, those which are equivalent with respect to the definition of $Av(T)$.

Define $DistDTime(T)$ to be the set of pairs (L, μ) such that

$$(\exists DTM M) L(M) = L \wedge (time_M, \mu) \in Av(T)$$

Define $AvDTime(T, C)$ to be the set of L such that

$$(\forall \mu \in C) (L, \mu) \in DistDTime(T)$$

Define AvP as

$$AvDTime(POL, POL\text{-}rankable)$$

Reischuk **et.al.** draw the picture, shown in Figure 5.2, of languages L_i whose recognition problem lies within the worst case complexity class NP and the average case complexity class $AvDTime(T, V\text{-}rankable)$, for varying recognition-problem worst-case time complexities T and ranking-function worst-case time complexities V . $h(T)$ is an upper bound on the complexity of ranking functions so that the problem still an efficient average case algorithm. That is, if we choose distribution functions with greater rankability than $h(T)$, the corresponding distributional problem will have no average case polynomial-time algorithm. In this picture we see:

- L_1 cannot be recognized efficiently on average, even with simple distributions.
- L_2 can be recognized efficiently on average only for distributions with complexity less than h . If there is such a language then $NP \not\subseteq AvP$, since L_2 is in NP but it is not in AvP :
- L_3 can be solved efficiently for all polynomial worst-case time complexity ranking functions. If there is such an L_3 which is NP -average complete then $NP \subseteq AvP$.
- L_4 is such that, if there is an NP -complete language in this part of the picture, then $P = NP$. That is, in this part of the picture, we allow exponential-time computable distribution functions, which can pick out only the hardest instances for a given algorithm, yet the algorithm still performs well on average. This means that there are no hard instances for this algorithm, i.e., it is a polynomial-time algorithm in the worst case.
- A central object of study in this area is then the **nose**, which for languages $L \in NP$ is the largest amount of rankability one can allow such that the average

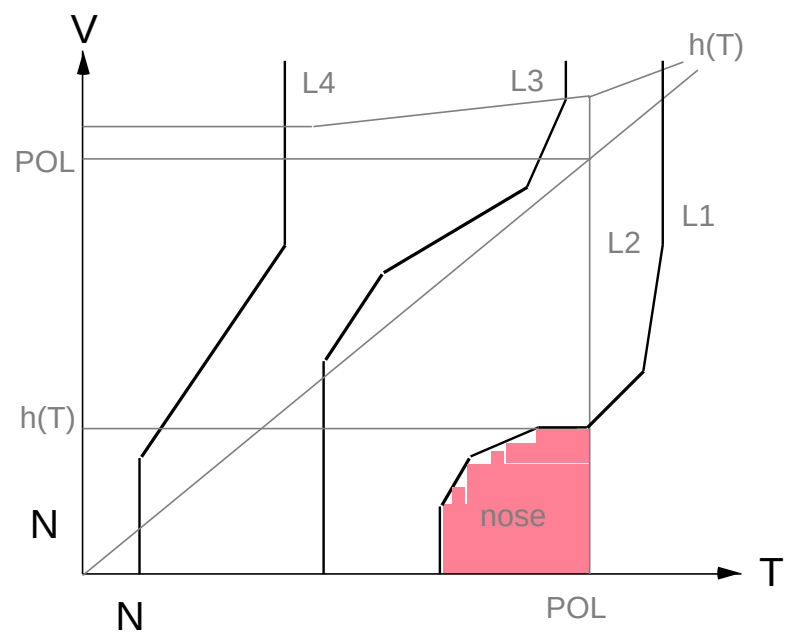


Figure 5.2: Average-Case Complexity of NP -Complete Languages

time complexity stays polynomial. This we may define formally as

$$nose(L) = \{(T, V) \in (POL, POL) : L \in AvDTime(T, V\text{-rankable})\}$$

5.4.3 The average case complexity of *SAT*

Reischuk and Schindelhauer (1993) prove that [RS93]:

Theorem 3 There is a ranking ρ of linear complexity such that the distributional problem (SAT, ρ) is NP-distributional complete. Therefore *SAT* is NP-average complete.

Reischuk and Schindelhauer note that their results have the following implications. Either:

- $NP \subseteq AvP$.
- Every problem in NP can be solved efficiently on average for every polynomial-time rankable distribution.
- Nondeterministic and deterministic exponential time are equal.

Or:

- There are infinitely many linearly-rankable distributions which frustrate every algorithm for *SAT*.

According to Goldberg [Gol79], *SAT* is decidable in $O(n^3)$ time on average under a very simple distribution. What these results strongly indicate is that there are other reasonably simple distributions (such as the one by [CS87]) that will frustrate any algorithm for SAT (e.g. Davis-Putnam), i.e. cause it to have superpolynomial average case behavior. Mitchell et. al. (1992) [DML92] point out that the distributions for which known algorithms have average case polynomial time behavior for *SAT* are somewhat artificial and not the distributions one encounters in Artificial Intelligence applications (e.g. in truth maintenance systems).

5.4.4 The average case complexity of *EMLS*

A polynomial-time invertible injective function f is **honest** if $|f^{-1}(y)| \leq r(|y|)$ for some polynomial r . Then the following theorem from [RS93] will enable us to prove the NP-average completeness of *EMLS*:

Theorem 4 Let f be an injective, polynomial-time invertible, and honest polynomial-time reduction from L_1 to L_2 . Then if L_1 is NP-average complete, then so is L_2 .

Let us define *MLS* to be propositions over *EMLS*. With respect to proving the following theorem, we recall Policriti's reduction f of *SAT* to *MLS* [Pol87a]. Let $\phi \equiv \phi_1 \wedge \dots \wedge \phi_n$ where each of the ϕ_i is a disjunction of literals (i.e., ϕ is in conjunctive normal form). Let m be the number of distinct propositional letters occurring in ϕ , and write the literals $\ell_1, \dots, \ell_m$. Construct $2m$ variables $x_1, \dots, x_{2m}$ in *MLS*, and associate the pair of literals $\ell_i, \neg\ell_i$ with the pair of *MLS* variables x_i, x_{2i} . Also construct n *MLS* variables $y_1, \dots, y_n$ and one variable z . Define ξ in *MLS* to be equivalent to $\xi_1 \wedge \xi_2 \wedge \xi_3 \wedge \xi_4$ where

$$\begin{aligned} \xi_1 &\equiv \bigwedge_{1 \leq i \leq 2m} \{y_j \in x_i : \ell_i \in \phi_j\} \\ \xi_2 &\equiv \bigwedge_{1 \leq i \leq 2m} \{y_j \in x_i : \ell_i \notin \phi_j\} \\ \xi_3 &\equiv \bigwedge_{1 \leq i \leq m} (x_i \in z \equiv x_{i+m} \notin z) \\ \xi_4 &\equiv \bigwedge_{1 \leq j \leq n} \bigvee_{1 \leq i \leq 2m} (y_j \in x_i \wedge x_i \in z) \end{aligned}$$

Subformula ξ_1 tells you which literals occur in which clauses. Subformula ξ_2 tells you which literals do not occur in which clauses. Subformulas ξ_3 and ξ_4 define the set z of all the literals assigned **True** in the satisfying assignment. So ξ_3 ensures that a literal and its complement are not simultaneously assigned **True**, and ξ_4 ensures that at least one literal in each clause is assigned **True**.

Theorem 5 *MLS* is NP-average complete.

Proof. In order to demonstrate that *MLS* is NP-average complete, we must show that Policriti's reduction f of *SAT* to *MLS* [Pol87a] satisfies the conditions of Theorem 4, i.e. that the reduction is injective, polynomial-time invertible and honest. Examination of the construction shows that each propositional formula is mapped to a unique *MLS* formula. Given an instance x of *SAT*, the construction $f(x)$ consists of a conjunct of four separate groups of subformulæ. We can determine syntactically if a given *MLS* formula falls within the range of this reduction mapping f . Further, from the first two groups of subformulæ, we can uniquely reconstruct in polynomial time the instance of *SAT*, x . That is, it tells us exactly which literals appear in each clause of x . Moreover, the length of the first group of subformulæ in the *MLS* instance $f(x)$ is proportional to the length of the original instance of *SAT*, x . Therefore the reduction is honest, that is, $|x| = |f^{-1}(y)| \leq |y| = |f(x)|$. Since *SAT* is NP-average complete by Theorem 3, by the conditions of Theorem 4, *MLS* is NP-average complete. $\square$

As for *SAT*, a consequence of this proof is that there are probability distributions of linear ranking complexity that will frustrate any algorithm for *MLS*, provided that deterministic exponential time is not equal to nondeterministic exponential time. Whether these simple distribution functions correspond to the distribution of instances one gets from Correct Program Technology is an open question.

5.4.5 The average case complexity of *FPILP*

As observed by Jim Cox [private communication],

Theorem 6 *FPILP* is NP-average complete.

Proof. Recall the standard reduction of *SAT* to *FPILP* [GJ79]. Suppose as above the instance of *SAT* consists of $\phi \equiv \phi_1 \wedge \cdots \wedge \phi_n$ where each of the ϕ_j is a disjunction of

literals, over m propositional variables x_i . The corresponding *FPILP* instance consists of the constraints

$$(1) \quad 0 \leq x_i \leq 1, \text{ for } 1 \leq i \leq m.$$

(2) Each ϕ_j is represented by the constraint

$$\left(\sum_{x_i \in \phi_j} x_i \right) + \left(\sum_{\neg x_i \in \phi_j} 1 - x_i \right) \geq 1$$

The reduction is injective, invertible by inspection, and honest, since the lengths of the two problems instances are proportional. Thus, *FPILP* is NP-average complete. $\square$

5.5 Relating expressiveness, proof size and verification time

Consider the problem of verifying that $129 + 1000 = 1129$ using propositional logic extended with the usual axiomatization of primitive recursive addition, i.e. defining $+$ in terms of predecessor and successor functions and an initial element 0. For example, this could easily be coded in PROLOG, and it would be quite slow. On the other hand, if we used an arbitrary-size (hence, logically correct) integer arithmetic package, as is commonly available in languages such as Common Lisp and Setl2, then this verification would take time linear in the size of the expression.

Obviously, for any given problem, some algorithms are better than others. What the above example shows is that, in the realm of theorem proving, some proof construction strategies are more efficient than others in an absolute sense: e.g., proof by breadth-first search on the axioms of primitive recursive addition is less efficient at constructing a proof of an integer arithmetic equality than the use of a correct arbitrary-precision arithmetic package.

These simple observations lead us to the question of whether, for example, by replacing proof over the Zermelo-Frankl axioms for set theory with the description of the actual parameters of a single call to a decision procedure for EMLS, we get a more efficiently checkable proof.

To make this question more precise, we separate the issues of proof search and proof checking. Suppose that we have found a proof of a theorem. The proof checking problem is to establish that one step of a proof follows logically from the previous steps. In a program verification system in which we apply decision procedures for a variety of sublanguages which may arise in particular contexts, each such decision procedure constitutes, in effect, an inference rule. So we can compare the cost of verifying a proof from axioms and syntactic inference rules (in which the cost of checking each step is essentially constant), with the cost of verifying a proof for the same theorem in which some of the steps are justified as applications of particular decision procedures (in which the cost of checking such a step is the cost of running the decision procedure).

In the case of intractable decision problems, for the proof checking problem, we may legitimately ask whether, in general, it is cheaper to check a short proof containing applications of decision procedures as inference rules, than it is to check a very long proof in which each step proceeds from purely syntactic construction on previous steps. An unexamined assumption of the Correct Program Technology approach to program verification is that proofs containing appeals to decision procedures **are** more efficient.

To our mind, the most tempting conjecture is that there is **no** advantage, in proof checking, in choosing to construct short proofs in which checking some steps requires the application of a decision procedure, over longer proofs with constant-time checkable steps. However, the example with which we started this section shows that this is not the case for relatively easy problems. In addition, for any **particular** theorem, it is always possible to add an axiom or inference rule which makes a particular theorem easy, or even denumerably many theorems of a particular form easy. We can try to fix our conjecture in one of two ways:

- By stipulating that a decision procedure that we add behaves uniformly worst-case on all of the inputs to which it applies. (This prevents special-case tuning

the decision procedure.)

- By stipulating that our conjecture applies to infinitely many proofs, but not all proofs. The implication is that, for fixed n , the number of proofs of size n for which the conjecture holds always dominates the number of proofs for which the conjecture is frustrated by some kind of special-casing.

The latter approach is more satisfying. We will thus restrict ourselves to the following, more hedged conjecture, restricted to languages whose decision problem is NP-Complete (the terms in the conjecture are defined in the remainder of the section):

Conjecture 5.1 Suppose a system of computational logic Σ has as its theorems exactly the theorems of an NP-Complete decidable theory T . Let Π be a proof of a theorem θ within Σ . Suppose that Π is Kolmogorov-random [LV88], in the sense that no other proof in Σ of θ is shorter in length than Π . Let the length of Π be m . Let Σ' be another computational logic proving exactly the theorems of T , which contains, in addition to the axioms and inference rules of Σ , some inference rules of the form “Follows from an application of decision procedure D ”. Let Π' be a shortest-length proof of θ in Σ' . Let the length of Π' be n . Let the cost of checking proof step i in Π be c_i , and the cost of checking proof step j in Π' be c'_j . Then, for infinitely many such $\langle \Pi, \Pi' \rangle$ for any fixed such $\langle S, S' \rangle$,

$$\sum_{i=1}^m c_i = \Theta \left(\sum_{j=1}^n c'_j \right)$$

Finally, to define our terms, let a **computational logic** S be a sextuple $\langle L, A, \alpha, I, \beta, C \rangle$ where

- L is the set of well-formed formulae.
- $A = \langle A_1, \dots, A_p \rangle$ are the **axioms**, which are **functions**

$$A_i : L^{\alpha_i} \rightarrow L$$

from well-formed formulae to well-formed formulae. α is thus a p -vector of natural numbers (we allow an A_i to be 0-ary).

- $I = \langle I_1, \dots, I_q \rangle$ are the **inference rules**. $I_i \subseteq L^{\beta_i}$ is a β_i -ary **relation** on well-formed formulae. β is thus a q -vector of positive integers.
- The set of **proofs** P of S is the subset of L^* , the set of sequences of well-formed formulae, such that if $\langle w_1, \dots, w_n \rangle$ is a proof, then each w_i is either
 - (1) An application of some axiom to some elements of L .
 - (2) Such that the tuple formed by some choice in some order from the $w_1, \dots, w_{i-1}$ together with w_i is a member of some inference rule.
- The set of **theorems** in S are just the i_n such that $\langle i_1, \dots, i_n \rangle$ is a proof of S .
- C is the characteristic function of the set of correct proofs of S computed by a Turing Machine program which always halts in an accepting state whenever it is started with the Gödel number of a proof in S expressed on its input tape, and always halts in a rejecting state otherwise.

Chapter 6

Elementary relational languages

We will begin this chapter by defining, computationally, a language of countable, constructable sets, and then a representation of elementary relational languages [Bri77].

We then describe a theorem-proving context that we assume for program verification. In this context, problems are reduced to deciding a sentence of an **elementary positive conjunct parametrised propositional language** (EPCPPL). We say why this is a good idea, and then we define the construction and counting of EPCPPL's. To round out our presentation of elementary relational languages in a model-theoretic sense, we define computational notions of realization, denotation and model for EPCPPL's.

As a heuristic, we want to simplify EPCPPL sentences syntactically. We define the concept of an EPCPPL pattern, and precisely state the match-counting problem for such patterns. (It will turn out, in Chapter 7, that this problem is $\#P$ -Complete, in the sense defined in [GJ79].) From such patterns we will define syntactic simplification rule sets which, when they apply, reduce the complexity of a sentence by eliminating some or all of the variables (the latter in case the sentence can be determined to be invalid in a purely syntactic fashion). It will turn out, in Chapter 8, that this does not pragmatically affect the average case complexity of the match counting algorithm. This is not surprising since:

- We have already determined, in the previous chapter, that the underlying problem of deciding sentences in *EMLS* is NP-average complete.

- The number of sentences grows as $2^{p(n)}$ in the number n of variables where $p(n) = 4n^2 + 2n^3$ so any effective rule set will have to simplify $2^{O(n^3)}$ sentences to have an effect on the average case under a uniform distribution of inputs. It is not clear that there is any valid such syntactic simplifier rule set.

So in those senses, all of this work is for naught, but nevertheless as logicians we find the ability to practical experiment with constructing and counting these structures to be aesthetically pleasing and intuitively rewarding.

6.1 Finite structure descriptions

Now we will define a language of finite mathematical structures. We describe certain finite structures by naming them. For example, the power set of pairs of positive integers from 1 to n is

`PowerSet[CartesianProduct[Pos[n],2]]`

In *Gedanken* we define two operations,

`Construct[D]`

and

`Cardinality[D]`

on such mathematical structure descriptions D . `Construct` builds the set described by D . `Cardinality` computes the number of elements in the set described by D . These are defined symbolically, so that we can compute the cardinality in the abstract. For example the following symbolically evaluates to 2^{n^2} :

`Cardinality[PowerSet[CartesianProduct[Pos[n],2]]]`

We were able to compute the above cardinality in time linear in the size of the description, whereas the construction operation would take time 2^{n^2} . This is a nice effect. This

is why we separate the operations of **Construct** and **Cardinality**. Unfortunately, it will not always be the case for all describable finite mathematical structures that counting is much easier than construction, for example, in the case of match counting to be encountered at the end of this chapter. Nevertheless, it is still a nice idea to experiment with, because it allows us to build intuition about the relative hardness of counting and construction.

Now we provide more details. In **Gedanken**, a mathematical structure description D describes a structure which is a set or vector. D is any one of:

- A list x . The construction **Construct** $[x]$ of a list x is x . The cardinality of a list x is just its Mathematica **Length**. For example, in **Gedanken**:

```
Cardinality[{1,2,5}]
```

```
3
```

- **Nat** $[n]$, n an integer, describing the integers from 0 to n , formally **Nat** $[n] = \{i : 0 \leq i \leq n\}$. **Cardinality** $[\text{Nat}[n]] = n + 1$.
- **Pos** $[n]$, n an integer, describing the integers from 1 to n , formally **Pos** $[n] = \{i : 1 \leq i \leq n\}$. **Cardinality** $[\text{Pos}[n]] = n$. For example, in **Gedanken**:

```
Construct[Pos[12]]
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}
```

- **Vector** $[m, k]$, m, k integers. This describes the m -vector of elements $\langle k \rangle_{i=1}^m$.
Then

$$\text{Cardinality}[\text{Vector}[m, k]] = m$$

For example, in **Gedanken**:

`Construct[Vector[3,2]]`

`{2, 2, 2}`

`Cardinality[Vector[2,3]]`

2

- `PowerSet[D']`, where D' is a mathematical structure description describing a set, and where S is the set described by D' , describes the power set $2^S = \{T : S \supseteq T\}$. Then

$$\text{Cardinality}[\text{PowerSet}[D']] = 2^{\text{Cardinality}[D']}$$

For example, in Gedanken:

`Construct[PowerSet[Nat[2]]]`

`{{}, {0}, {1}, {2}, {0, 1}, {0, 2}, {1, 2}, {0, 1, 2}}`

`Cardinality[PowerSet[Nat[32]]]`

8589934592

- `CartesianProduct[{D'_1, ..., D'_n}]`, where the description D'_i for $1 \leq i \leq n$ describes set S_i , describes the Cartesian product

$$\times_{i=1}^n S_i = \{(p_1, \dots, p_n) : p_i \in S_i\}$$

If $\#S_i = n_i$, then

$$\text{Cardinality}[\text{CartesianProduct}[\{S_1, \dots, S_n\}]] = \prod_{1 \leq i \leq n} n_i$$

For example, in Gedanken:

`Construct[CartesianProduct[{{a,b}, {c}}]]`

`{{a, c}, {b, c}}`

`Cardinality[CartesianProduct[{{a,b}, {c}, {d,e,f}}]]`

6

- `CartesianProduct[ $D'$ ,  $k$ ]`, where D' is a mathematical structure description describing a set S , and k an integer, describes the k -fold Cartesian product S_k .
If $\#S = n$, then

$$\text{Cardinality}[\text{CartesianProduct}[S, k]] = n^k$$

For example, in Gedanken:

`Construct[CartesianProduct[{0,1},3]]`

`{{0, 0, 0}, {0, 0, 1}, {0, 1, 0}, {0, 1, 1}, {1, 0, 0},
{1, 0, 1}, {1, 1, 0}, {1, 1, 1}}`

`Cardinality[CartesianProduct[{0,1},3]]`

8

6.2 Elementary relational structure

An **elementary relational structure** (ers)

$$U = \langle A, \langle R_1, \dots, R_L \rangle, \langle C_1, \dots, C_M \rangle, B \rangle$$

is a quadruple such that A is a nonempty set, called the domain, there are L B_i -ary relations R_i and there are M constants C_i . We define the **elementary type** $\delta(U)$ of

U as the ordered pair $\langle M, B \rangle$. For example, in **Gedanken**, here is an ers of sets with relations $i = j$ and $i = j \setminus k$:

```
SED = ERS["Sets", {#1==#2&, #1==Complement[#2, #3]&}, {{}}, {2,3}]
```

In **Gedanken**, for an ers U , we define

- $\text{Domain}[U] = A$.
- $\text{Relns}[U] = \langle R_1 \dots R_L \rangle$.
- $\text{Consts}[U] = \langle c_1 \dots c_M \rangle$.
- $\text{Arity}[U] = \beta$.
- The **elementary type** $\text{delta}[U] = \langle m, \beta \rangle$, where m is the number of constants in $\mathcal{U}$ and β is a vector of the arities of its relations. For example, in **Gedanken**, using the previously defined ers **SED**,

```
delta[SED]
```

```
ElementaryType[1, {2, 3}]
```

6.3 Theorem proving paradigm we assume

Now we will define the theorem proving paradigm that we're going to assume:

- (1) We want to prove $\vdash \forall x. \phi(x)$.
- (2) This reduces to proving $\neg \models \exists x \neg \phi(x)$, that is, $\exists x \neg \phi(x)$ is not satisfiable.
- (3) $\neg \phi(x)$ is transformed into conjunctive normal form: A conjunct of disjuncts of literals which are negated and non-negated atoms.
- (4) A conjunct C is then formed by picking one literal from each disjunct.

- (5) This conjunct is rendered into “elementary positive” form, to be described in the next section. We will call a language of sentences in this form an **elementary positive conjunct parametrised propositional language**, or EPCPPL. E.g., $ax+b \neq cd+w$ is rendered as $m = cd \wedge n = m+w \wedge p = ax \wedge q = p+b \wedge n \neq q$. Then:

- “Positive” means that negation is pushed down into the relation set, and
- “Elementary” means that there are no subexpressions: The arguments to relations are just constants and variables.
- “Parametrised propositional” means that, for a fixed number n of variables, there are a fixed number of literals, and hence the language is effectively propositional for each n .

The benefits of studying EPCPPL’s in this paradigm are that:

- (1) The complexity $T(n)$ of deciding a conjunct of elementary positive literals, as a function of the “degree” n (that is, the number of distinct variables in the conjunct), is a more natural measure of the difficulty of deciding a class of relations, than, say, where n is the number of literals in the conjunct, or the syntactic length of the expression. This is because our experience is that degree in the above sense is the key parameter of time complexity.
- (2) In holding $Q \wedge Q$ to be identical to Q (that is, conjuncts of literals are represented as sets), we are implying a “choice model” in which conjuncts can be viewed as boolean n -vectors, where n is the number of distinct atoms in the language, as defined earlier. This brings out the naturalness of using n , the number of variables, as the key complexity parameter, since the relations and constants are typically fixed, so big N is a function of little n ($N = f(n)$) in practice, and we see that the general decision problem is then one of determining the

satisfiability in a realization of a given elementary relational structure of an interpretation of any given Boolean $f(n)$ -vector.

- (3) If we dealt with a language in which, for a given number of variables, there were an infinite number of terms, for example $x = e + e$, $x = e + (e + e) \dots$ and so on, then it would be harder to study the matching problem for simplification rules, which is intuitively easier to think about when, for a given n , we are in a setting where there are only a finite and precisely computable number of elements of the language to which a given simplification rule R can apply. This setting gives us a more natural and definite measure of the effectiveness of a given simplifier rule, since we know both the total number of elements degree n , and the number of such elements which match rule R .

6.4 EPCPPLs

Let N be the number of variables. Let $\delta = \langle m, \langle \beta_1, \dots, \beta_k \rangle \rangle$ be the type of an elementary relational structure with k relations and m constants. The **elementary positive conjunct parametrised propositional language** (epcppl) of degree n is then given by the grammar

$$\begin{aligned}
\mathbf{Conjunct} &::= \{\mathbf{Atom}^*\} \\
\mathbf{Atom} &::= \mathbf{Relation}_i(\mathbf{Term}_1, \dots, \mathbf{Term}_{\beta_i}) \\
\mathbf{Relation} &::= r_1 \mid \dots \mid r_k \\
\mathbf{Term} &::= \mathbf{Id} \mid \mathbf{Const} \\
\mathbf{Id} &::= x_1 \mid \dots \mid x_n \\
\mathbf{Const} &::= c_1 \mid \dots \mid c_m
\end{aligned}$$

Let us pause to understand the adjectives in “epcppl”:

Syntactic Category	Constructors	Purpose
EPCPPL $[\delta]$ $[n]$	Conjunct $[a_1, \dots]$	Conjuncts of atoms
	Conjunct $[\]$	Empty conjunct
EPCPPLVars $[\delta]$ $[n]$	IndVar $[j]$	Individual vars
EPCPPLConsts $[\delta]$ $[n]$	IndConst $[j]$	Individual consts
EPCPPLTerms $[\delta]$ $[n]$	IndVar $[j]$	Individual terms
	IndConst $[j]$	
EPCPPLPredLets $[\delta]$ $[n]$	PredicateLetter $[j]$	Predicate letters
EPCPPLTermSubsts $[\delta]$ $[n]$		Term substs

Figure 6.1: Abstract syntax of elementary languages in Gedanken

- **Elementary.** Terms are not built up from operations on terms, that is, terms are of constant size.
- **Positive.** There is no negation relation on atoms.
- **Conjunct.** There is no disjunction in the language.
- **Parametrised propositional.** If $n = \infty$, then the above language would be called first order. If n is finite, then we use the phrase parametrised propositional to call attention to the fact that the language is equivalent to propositional calculus over $Q = \sum_{i=1}^k (m + n)^{\beta_i}$ propositional letters. There are exactly 2^Q distinct sentences in this language, when conjuncts are interpreted as **sets** of atoms (i.e., order of atoms is unimportant, and an atom occurs only once in a conjunct).

In Gedanken we define the **abstract syntax** for this class of languages in Gedanken shown in Figure 6.1. We use the term abstract syntax in the tradition of denotational semantics, to give the essential features of a language from which inessential details necessary for parsing are elided (the latter form of syntax being called a concrete syntax) [Gor79, §3.1]. In this figure, $\delta = (m, \beta)$ is assumed to be the type of an elementary relational structure: Note that, referring to the figure, EPCPPLTermSubsts is not really a syntactic category, but is built up from a syntactic category (it will be explained later).

The syntactic categories may be defined with respect to their constructors and other syntactic categories as follows (recall that 2^S , where S is a set, is our notation for the power set of S):

$$\begin{aligned}
\text{EPCPPL}[\delta][n] &= 2^{\text{EPCPPLAtoms}[\delta][n]} \\
\text{EPCPPLAtoms}[\langle m, \beta \rangle][n] &= \\
&\{R_i(t_1, \dots, t_{\beta_i}) : 1 \leq i \leq L \wedge t_j \in \text{EPCPPLTerms}[\delta][n]\} \\
\text{EPCPPLTerms}[\delta][n] &= \text{EPCPPLConsts}[\delta][n] \cup \text{EPCPPLVars}[\delta][n] \\
\text{EPCPPLPredLets}[\langle m, \langle n_1 \dots n_L \rangle \rangle][n] &= \{R_i : 1 \leq i \leq L\} \\
\text{EPCPPLVars}[\delta][n] &= \{\text{IndVar}[j] : 1 \leq j \leq n\} \\
\text{EPCPPLConsts}[\langle m, \beta \rangle][n] &= \{\text{IndConst}[j] : 1 \leq j \leq m\} \\
\text{EPCPPLTermSubsts}[\delta][n] &= (\text{EPCPPLTerms}[\delta][n])^n
\end{aligned}$$

The representation of variables V_i in our subsequent coding will be as their integer indices i , so we define $\text{IndVar}[i] = i$. We write an individual predicate letter as $\text{PredicateLetter}[i] = i$, where predicate letters R_i have arity β_i . A term is a variable or constant. An atom is formed by applying a predicate letter to terms.

The operation $\text{Construct}[S]$ on an abstract syntax category S builds the set of elements of S . We extend this operation, defined earlier in this chapter to EPCPPL abstract syntax categories, by structural induction in the obvious way. We will give examples in the next subsection.

We extend the operation $\text{Cardinality}[S]$ defined earlier in this chapter to EPCPPL abstract syntax categories. Let us first define in *Gedanken* a realization that we will use for subsequent examples:

```

SED = ERS["Sets",
  {#1 == #2 & , #1 == Complement[#2, #3] & },
  {{}},
  {2,3}]

```

Now let us define **Cardinality** by structural induction as follows:

- The size of $\text{EPCPPLVars}[\delta][n]$ is n . For example, in Gedanken:

$\text{Construct}[\text{EPCPPLVars}[\text{delta}[\text{SED}]]][3]$

$\{1, 2, 3\}$

$\text{Cardinality}[\text{EPCPPLVars}[\text{delta}[\text{SED}]]][3]$

3

- The size of $\text{EPCPPLConsts}[\delta][n]$ is m . For example, in Gedanken:

$\text{Construct}[\text{EPCPPLConsts}[\text{delta}[\text{SED}]]][3]$

$\{\text{IndConst}[1]\}$

$\text{Cardinality}[\text{EPCPPLConsts}[\text{delta}[\text{SED}]]][3]$

1

- The size of $\text{EPCPPLTerms}[\delta][n]$ is $n + m$. For example, in Gedanken:

$\text{Construct}[\text{EPCPPLTerms}[\text{delta}[\text{SED}]]][3]$

$\{\text{IndConst}[1], 1, 2, 3\}$

$\text{Cardinality}[\text{EPCPPLTerms}[\text{delta}[\text{SED}]]][3]$

4

- The size of $\text{EPCPPLPredLets}[\langle m, \langle n_1 \dots n_L \rangle \rangle][n]$ is L . For example, in Gedanken:

`Construct[EPCPPLPredLets[delta[SED]]]`

`{1, 2}`

`Cardinality[EPCPPLPredLets[delta[SED]]]`

2

- The size of `EPCPPLTermSubsts[δ][k]` is $(n + m)^k$. For example, in Gedanken:

`Construct[EPCPPLTermSubsts[delta[SED]][2]]`

`{{IndConst[1], IndConst[1]}, {IndConst[1], 1},
 {IndConst[1], 2}, {1, IndConst[1]}, {1, 1}, {1, 2},
 {2, IndConst[1]}, {2, 1}, {2, 2}}`

`Cardinality[EPCPPLTermSubsts[delta[SED]][2]]`

9

- The size of `EPCPPLAtoms[⟨m, β⟩][n]` is $\sum_{1 \leq i \leq L} (n + m)^{\beta_i}$. Another way of describing β is as a vector of length L of k 's, in case every relation in the language has the same arity. We use our language of mathematical structures to do so, the resulting description being `EPCPPLAtoms[⟨m, Vector[L, k]⟩][n]`. In this case, the size of the syntactic category is $L(n + m)^k$. For example, in Gedanken:

`Construct[EPCPPLAtoms[delta[SED]][1]]`

`{1[IndConst[1], IndConst[1]], 1[IndConst[1], 1],
 1[1, IndConst[1]], 1[1, 1],
 2[IndConst[1], IndConst[1], IndConst[1]],`
 132

```

2[IndConst[1], IndConst[1], 1],
2[IndConst[1], 1, IndConst[1]], 2[IndConst[1], 1, 1],
2[1, IndConst[1], IndConst[1]], 2[1, IndConst[1], 1],
2[1, 1, IndConst[1]], 2[1, 1, 1]}

```

Cardinality[EPCPPLAtoms[delta[SED]] [1]]

12

- The empty conjunct `Conjunct[]` is valid. Any subset of atoms thus denotes a conjunct. The conjuncts are the well-formed formulæ

$$\text{EPCPPL}[\delta][n]$$

The size of

$$\text{EPCPPL}[\delta][n]$$

is thus

$$2^{\#\text{EPCPPLAtoms}[\delta][n]}$$

For example, in Gedanken:

Construct[EPCPPL[delta[SED]] [0]]

```

{Conjunct[], Conjunct[1[IndConst[1], IndConst[1]]],
  Conjunct[2[IndConst[1], IndConst[1], IndConst[1]]],
  Conjunct[1[IndConst[1], IndConst[1]],
  2[IndConst[1], IndConst[1], IndConst[1]]]}

```

Cardinality[EPCPPL[delta[SED]] [6]]

```

1008691358627698667834342656367651341004132532391543469\
9476311148690477350328591652205216125053840404649676551\

```

- The **stratified** size k of $\text{EPCPPL}[\delta][k]$, for $k \leq n$, written

$$\#\text{Stratified}[\text{EPCPPL}[\delta][k]]$$

is the number of conjuncts of $\text{EPCPPL}[\delta][k]$ of degree exactly k , which equals $\#\{\pi : \pi \in \text{EPCPPL}[\delta][k] \wedge \deg(\pi) = k\}$. If $k = 0$, then

$$\#\text{Stratified}[\text{EPCPPL}[\delta][k]] = \#\text{EPCPPL}[\delta][0]$$

Otherwise, we choose k of n variables, and for each such choice, the number of sentences of degree k formed from those k variables is the number of sentences of any degree formed from those k variables, less the number of sentences of degree $k - 1, k - 2, \dots, 0$ that may be formed from k variables. That is, the following relationship holds:

$$\begin{aligned} \#\text{Stratified}[\text{EPCPPL}[\delta][k]] = \\ \#\text{EPCPPL}[\delta][k] - \sum_{i=1}^k \binom{k}{i} \#\text{Stratified}[\text{EPCPPL}[\delta][k-i]] \end{aligned}$$

This relationship will seen later, so we define the following recursive functional:

$$\begin{cases} \text{Stratify}(F)(0) &= F(0) \\ \text{Stratify}(F)(n > 0) &= F(n) - \sum_{i=1}^n \binom{n}{i} \text{Stratify}(F)(n-i) \end{cases}$$

and so

$$\#\text{Stratified}[\text{EPCPPL}[\delta]][k] = \text{Stratify}(\#(\text{EPCPPL}[\delta]))[k]$$

For example, in Gedanken:

`CardinalityStratified[EPCPPL[delta[SED]]][6]`

100869135862769866783434342656367651341003698312056903534\

2147927236752649412912554968840433014055554931777313806\

7431420

We define a function **Occurs** which gives the set of indices of variables occurring in a sentence. We define it by cases as follows:

$$\begin{aligned}
\text{Occurs}[V_j] &= \{j\} \\
\text{Occurs}[C_j] &= \{\} \\
\text{Occurs}[R_i()] &= \{\} \\
\text{Occurs}[R_i(T_1, \dots, T_{n_i})] &= \bigcup_{1 \leq k \leq n_i} \text{Occurs}[T_k] \\
\text{Occurs}[\{A_1, \dots, A_j\}] &= \bigcup_{1 \leq k \leq j} \text{Occurs}[A_k]
\end{aligned}$$

For example, in Gedanken:

```
Occurs[Conjunct[
  PredicateLetter[6][
    IndConst[1], IndVar[2], IndVar[2]]]]
{2}
```

A **term substitution** s is an n -vector of terms, one for each variable in the language. We define, by cases, a function **TermSubstitute** which uses a term substitution to substitute terms for variables in sentences:

$$\begin{aligned}
\text{TermSubstitute}[s][V_i] &= s_i \\
\text{TermSubstitute}[s][C_j] &= C_j \\
\text{TermSubstitute}[s][R_i()] &= R_i \\
\text{TermSubstitute}[s][R_i(T_1, \dots, T_{n_i})] &= R_i(\text{TermSubstitute}[s][T_1], \dots, \\
&\quad \text{TermSubstitute}[s][T_{n_i}])
\end{aligned}$$

We say that one conjunct **precedes** another if it occurs earlier in a lexicographic ordering of patterns. We define a well order relation

$$\text{Precedes}[a_1, a_2]$$

to encode this notion. For example, in Gedanken:

`Precedes[Conjunct[1[], 2[], 3[]], Conjunct[1[], 2[]]]`

`False`

`Precedes[Conjunct[1[], 2[], 3[]], Conjunct[1[], 3[]]]`

`True`

Define the **degree** of a conjunct $\pi \in \text{EPCPPL}[\delta][n]$, written $\text{Deg}[\pi]$, to be the number of distinct variables V_i occurring in π . That is,

$$\text{Deg}[\pi] = \# \bigcup \{ \{t_i : t_i = V_i\} : R_i(t_1, \dots, t_{\beta_i}) \in \pi \}$$

For example, in *Gedanken*:

`Deg[Conjunct[6[IndConst[1], 2, 2], 1[1, 1]]]`

`2`

A conjunct p is in canonical form `Canonical[p]` if its terms are listed in ascending well order, and if a minimal number of variables is used to express the conjunct, with the lowest-numbered variable occurring first. For example, in *Gedanken*:

`Canonical[Conjunct[1[5], 1[], 3[]]]`

`Conjunct[1[], 1[1], 3[]]`

`Canonical[Conjunct[3[], 1[], 3[]]]`

`Conjunct[1[], 3[]]`

6.5 Realization, Denotation and Model

A given relational structure $\mathcal{U}$ of type δ is called a **realization** for

$$\text{EPCPPL}[\delta][n]$$

For purposes of examples below, let us construct the following realization in Gedanken:

$$\text{EQ} = \text{ERS}["\text{Sets}", \{\#1 == \#2\}, \{"K"\}, \{2\}]$$

Let $\alpha \in A^n$ where A is the domain of relational structure $\mathcal{U}$ of type δ . The **denotation** of a term t in

$$\text{EPCPPL}[\delta][n]$$

with respect to α ,

$$\text{Denotation}[\mathcal{U}, \alpha][t]$$

may be defined by cases as follows:

- If t is V_i , then

$$\text{Denotation}[\mathcal{U}, \alpha][t] = \alpha_i$$

For example, in Gedanken:

$$\text{Denotation}[\text{EQ}, \{\{1,2\}, \{a,b\}\}][\text{IndVar}[2]]$$

$$\{a, b\}$$

- If t is C_k , then $\text{Denotation}[\mathcal{U}, \alpha][t] = c_k$. For example, in Gedanken:

$$\text{Denotation}[\text{EQ}, \{\{1,2\}, \{a,b\}\}][\text{IndConst}[1]]$$

$$K$$

Let $\pi \in \text{Conjuncts}[\delta, n]$. Then α **satisfies** π , written

$$\text{Satisfies}[\mathcal{A}, \alpha][\pi]$$

if π is “true under α ”. We define this by cases as follows:

$\text{Satisfies}[\mathcal{U}, \alpha][R_i(t_1, \dots, t_{\beta_i})] \equiv$
 $\langle \text{Denotation}[\mathcal{U}, \alpha][t_1], \dots,$
 $\text{Denotation}[\mathcal{U}, \alpha][t_{\beta_i}] \rangle \in R_i$
 $\text{Satisfies}[\mathcal{U}, \alpha][\mu_1 \wedge \dots \wedge \mu_m] \equiv$
 $(\forall 1 \leq j \leq m) \text{Satisfies}[\mathcal{U}, \alpha][\mu_j]$

For example, in `Gedanken`:

```

Satisfies[EQ, {{1,2},{a,b}}][
  Conjunct[PredicateLetter[1][IndVar[1], IndVar[1]]]]

True

```

Let $\pi \in \text{EPCPPL}[\delta, n]$. We say that π is **valid** iff for all $\alpha \in A^n$, $\mathcal{U} \models_\alpha \pi$. We then say that the relational structure $\mathcal{U}$ is a model for π . We make no other use of the term “model”.

We say that π is satisfiable iff for some realization $\mathcal{U}$ of $\text{EPCPPL}[\delta, n]$ and some sequence $\alpha \in A^n$ where $A = \text{Domain}[\mathcal{U}]$, $\mathcal{U} \models_\alpha \pi$. In this case, we also say that α is a satisfying assignment for π . If π is not satisfiable, then we say that π is invalid.

6.6 Patterns and match-counting

We define what it means to be a pattern set R , and what it means for a pattern set to match a sentence. We show in an intractable but completely effective way how to compute how often R matches a sentence in $\text{EPCPPL}[\delta][n]$. That is, we describe the set of matches of a pattern set as the mathematical structure $\text{Matches}[\text{EPCPPL}[\delta][n]][R]$ and we show how in Mathematica to correctly, exactly and completely construct and compute the cardinality of this described mathematical structure. This tells us that the question of the number of matches is a well-posed problem with a unique computable integer answer for every R, δ, n , and begs the central question of this thesis, which is how to compute the number of matches tractably.

A pattern set $\text{PatSet}[C_1, \dots, C_j]$ is set of j conjuncts C_i . Variables which occur in terms in conjuncts of pattern sets are interpreted as pattern variables, that is, variables which may range over all variables in the language. In **Gedanken**, let

$$\text{EPCPPLPatSets}[\delta][n] = 2^{\text{EPCPPL}[\delta][n]}$$

describe the set of pattern sets.

Let $p, q \in \text{EPCPPL}[\delta][n]$. Let $\sigma \in \text{EPCPPLTermSubsts}[\delta][n]$ be a term substitutions. We say that π is **more general than** q , written $p \Rightarrow q$ or **IsMoreGeneral** $[\delta, n][p, q]$, if there is a term substitution σ such that $\sigma(\pi)$ is a subset of q . Formally, let us define this as $p \Rightarrow q \equiv (\exists \sigma) q \supseteq \sigma(p)$. Let $P \in \text{EPCPPLPatSets}[\delta][n]$, $q \in \text{EPCPPL}[\delta][n]$. We say that P is more general than q , written $P \Rightarrow q$ or **IsMoreGeneral** $[\delta, n][P, q]$, if there is a $p \in P$ such that p matches with q . Formally, let us define this as $P \Rightarrow q \equiv (\exists p \in P) p \Rightarrow q$.

To each element P of the abstract syntax category $\text{EPCPPLPatSets}[\delta][n]$ there corresponds an abstract syntax category

$$\text{Matches}[\text{EPCPPL}[\delta][n]][P] = \{q : q \in F(n) \wedge P \Rightarrow q\}$$

which are the matches of a set of patterns P to sentences in $\text{EPCPPL}[\delta][n]$.

The operations **Construct** and **Cardinality** work for

$$\text{EPCPPLPatSets}[\delta][n]$$

and

$$\text{Matches}[\text{EPCPPL}[\delta][n]][R]$$

Clearly

$$\#\text{EPCPPLPatSets}[\delta][n] = 2^{\#\text{EPCPPL}[\delta][n]}$$

(But note that this is larger than the number of canonical pattern sets, as defined below. We do not explore the number of canonical pattern sets in this thesis, because it is not something we need to know.) We can intractably compute $\#\text{Matches}[\text{EPCPPL}[\delta][n]][R]$

by just constructing all the matches and then counting the size of the resulting constructed set. For example, here are all the patterns sets over 1 0-ary relation, in Gedanken:

```
Construct[EPCPPLPatSets[ElementaryType[0, Vector[1,0]]][0]]
```

```
{PatSet[], PatSet[Conjunct[]], PatSet[Conjunct[1[]]],
  PatSet[Conjunct[], Conjunct[1[]]]}
```

and the number of them is:

```
Cardinality[EPCPPLPatSets[ElementaryType[0, Vector[1,0]]][0]]
```

4

and here are all the matches in 0 variables of a constant pattern over a language with two relations, in Gedanken:

```
SED = ERS["Sets",
  {#1 == #2&, #1 == Complement[#2,\3]&},
  {{}},
  {2,3}];
```

```
P = PatSet[Conjunct[1[IndConst[1],IndConst[1]]]];
```

```
Construct[EPCPPL[delta[SED]][0]]
```

```
{Conjunct[], Conjunct[1[IndConst[1], IndConst[1]]],
  Conjunct[2[IndConst[1], IndConst[1], IndConst[1]]],
  Conjunct[1[IndConst[1], IndConst[1]],
    2[IndConst[1], IndConst[1], IndConst[1]]]}
```

```
Construct[Matches[EPCPPL[delta[SED]][0]][P]]
```

```
{Conjunct[1[IndConst[1], IndConst[1]],
  Conjunct[1[IndConst[1], IndConst[1]],
    2[IndConst[1], IndConst[1], IndConst[1]]]}
```

Variables which occur in terms in patterns are interpreted as pattern variables, that is, variables which may range over all variables in the language. `PatOccurs[P]`, gives the occurrences of pattern variables in a pattern set P, in the order of occurrence of the variables, without duplicates. For example, in *Gedanken*:

```
PatOccurs[PatSet[Conjunct[4[1], 5[2], 7[6, 1]],
  Conjunct[6[1], 11[5, 3, 3]]]]
```

```
{1, 2, 6, 5, 3}
```

`Relations[P]` gives the set of relations (in sorted order) occurring in any conjunct in pattern set P. For example, in *Gedanken*:

```
Relations[PatSet[Conjunct[4[1], 5[2], 7[6, 9]],
  Conjunct[6[1], 11[5, 3, 3]]]]
```

```
{4, 5, 6, 7, 11}
```

The result of term substitution on a pattern set is the pattern set of the term substitution applied to its conjuncts. For example, in *Gedanken*:

```
TermSubstitute[{3, 4, 5}][PatSet[Conjunct[1[1, 1, 2], 3[]]]]
```

```
PatSet[Conjunct[1[3, 3, 4], 3[]]]
```

A pattern set is in canonical form if its conjuncts are in canonical form and are listed in ascending well order. The canonical form of a pattern set is computed with `Canonical[P]`. For example, in *Gedanken*:

```
Canonical[PatSet[Conjunct[2[1], 1[]], Conjunct[1[], 3[]],
  Conjunct[1[], 2[], 3[]]]]
```

```
PatSet[Conjunct[1[], 2[], 3[]], Conjunct[1[], 2[1]],
      Conjunct[1[], 3[]]]
```

A pattern set r matches with a pattern π , written `MatchesQ[r,  $\pi$ ]`, if some pattern in r is more general than π . For example, in `Gedanken`:

```
MatchesQ[PatSet[Conjunct[1[], 2[]], Conjunct[1[], 3[]]],
        Conjunct[3[], 1[]]]
```

`True`

```
MatchesQ[PatSet[Conjunct[1[], 2[]],
                Conjunct[1[], 3[]]],
        Conjunct[9[], 2[], 3[]]]
```

`False`

A most general pattern set `MostGeneral[P]`, is that $R \subseteq P$, P a pattern set, in which no pattern in R is more general than another in R . We check this by eliminating patterns in P which are less general than some pattern in P . For example, in `Gedanken`:

```
MostGeneral[PatSet[Conjunct[1[], 2[]], Conjunct[1[], 2[], 3[]],
                  Conjunct[1[], 3[]]]]
```

```
PatSet[Conjunct[1[], 2[]], Conjunct[1[], 3[]]]
```

Let A and B be pattern sets. Then by $A \vee B$ or `Disjoin[A, B]` we mean the pattern set such that, if a sentence matches $A \vee B$, then it matches a pattern in A or a pattern in B . Clearly $A \vee B = A \cup B$. For example, in `Gedanken`:

```
Disjoin[PatSet[Conjunct[1[]], Conjunct[2[]]],
        PatSet[Conjunct[3[]], Conjunct[4[]]]]
```


PatSet[Conjunct[1[]], Conjunct[2[]], Conjunct[3[]],
 Conjunct[4[]]]

6.7 Syntactic simplification rule sets

Let ϕ and ψ be conjuncts in some EPCPPL of degree n and type δ . Let σ be a substitution, that is, a mapping from conjuncts to conjuncts which takes variables to variables or constants, and the special symbol ϕ to an entire conjunct. If $\phi = \sigma(\rho)$, we say ρ **subsumes** or is more general than ϕ (see Huet [Hue80, p. 808] or Robinson [Rob65] for this terminology), and that σ **unifies** or is a match for ρ and ϕ . Consider ρ as a “pattern”. Then the set of matches of ρ with sentences in the EPCPPL of degree n is the number of ϕ for which there exists a σ such that $\phi = \sigma(\rho)$. Write this as

$$\text{Matches}_\rho(\delta, n) = \{\phi \in \text{EPCPPL}(\delta, n) \mid \exists \sigma \in \text{EPCPPLSubsts}(\delta, n) \wedge \phi = \sigma(\rho)\}$$

A **syntactic simplification rule** for a language L is a rewrite rule with a pattern on the left-hand-side and an editing instruction on the right-hand-side which has the effect of reducing the complexity of a sentence to which it applies. For example:

$$x = x \setminus x \wedge \phi \Rightarrow \phi[\emptyset/x]$$

for a matching sentence, deletes the literal $x = x \setminus x$ from the sentence, and substitutes the constant $\emptyset$ for the variable x in the remainder, thus reducing it both in degree and number of literals.

The **degree** of a sentence is its number of distinct variable occurrences. There are two aspects in measuring the effectiveness of a rule:

- The extent to which the degree of matching sentences is reduced. Thus “simplification” of a sentence π is synonymous with the transformation of π to π' , a sentence logically equivalent to π of degree $m < n$. We investigate two kinds of problem space reductions: **elimination** simplifiers which reduce the degree

of applicable sentences by one or more variables, and **invalidating** simplifiers which, by detecting invalid sentences, effectively reduce the degree of applicable sentences to 0.

- The number of sentences of a given degree, at most or (stratified) exactly, which match a given rule.

Thus a measure of the **average case effectiveness** E of a rule ρ which reduces the degree of applicable sentences in a language L of type δ by degree m on an algorithm $\mathcal{A}$ whose worst-case time complexity $T(n)$ is measured in terms of the degree n of an input sentence, is

$$E = \frac{\#L_n T(n)}{\#\text{Matches}_\rho(\delta, n) T(n - m) + (1 - \#\text{Matches}_\rho(\delta, n)) T(n)}$$

E is the extent to which $\mathcal{A}$ is speeded up by preprocessing with R for inputs of size at most n .

6.7.1 Elimination simplifiers

An **elimination simplifier** ρ reduces a conjunct in n variables into an equisatisfiable one in fewer than n variables. The degree simplifiers we present eliminate one or two variables per application. Let the notation $e[j/i]$ denote well-formed formula e with j substituted for i , via syntactic substitution (we do not have bound variables in this thesis so we do not need to make a more elaborate definition than that). Then we will write an elimination simplifier S in one of the following forms, depending on whether the simplifier eliminates one or two variables, and depending on whether it depends on the presence of one or two atoms of a specific form:

$$ES : A \wedge \pi \rightarrow \pi[e/i]$$

This says that S simplifies conjunct $A \wedge \pi$, A an atom, in which pattern variables i, j occur in A , to conjunct $\pi[e/i]$ in which pattern variable i has been replaced by term e .

6.7.2 Invalidating simplifiers

An **invalidating simplifier** ρ is one that immediately tells us that the sentence is invalid. An example of a simple invalidating simplifier in set theory is

$$IS_1 : i \in i \wedge \pi \rightarrow \perp$$

We use $\perp$ to indicate falsehood, or the unsatisfiable proposition. $\perp$ can be thought of as an abbreviation for any unsatisfiable form, for example, $i \neq i$.

Chapter 7

The match counting problem

Let us begin this chapter by reviewing our application in its simplest terms. We want to simplify sentences in an elementary relational language. In particular, we are interested in simplifying sentences in elementary multilevel syllogistic (*EMLS*), the language of conjuncts of relations over individual set-valued variables $i, j, \dots$ and one constant, $\emptyset$, where the relations are either of the 2-ary form

$$i = j \quad i \neq j \quad i \in j \quad i \notin j$$

or of the 3-ary form

$$i = j \cup k \quad i = j \setminus k$$

If there are exactly n individual variables in the language, then *EMLS* is a finite language, which we write *EMLS_n*.

For example, if the individual variables are i_1 through i_{10} (that is, we are in *EMLS₁₀*), then the following is an EMLS sentence:

$$\phi : i_{10} = i_3 \wedge i_{10} = i_2 \setminus i_2 \wedge i_{10} \in i_4$$

and we can apply the reductions of an elimination simplifier rule set **twice** to reduce ϕ to the equivalent (we will show how in a moment):

$$\phi' : \emptyset \in i_4$$

The elimination simplifier rule set $\Pi = \{\pi_1, \pi_2\}$ we use is:

$$\pi_1 \quad : \quad p = q \wedge \phi \Rightarrow \phi[q/p]$$

$$\pi_2 \quad : \quad p = q \setminus q \wedge \phi \Rightarrow \phi[\emptyset/p]$$

Rule π_1 says that if an individual variable p is equated to an individual variable q , then p may be replaced by q everywhere it occurs in the remaining expression. Rule π_2 says that if $p = q \setminus q$ then p is the empty set. So π_1 reduces ϕ to

$$i_3 = i_2 \setminus i_2 \wedge i_3 \in i_4$$

and then π_1 reduces this to

$$\emptyset \in i_4$$

Notice that p and q in Π can match any combination of i_1 through i_{10} , that is, can be instantiated with any pair of individual variables in the set

$$\Sigma = \{(p, q) : p, q \in \{i_1, \dots, i_{10}\}\}$$

Hence, we call p and q **pattern variables**, and distinguish them thus from **individual variables**.

Thus to count the number of matches of, say, π_2 , to all sentences of $EMLS_{10}$ means to count the number of distinct sentences in $EMLS_{10}$ that can be formed by conjoining the pattern part of the left-hand-side of π_2 , namely the atom $p = q \setminus q$, after performing the assignment $(p, q) = \sigma$ for any $\sigma \in \Sigma$, with any sentence in $EMLS_{10}$. We represent sentences (conjuncts) as **sets** of literals, so when we count “the number of distinct sentences”, we equate, for example,

$$i = i \wedge i = j \wedge i = j$$

$$i = j \wedge i = i$$

$$i = j \wedge i = i \wedge i = j$$

A sentence ϕ in $EMLS_n$ matches a rule set $\Pi = \{\pi_1, \dots, \pi_m\}$, where the rules in Π range over pattern variables $p_1, \dots, p_w$, if ϕ matches any one of the π_i , $1 \leq i \leq m$.

The chapter starts by showing how to reduce the problem of counting the matches of a pattern set in $EMLS_n$ to the problem of counting matches for a related pattern set in a much simpler language PC_k of conjuncts of propositional constants. In the rest of the chapter, having made our setting clear, we give:

- Methods of counting matches for conjunctions and disjunctions of pattern sets
- Worst-case time complexity analyses of the basic algorithms.
- Some optimizations.
- Proof that match counting for PC_k is in $\#P$, and conjecture that it is $\#P$ -Complete.

Finally, we explore the use of generating functions to encode, for a given rule set Π , as n increases, the number of matches of Π with $EMLS_n$. We note that the obvious sequence of generating function constructions fails early on, at the stage of counting the elements of a power set. Our hope in exploring generating functions is to find a method of computing, in polynomial time, an approximation to the match count valid to within an order of magnitude. We conjecture that there is no solution to finding a closed-form generating function for the number of elements of a power set of a set in a family of sets indexed by n whose size is given by another generating function, and from that failure we conjecture that the problem of approximating the match count for PC_k is just as hard as computing the exact match count.

7.1 Counting matches in $EMLS_n$

We will proceed in the following manner to show how to count the number of matches of Π with $EMLS_n$:

- We equate $EMLS_n$ with $L_{4,2,3,n}$, where $L_{j,k,m,n}$ is the language consisting of one constant, j $(m-1)$ -ary relations, k m -ary relations, and n individual variables.
- We show how match counting in $L_{j,k,m,n}$ reduces to match counting in $L_{k(n+1)+j,m-1,n+1}$, where $L_{k,m,n}$ is the language consisting of k m -ary relations and n individual variables.
- We show how match counting in $L_{k,m,n}$ reduces to match counting in $L_{kn,m-1,n}$.
- We show how match counting in $L_{k,1,n}$ reduces to match counting in $L_{kn,0,n}$, the base case which consists of kn “0-ary” relations, otherwise known as constants.
- We note that $L_{k,0,n}$ is equal to a language PC_k of conjuncts of sets of k constants. We define pattern sets and give an algorithm for match counting for PC_k .

7.1.1 Reduction of $EMLS_n$ to $L_{4,2,3,n}$

Define $EMLS_n$ as an EPCPPL with type $\langle 1, \langle 2, 2, 2, 2, 3, 3 \rangle \rangle$ and n individual variables. Define $L_{j,k,m,n}$ to be an EPCPPL with type $\langle 1, \langle m-1 \rangle^j; \langle m \rangle^k \rangle$ and n individual variables. Clearly $EMLS_n = L_{4,2,3,n}$.

Let’s assume that in $L_{j,k,m,n}$ the relation symbols for the j $(m-1)$ -ary relations are denoted by integers from 1 to j , and the relation symbols for the k m -ary relations are denoted by integers from 1 to k .

Let’s fix the following pattern set in $EMLS_2$ as an example:

- In terms of $EMLS_2$, the pattern set is

$$\Pi_0 = \{j = i \setminus i \wedge \phi \rightarrow \phi[\emptyset/j], i \in \emptyset \wedge \phi \rightarrow \perp\}$$

- In terms of $L_{4,2,3,2}$, let’s just represent this (ignoring the “ $\wedge\phi$ ” and the right-hand side, which are not relevant for match counting purposes) as

$$\Pi_1 = \{\{1(2, 1, 1)\}, \{1(1, K)\}\}$$

We express this in Gedanken as:

Pi1 =

PatSet[Conjunct[1[2,1,1]],
Conjunct[1[1,IndConst[1]]]];

7.1.2 Reduction of $L_{j,k,m,n}$ to $L_{k(n+1)+j,m-1,n+1;V}$

Let $L_{k,m,n}$ be the language of k m -ary relations over n individual variables. $L_{k,m,n;V}$ is that language extended by some pattern variables $V = \{v_1, v_2, \dots\}$.

The distinct conjuncts (i.e., sentences) in $L_{j,k,m,n}$ are those that can be obtained (recalling that there is one constant and n individual variables) by choosing one or more atoms from $L_{j,m-1,n+1}$ and choosing one or more atoms from the disjoint (since the arity of atoms is different) set $L_{k,m,n+1;V}$. So, writing $\#L$ for the number of distinct conjuncts in elementary relational language L , where conjuncts are represented as ordered sets without repetitions,

$$\begin{aligned}\#L_{j,k,m,n} &= 2^{j(n+1)^{m-1}+k(n+1)^m} \\ &= \#L_{j,m-1,n+1} \times \#L_{k,m,n+1}\end{aligned}$$

But

$$\begin{aligned}\#L_{k(n+1)+j,m-1,n+1} &= 2^{(k(n+1)+j)(n+1)^{m-1}} \\ &= 2^{j(n+1)^{m-1}+k(n+1)^m} \\ &= \#L_{j,k,m,n}\end{aligned}$$

so it is clear that $L_{j,k,m,n}$ is equinumerous with $L' = L_{k(n+1)+j,m-1,n+1}$, so since L' is in the family $L_{k,m,n}$, and we happen to know how to count matches in $L_{k,m,n}$, we will

construct a bijection between pattern sets Π in $L_{j,k,m,n}$ and pattern sets Π in L' , so that the number of matches in L' of Π' for $n + 1$ individual variables will be the same as the number of matches in $L_{j,k,m,n}$ of Π with n individual variables and one constant.

The construction proceeds as follows for our running example:

- (1) Replace integer indices i for individual variables occurring in patterns in Π with symbols v_i for $1 \leq i \leq n$. This is to represent that these variables are pattern variables, where by “pattern variable” we mean the notion described at the beginning of this chapter. Implicitly, pattern variables may range from 1 to $n + 1$. We leave them as “bound parameters” for convenience, although they could be equivalently substituted for to obtain “constant patterns” at this point, that is, patterns which match only a specific choice of substitutions of individual variables for pattern variables. In our example, Π_1 becomes

$$\Pi_2 = \{\{1(v_2, v_1, v_1)\}, \{1(v_1, K)\}\}$$

This appears in a trace of the operation of Gedanken’s Rule Set Analyzer code as:

```
Pi2 =
  PatSet[Conjunct[1[Var[2], Var[1], Var[1]]],
    Conjunct[1[Var[1], IndConst[1]]]]
```

- (2) Change occurrences of the unique constant in $L_{j,k,m,n}$ in Π to the variable index $n + 1$. For example, Π_2 becomes

$$\Pi_3 = \{\{1(v_2, v_1, v_1)\}, \{1(v_1, 3)\}\}$$

This appears in a trace of the operation of Gedanken’s Rule Set Analyzer code as:

Pi3 =

PatSet[Conjunct[1[Var[2], Var[1], Var[1]]],
 Conjunct[1[Var[1], 3]]]

(3) Simultaneously

- Replace atoms $e(f_1, \dots, f_m)$ by atoms

$$((e - 1)(n + 1) + f_1)(f_2, \dots, f_m)$$

that is, the index of the relation in L' is a linear expression in f_1 , where f_1 may be $n + 1$ or one of the indeterminates v_i for $1 \leq i \leq n$. The idea is to encode possible values of the pattern variable f_1 into the relation symbol, understanding that the range of possible instances can be obtained by varying the variable v_i from 1 to $n + 1$.

- Replace atoms $e(f_1, \dots, f_{m-1})$ by atoms

$$(e + k(n + 1))(f_1, \dots, f_{m-1})$$

The idea here is that the substitution of individual variable indices for v_i can **never** result in

$$(e - 1)(n + 1) + f_1 = e + k(n + 1)$$

and together, all such substitutions into $(e - 1)(n + 1) + f_1$ and $e + k(n + 1)$ completely span the space of relation symbols $k(n + 1) + j$ of L' .

We do these steps simultaneously so that $(m - 1)$ -ary atoms created in the first step do not get “doubly translated” by the second step. In our example, Π_3 becomes

$$\Pi_4 = \{\{v_2(v_1, v_1)\}, \{7(v_1, 3)\}\}$$

in $L_{10,2,3;\{v_1,v_2\}}$.

This appears in a trace of the operation of *Gedanken*'s Rule Set Analyzer code as:

```
Pi4 =
  HoldVars[2] [
    PatSet[Conjunct[Var[2][Var[1], Var[1]]],
      Conjunct[7[Var[1], 3]]]]
in L[10,2,3]
```

The “HoldVars[2]” in the trace indicates that we are in $L_{10,2,3;\{v_1,v_2\}}$.

The matches of the resulting pattern set will be equinumerous in L' as the original pattern set in $L_{j,k,m,n}$. This is because all we have done is encode the possible instantiations of the “extra” argument of the m -ary relations by parameterising the relation index of the $m - 1$ version of the m -ary atom on a pattern variable. So we now turn to the match-counting problem for the class of “languages $L_{k,m,n}$ whose relation indices are extended by pattern variables”, of which L' is a member.

7.1.3 Reduction of $L_{k,m,n;V}$ to $L_{kn,m-1,n;V}$ for $m > 0$

Let's assume, without loss of generality, that we are given a pattern set Π in $L_{k,m,n;V}$ which is such that

- No pattern in Π is subsumed by another.
- Variable and relation symbols are renumbered so that lowest numbered symbols are used first, so that we use the minimum number of variable and relation symbols needed for counting purposes.
- Individual variable integer indices i in atoms in Π are replaced by pattern variables v_i .

We may then reduce the match counting problem for $L_{k,m,n;V}$, $m > 1$, to that for $L_{kn,m-1,n}$, as follows:

- (1) Replace each atom $e(f_1, \dots, f_m)$ by $(e(n-1) + f_1)(f_2, \dots, f_m)$.
- (2) Compute the matches of the result in $L_{kn, m-1, n}$.

For our running example of the preceding section, Π_4 in $L_{10,2,3}$ becomes

$$\Pi_5 = \{\{(v_1 + 3v_2 - 3)(v_1)\}, \{(18 + v_1)(3)\}\}$$

in $L_{30,1,3;\{v_1, v_2\}}$ and this becomes, in $L_{90,0,3;\{v_1, v_2\}}$

$$\Pi_5 = \{\{(9v_2 + 4v_1 - 12)()\}, \{(54 + 3v_1)()\}\}$$

This appears in a trace of the operation of Gedanken's Rule Set Analyzer code as:

Pi5 =

HoldVars[2] [

PatSet[Conjunct[(-3 + Var[1] + 3 Var[2])[Var[1]]],

Conjunct[(18 + Var[1])[3]]]

in L[30,1,3]

Pi5 =

HoldVars[2] [

PatSet[Conjunct[(-12 + 4 Var[1] + 9 Var[2])[]],

Conjunct[(54 + 3 Var[1])[]]]]

in L[90,0,3]

7.1.4 Reduction of $L_{k,0,n;V}$ to PC_k

Now define PC_k to be the language of conjuncts of atoms, where the atoms are just the indices of propositional constants c_i , $1 \leq i \leq k$.

By convention, we set $L_{k,0,n} = PC_k$ for any number of individual variables n . This is because, interpreting propositional constant as "0-ary relations", since sentences are

conjuncts of atoms, and atoms are applications of relations to arguments, and variables occur in the language only as arguments to relations, if all relations are 0-ary, then we can add any number of variables to the language without changing its size, so $L_{k,0,0} = L_{k,0,1} = \dots = L_{k,0,n}$ for any n , and indeed $L_{k,0,n}$ is a finite language with 2^k sentences. Since this may be confusing, arising as it does as the base case of a larger preceding construction, since n finally plays no role in $L_{k,0,n}$, we rename $L_{k,0,n}$ as PC_k , thereby eliminating structure which is unnecessary at this stage.

We reduce the match counting problem for $L_{k,0,n;V}$ to $L_{k,0,n} = PC_k$ as follows: Let $w = |V|$ be the number of distinct pattern variables. Form the Cartesian product $\{i : 1 \leq i \leq n\}^w$. Interpret this as a set of n^w distinct assignments to $\langle v_1, \dots, v_w \rangle$. Form the pattern set in PC_{kn} which is the union of the pattern set under each possible assignment. We will show how to count the matches of this pattern set in the next section, which is equal to the number of matches of the original pattern set. For example, Π_5 in $L_{90,0,3;\{v_1,v_2\}}$ becomes, under the 9 possible assignments $\{1, 2, 3\}^2$, the pattern set Π_6 of 12 distinct patterns purely in PC_{90} equal to:

$$\begin{aligned} \Pi_6 = \{ & \{1()\}, \{5()\}, \{9()\}, \{10()\}, \{14()\}, \{18()\}, \\ & \{19()\}, \{23()\}, \{27()\}, \{57()\}, \{60()\}, \{63()\} \} \end{aligned}$$

This appears in a trace of the operation of Gedanken's Rule Set Analyzer code as:

Pi6 =

```
PatSet[Conjunct[1[]], Conjunct[5[]], Conjunct[9[]],
      Conjunct[10[]], Conjunct[14[]], Conjunct[18[]],
      Conjunct[19[]], Conjunct[23[]], Conjunct[27[]],
      Conjunct[57[]], Conjunct[60[]], Conjunct[63[]]]
```

in PC[90]

7.1.5 The match counting algorithm for PC_k

PC_k is a language of conjuncts of k propositional constants, which we represent by the “0-ary relation” integer indices from 1 to k , applied to the empty argument list. Recall that conjuncts are represented as sets. A sentence ϕ in PC_k matches a pattern π in PC_k if ϕ as a set is a superset of π as a set.

Another way of thinking of a sentence in PC_k is as a k -bit binary word. We see that there are 2^k distinct sentences in PC_k . There are

$$2^{2^k}$$

(not necessarily effectively distinct) pattern sets in PC_k .

Now let us define the Inclusion-Exclusion Principle, on which we will base an algorithm for counting the number of matches of a pattern set Π in PC_k with the sentences of PC_k . First, let us define PC_k formally as

$$PC_k = \{k : k \subseteq \{i() : 1 \leq i \leq k\}\}$$

where i denotes the 0-ary relation with integer index i , or the propositional constant k .

Then the Inclusion-Exclusion Principle is as follows: Let S_0 be a set. Suppose we can decompose S_0 into m (not necessarily disjoint) sets S_i so that

$$S_0 = S_1 \cup \dots \cup S_m$$

Suppose we wanted to compute the number of elements in S_0 , written $N_0 = |S_0|$. This is equal to $|S_1 \cup \dots \cup S_m|$. Then the Inclusion-Exclusion Principle, proven in [BW91, §4.3], states that

$$N_0 = |S_0| = |S_1 \cup \dots \cup S_m| = \sum_{i=1}^m (-1)^{i-1} N_i$$

where we define

$$N_i = \sum_j |S_{j_1} \cap \dots \cap S_{j_i}|$$

```

Matches( $\Pi, PC_k$ ) :=
begin
  for  $i := 1$  to  $p$ 
  do  $R := \{q : q \subseteq \{m : 1 \leq n \leq p\} \wedge |q| = i\}$ 
     $N_i := 0$ 
    for  $S \in R$ 
    do  $\pi := \cup_{j \in S} \pi_j$ 
       $N_i := N_i + 2^{k-|\pi|}$ 
    od
  od
  return  $\sum_{i=1}^p (-1)^{k-1} N_i$ 
end

```

Figure 7.1: Match counting algorithm for patterns over PC_k

where the sum is over all $j = \{j_1, \dots, j_i\}$ in

$$\{q : q \subseteq \{k : 1 \leq k \leq m\} \wedge |q| = i\}$$

Now we give the match-counting algorithm for pattern sets in PC_k . Let $\Pi = \{\pi_i : 1 \leq i \leq p\}$ be a set of patterns. Let S_i denote the set of sentences of PC_k which match pattern π_i . If π_i has ℓ_i atoms, then the number of matching sentences in PC_k is just $2^{k-\ell_i}$. We can see this by viewing π_i as a k -bit word in which ℓ_i bits are set. Then there are $k - \ell_i$ bits which may vary. So let S_0 be all the matches of Π with PC_k .

The intersection $\pi_i \otimes \pi_j$ of two patterns π_i and π_j is the pattern whose matching sentences match both π_i and π_j . Clearly

$$\pi_i \otimes \pi_j = \pi_i \cup \pi_j$$

Let $|\pi_i|$ be the number of elements in conjunct π_i where the conjunct is represented as a set without repetitions. Given these facts and applying the Inclusion-Exclusion Principle, our algorithm to count the number of matches of pattern set Π with PC_k is shown in Figure 7.1.

For example, the number of matches with PC_{90} of our previous running example Π_8 is just 1237637807830476617605447680.

7.2 Conjoining and disjoining pattern sets

7.2.1 Conjoining

The conjunction (that is, the intersection) of $P_1 \wedge P_2$ of two patterns P_1 and P_2 describes sentences which match both patterns.

7.2.1.1 PC_k

Thinking of patterns in PC_k as k -bit binary words, and using $\pi_1 \oplus \pi_2$ to denote bit-wise OR, the conjunction is just

$$P_1 \wedge P_2 = \{\pi_1 \oplus \pi_2 : \pi_1 \in P_1 \wedge \pi_2 \in P_2\}$$

To denote the conjunction of two pattern sets in *Gedanken*, we use the `Conjoin` operator, as in the following example:

```
Cardinality[
  Matches[L[4,0,0]] [
    Conjoin[PatSet[Conjunct[1[]], Conjunct[2[]]],
      PatSet[Conjunct[3[]], Conjunct[4[]]]]]]
```

9

7.2.1.2 $L_{k,m,n}$

Now we consider the problem of counting conjunctions of pattern sets for $L_{k,m,n}$ where $m > 0$. Let P_1, P_2 be patterns in $L_{k,m,n}$, $m > 0$. The matches of the conjunction $P_1 \wedge P_2$, that is, sentences which match a pattern in P_1 and one in P_2 , are obtained by:

- (1) In P_1 and P_2 separately, replacing atoms $e(f_1, \dots, f_m)$ by

$$(e(n-1+f_1))(f_2, \dots, f_m)$$

giving P'_1, P'_2 , where $m > 1$, and then counting the matches of $P'_1 \wedge P'_2$ in

$$L_{kn,m-1,n}.$$

- (2) For $m = 1$, replacing $e(f_1)$ by $(e(n-1) + f_1)()$ in P_1 and P_2 giving P'_1 , P'_2 , and then computing the matches in $L_{k,0,0}$ of

$$P'_1 \wedge P'_2 = \{p_1 \wedge p_2 \mid p_1 \in P'_1 \wedge p_2 \in P'_2\}$$

Note this construction is not similar to that of the “ $\wedge$ ” construction for PC_k . If we were to apply that construction blindly, the net result after projecting down, if the matches of P_1 are $P_1 = A_1 \cup A_2$ and $P_2 = B_1 \cup B_2$, will be to count the values of $(A_1 \wedge B_1) \cup (A_2 \wedge B_2)$, not

$$(A_1 \wedge B_1) \cup (A_1 \wedge B_2) \cup (A_2 \wedge B_1) \cup (A_2 \wedge B_2)$$

which is what we want.

An example in *Gedanken* of counting the matches of a conjunction of pattern sets is as follows:

```
Cardinality[
  Matches[L[2,1,2]] [
    Conjoin[PatSet[Conjunct[1[1]]],
      PatSet[Conjunct[1[1]]]]]
```

12

7.2.2 Disjoining

The disjunction of two pattern sets Π_a and Π_b , that is, the pattern set which matches the sentences which match either Π_a or Π_b , is just their union $\Pi_a \cup \Pi_b$.

7.3 Match counting worst-case time cost

7.3.1 PC_k

Theorem 7 The time cost of match counting for a pattern set of p patterns in PC_k is of the same order as $p2^{p-1}(2+k) - p$.

Proof. Suppose we have a p -element set S represented by a p -bit binary word. To write down a subset of S takes, then, $O(p)$ time steps. There are $\binom{p}{i}$ subsets of i elements. Thus the step in Figure 7.1 of forming set R takes time $O(p\binom{p}{i})$. Then for $O(|R|)$ times, that is, $O(\binom{p}{i})$ times, we intersect i patterns, which takes time, for each pattern, $O(ki)$. So the time cost of the algorithm of Figure 7.1 is on the order of

$$\begin{aligned} \sum_{i=1}^p p \binom{p}{i} + \binom{p}{i} ki &= \\ &= p \sum_{i=1}^p \binom{p}{i} + k \sum_{i=1}^p i \binom{p}{i} \end{aligned}$$

(see [Liu68, Example 2-3])

$$\begin{aligned} &= p \sum_{i=1}^p \binom{p}{i} + kp2^{p-1} \\ &= p \left(\sum_{i=1}^p \binom{p}{i} \right) + k2^{p-1} \end{aligned}$$

which equals (again, see [Liu68, Example 2-3]):

$$= p(2^p - 1) + k2^{p-1} = p2^{p-1}(2 + k) - p$$

□

7.3.2 $L_{k,m,n}$

We show that the worst-case time complexity of counting matches in the language is $O(pn^w 2^{pn^w-1}(2 + kn^m))$, where p is the number of patterns in the pattern set, and w is the number of pattern variables.

Theorem 8 The worst-case time complexity of counting matches of a pattern set in $L_{k,m,n}$ with p patterns over w pattern variables is of the same order as

$$pn^w 2^{pn^w-1}(2 + kn^m)$$

Proof. We project down to PC_{kn^m} , at which time we compute a pattern set of up to pn^w (that is, the union of n^w pattern sets of size p) conjuncts. Then, applying Theorem 7, the cost is:

$$pn^w 2^{pn^w-1}(2 + kn^m) - pn^w = pn^w 2^{pn^w} + kpn^{m+w} 2^{pn^w-1} = pn^w 2^{pn^w-1}(2 + kn^m)$$

□

Recall that the number of sentences in $L_{k,m,n}$ is just 2^{kn^m} . The cost in the worst case is comparable to that of enumerating every sentence in the language and unifying each such sentence individually with the pattern set.

7.4 Some optimizations

7.4.1 PC_k

Now let's define some special-case optimizations for the match counting algorithm for $L_{k,0,n}$. Write $M_{P,k}$ for the number of matches of pattern set P in $L_{k,0,n}$.

Theorem 9 If two pattern sets P_1, P_2 have non-intersecting atoms, and the number of atoms in P_1 (P_2) is r_1 (r_2), then

$$M_{P_1 \cup P_2, k} = 2^{k-r_1-r_2} (2^{r_1} M_{P'_2, r_2} + 2^{r_2} M_{P'_1, r_1} - M_{P'_1, r_1} M_{P'_2, r_2})$$

where P'_1 (P'_2) be the result of renumbering the r_1 (r_2) relation occurrences in P_1 (P_2) from $1 \dots r_1$ ($1 \dots r_2$).

Proof. The $2^{k-r_1-r_2}$ represents choices of parts of sentences that are matched by neither P_1 or P_2 . To see the logic of the remainder of the expression, form a Venn diagram, that is, this is the base case of the Inclusion-Exclusion Principle that, if $S_0 = S_1 \cup S_2$, then $|S_0| = |S_1| + |S_2| - |S_1 \cap S_2|$. □

Theorem 10 If two pattern sets P_1, P_2 have non-intersecting atoms and P_1 has just one pattern, then

$$M_{P_1 \cup P_2, k} = 2^{k-r_1} + (2^{r_1} - 1) M_{P'_2, k-r_1}$$

Proof. The total number of matches is that of P_1 , plus those of P_2 in the part of the word not occupied by P_1 (recall, the patterns don't intersect) for the number of times that P_1 doesn't hold, which is $2^{r_1} - 1$ because, for a pattern in which r_1 atoms must be present, there are $2^{r_1} - 1$ ways for those atoms not to be present. $\square$

We may use these observations to simplify a pattern set whenever it applies, that is, whenever the pattern has more than one connected component with respect to the occurrence of relations.

In *Gedanken*, we define the operation `RetractRelations` to perform the renumbering described in the statement of the above theorems. This has the simplifying property of “getting rid of” unused relations. For example:

```
RetractRelations[
  PatSet[Conjunct[7[1,2,3],4[5,3]],
    Conjunct[12[[]]]]
PatSet[Conjunct[2[1, 2, 3], 1[5, 3]], Conjunct[3[[]]]
```

The following application of the `Cardinality` operator in *Gedanken* results in an application of the above optimizations:

```
Cardinality[
  Matches[L[12, 0, 0]] [
  PatSet[Conjunct[1[], 4[]],
    Conjunct[1[], 5[]],
    Conjunct[2[], 3[]],
    Conjunct[2[], 6[[]]]]]
```

7.4.2 $L_{k,m,n}$

We present an optimization to the match-counting algorithm for $L_{k,m,n}$ when $m > 0$. Suppose pattern set P is decomposable into two pattern sets P_1, P_2 such that the set of atoms in P_1 is disjoint from that in P_2 . Let the number of distinct relation symbols in P_1 (P_2) be r_1 (r_2). Let P'_1 (P'_2) be the result of renaming relations of P_1 (P_2) so that relation symbols are numbered from 1 to r_1 (r_2). Let $M_{x,y,m,n}$ be the number of matches of pattern set x in $L_{y,m,n}$, for $m > 0$. Let $d = n^m$. Then

Theorem 11 The number of matches of P in $L_{k,m,n}$ is

$$2^{k-r_1-r_2-d}(2^{r_1d}M_{P'_2,r_2,m,n} + 2^{r_2d}M_{P'_1,r_1,m,n} - M_{P'_1,r_1,m,n}M_{P'_2,r_2,m,n})$$

Proof. Via the Inclusion-Exclusion Principle. That is, factor out the “freely substitutable” atoms wherever possible, and then count in the factored problem using the Inclusion-Exclusion Principle. $\square$

7.5 Match counting for PC_k is in $\#P$

The match counting problem falls naturally into the class of “ $\#P$ ” problems [GJ79, §7.3, The Complexity of Enumeration Problems]. In this section we prove that match counting for pattern sets in PC_k is in $\#P$. The enumeration problem for *SAT* of counting the number of satisfying assignments of a proposition in conjunctive normal form is $\#P$ -Complete. We conjecture that the match-counting problem is also $\#P$ -Complete, since the reduction to SAT which occurs to us, which we describe below, involves a potentially exponential-time transformation. We use the terminology of Garey and Johnson (1979) [GJ79] regarding polynomial time Turing reduction $\Pi \propto_T \Pi'$. Assuming that concept, for convenience we re-state some additional terminology here.

Let $\Pi = \langle D_\Pi, S_\Pi \rangle$ be a problem, where D_Π is the **domain** of problem instances $I \in D_\Pi$ and $S_\Pi(I) = \{x : x \text{ solves } I\}$ is the (possibly empty) **solution set** of instance I . Then

- The **search problem** is to find one element of $S_{\Pi}(I)$, or return **none** if $S_{\Pi}(I)$ is empty.
- The **decision problem** is to return **False** if $S_{\Pi}(I)$ is empty, **True** otherwise.
- The **enumeration problem** is to compute $\#S_{\Pi}(I)$, the number of solutions of problem instance I .

An enumeration problem Π is **in** the class $\#P$ (“Number P”) if there is a nondeterministic algorithm such that for each $I \in D$, the number of distinct guesses that lead to acceptance of I is exactly $\#S_{\Pi}(I)$, and such that the length of the longest accepting computation is polynomial in $\#I$, the length of I . Then Π is $\#P$ -Complete if for all Π' in $\#P$, $\Pi' \propto_T \Pi$.

Let Π, Π' be two search problems. A **parsimonious transformation** $f : \Pi \mapsto \Pi'$ is a function $f : D_{\Pi} \rightarrow D_{\Pi'}$ computable in polynomial time and such that

$$(\forall I \in D_{\Pi}) \#S_{\Pi}(I) = \#S_{\Pi'}(f(I))$$

Theorem 12 If there exists a parsimonious transformation from Π to Π' , then $\Pi \propto_T \Pi'$.

The enumeration problem for *SAT* is to compute the number of satisfying assignments to a proposition in **conjunctive normal form**. The following theorem is due to Simon [Sim75]:

Theorem 13 The enumeration problem for *SAT* is $\#P$ -Complete.

Now, we state our theorem and conjecture concerning pattern matching in PC_k . Let $R = \{\pi_1, \dots, \pi_p\}$ be a set of p patterns in PC_k . Define the problem size m to be $m = pk$. Each pattern π_i is effectively just a k -bit binary word, where bit j is set to 1 if relation $j()$ occurs in the pattern. The pattern-matching enumeration problem $\#Matches_{PC_k}(R)$ of size m in PC_k is to count the number of matches of R in PC_k .

Theorem 14 The enumeration problem $\#Matches_{PC_k}(R)$ is in $\#P$ for m .

Proof. To accept a sentence $s \in PC_k$ it suffices to check that s matches one of the π_i . s matches a π_i if, whenever there is a bit set to 1 in π_i , the corresponding bit in s is 1. This can be checked in time $O(k)$ for each π_i , so in time $O(m)$ per problem instance. A nondeterministic algorithm guesses a k -bit word, and checks in $O(m)$ time whether or not it matches R . $\square$

To show that $\#Matches_{PC_k}(R)$ is $\#P$ -Complete for m , it is sufficient to find a parsimonious reduction of SAT to pattern-matching in PC_k . However, imagine that a match ψ for R is interpreted as a satisfying assignment, in the sense that if bit i in ψ is a 1, then we assign **True** to propositional letter i , otherwise **False**. Then the sentences matched by R , and hence the satisfying assignments determined by R , are exactly the sentences **implied** by R , when R is interpreted as a disjunct of conjuncts, where if relation i is present in R , then propositional letter i is positively asserted in the interpretation, otherwise it is not present. If we used two bits for each propositional letter, one to assert the letter, the other to assert its negation, then we could construct a pattern R whose sentence interpretation implied exactly the sentences implied by a sentence in SAT , under a suitable interpretation of the two bits. However, to get from the sentence in SAT to the corresponding pattern R , we would have to do a potentially exponential-time transformation from conjunctive normal form to disjunctive normal form. It is the fact that we may have to do this transformation of this complexity for the natural reduction that causes us to make the following conjecture:

Conjecture 7.1 The enumeration problem

$$\#Matches_{PC_k}(R)$$

is $\#P$ -Complete.

7.6 The generating function approach and approximability

Recall that earlier in this thesis we took pains to describe a language for mathematical structures which allowed us to compute the cardinality of a structure separately from, and one hopes easier than, computing the construction of the structure. The idea was that, following the strategy of Flajolet and Vitter’s LUO system for automatically computing the average-case complexity of algorithms [FV87], we could automatically construct a generating function for the size of any describable mathematical structure when the description of that structure was parameterized on a single variable n . Fixing pattern set and language, such a structure is the set of matches of a pattern set with an elementary relational language of size n .

However, this strategy fails quite early on, in the following sense: It is easy, for example, to construct a generating function for any series $\langle p_n \rangle_{n \geq 0}$. However, in constructing the set of sentences of an elementary relational language, which encounter the power set operation. Then again we encounter the power set operation compounded twice when we consider the set of all pattern sets. But suppose the size of a set in a family parameterized by n is given by some series $\langle a_n \rangle_{n \geq 0}$, with generating function $A(x)$. It is the purpose of this section to argue that the Flajolet-Vitter strategy fails in general precisely, that is, we can’t find a closed-form generating function for $\#Matches_{PC_n}(R)$, because early on in the construction of $\#Matches_{PC_n}(R)$, we perform a power set construction, and it would appear to be the case that there is no general generating-function constructing function (call this a “generating functional”) $P(A, x)$ such that for any closed-form generating function $A(x)$, the series $\langle p_n \rangle_{n \geq 0}$ described by $P(A, x)$ is such that $p_n = 2^{a_n}$.

That is, if $C(x)$ is the generating function for the sequence $\langle c_n \rangle_{n \geq 0}$ in any **generating function context** (that is, there are ordinary, exponential, Dirichlet, etc. generating functions, and we prove the theorem for all such classes), then there is no

numerically approximable form for the generating function (in that context) $P(C(x), x)$ of the sequence $\langle 2^{c_n} \rangle_{n \geq 0}$.

More precisely, let $F = (f_n(x))_{n \geq 0}$ be a fixed sequence of complex-valued functions, and $a = (a_n)_{n \geq 0}$ be a sequence of complex numbers. Recall that the generating function of a with respect to F is the formal series $G[a](z) = \sum_{n=0}^{\infty} a_n f_n(z)$ ([Pet90, §1.1], [Wil90]). Call F a **generating function context** or **gfc**.

Let R be a pattern set for PC_k . Consider $\#Matches_{PC_k}(R)$. In general, is there a generating function $\mathbf{M}_R(x)$ in some gfc F such that the coefficient of the $f_k(z)$ in the series expansion of $\mathbf{M}_R(x)$ equals $\#Matches_{PC_k}(R)$?

Let us consider a particular instance of this question. In PC_{k^3} , let R consist of the patterns $i()$ for $1 \leq i \leq k^3$. Then R matches every sentence in PC_{k^3} . In this case, $\#Matches_{PC_{k^3}}(R) = 2^{k^3}$. Then the question becomes, is there a generating function $\mathbf{M}(x)$ in some gfc F for the sequence $\langle 2^{k^3} \rangle_{k \geq 0}$, for which there is a closed-form rational function expression for $\mathbf{M}(x)$?

In general, let $\langle a_n \rangle_{n \geq 0}$ be any series in some gfc F , and let $\mathbf{A}(x)$ generate a_n . Is there a “generating functional” $\mathbf{P}(\mathbf{A}(x), x)$ which has as its value a generating function with a closed-form rational function expression in F for the sequence $\langle 2^{a_n} \rangle_{n \geq 0}$ for any $\mathbf{A}(x)$? Note that $\mathbf{P}(\mathbf{A}(x), x)$ generates the cardinality of the power set of a set of size a_n .

The utility of $\mathbf{P}$ and a suitable F is that it would allow us to use computations on generating functions to compute $\#Matches_{L_{k,0,n}}(R)$ either exactly or approximately. Approximately, we could use techniques such as Lagrange Inversion [BW91, p. 362] and asymptotic approximations from complex analysis [FV87, §2], to get a “ballpark” estimate of the magnitude of the match count, and thus circumvent the apparent intractability of solving problems in class $\#P$.

We conjecture that $\mathbf{P}$ does not exist in the sense defined. The reasons why not are as follows:

(1) Suppose $a_n = n^2$. This leads us to try to solve $P(x) = 1 + 2xP(4x)$. But this is a class of equations we don't know how to solve. Indeed, Goldberg [Gol79], in his thesis analyzing a question related to match-counting, encountered a similar equation, $G(x) = xe^x + 2(e^{qx} - 1)G(px)$ which he didn't know how to solve. Let us call the class $\mathcal{C}$ of which this equation is a member, "the class $\mathcal{C}$ of rational function equations of the form $G(x) = h(x, G(f(x)))$, where $f(x) \neq x$ ". The language Goldberg was working with (*SAT*) is not relevant. What is relevant is that:

- Equations of class $\mathcal{C}$ arise naturally in this kind of work.
- There seems to be no literature on the algorithmic algebraic methods for solving members of class $\mathcal{C}$ where by "solving", we mean either:
 - (a) Computing a closed-form rational function expression for $G(x)$.
 - (b) Computing a numerical approximation to a_n for any $n \geq 0$, where $\langle a_n \rangle_{n \geq 0}$ is the series generated by $G(x)$.

(2) Petkovsek, in his thesis on the solution of generating functions [Pet90], notes that

- (a) The method of generating functions will not give solutions which grow too fast, unless we can solve the associated differential equation as an equation in formal power series.
- (b) Generating functions are best for nonlinear equations whose nonlinearity has the form of a convolution.

Hence we make the following:

Conjecture 7.2 Fix a gfc F . Let $\mathbf{A}(x)$ generate $\langle a_n \rangle_{n \geq 0}$ in F . There is no generating functional $\mathbf{P}(\mathbf{A}(x), x)$ which for all such $\mathbf{A}$ constructs a closed-form rational function expression which generates $\langle 2^{a_n} \rangle_{n \geq 0}$ in F .

This is disappointing, as there has been in the past considerable enthusiasm for the “method of generating functions”. For example, Flajolet and Vitter in their 1987 paper [FV87] proposed using generating function methods as the basis for an automated method of constructing the functional equations that arise from computing the complexity of any given combinatorial algorithm. But the power set construction is very frequent in combinatorial algorithms, thus apparently limiting the general utility of the technique.

Flajolet and Vitter provided numerous techniques for approximating the coefficients in the power series expansion of functions for which a closed form expression for those coefficients was difficult or impossible to obtain. The apparent unavailability of the generating function method for this problem, which then implies the unavailability of these approximation techniques, leads us to the following:

Conjecture 7.3 The problem of approximating the number of matches of a pattern set with sentences of size n to within an order of magnitude is #P-Complete.

Chapter 8

Deciding EMLS Faster On Average

In this chapter we do an algorithm-level analysis of the prospects for improving the average case complexity of deciding sentences in *EMLS*. We “fall back” to simpler average case analysis techniques than those used in Chapter 5. We proceed as follows:

- We define *EMLS* as an elementary relational structure.
- We sketch the constructions of the Model Graph Algorithm of Policriti for deciding *EMLS* [Pol87a], and we give some simplification rule sets for *EMLS* defined Policriti, who hoped to speed up the Model Graph Algorithm in practical use [Private communication, 1992].
- We do a small practical analysis, using the rule set analyser component of Gedanken described in the previous chapter, of a subset of Policriti’s rule sets. We can show a speedup which is significant but which still does not affect the order of magnitude cost of deciding *EMLS* for practical problem sizes.

8.1 Elementary multi-level syllogistic

As a language, we may equate $EMLS_n$ (sets with $\cup, \setminus, =, \neq, \in, \notin$ and n set-valued variables [FOS80]) with $L_{4,2,3,n}$:

We may also define the relational structure of *EMLS* with Gedanken as follows:

`SetEq[a_,b_] := MatchQ[a,b];`

```

SetNotEq[a_,b_] := !SetEq[a,b];
SetIn[a_,b_] := MemberQ[b, a];
SetNotIn[a_,b_] := !SetIn[a,b];
SetEqUnion[a_,b_,c_] := MatchQ[a, Union[b,c]];
SetEqDiff[a_,b_,c_] := MatchQ[a, Complement[b,c]];

plEq = PredicateLetter[1];
plNotEq = PredicateLetter[2];
plIn = PredicateLetter[3];
plNotIn = PredicateLetter[4];
PlEqUnion = PredicateLetter[5];
plEqDiff = PredicateLetter[6];

RelationalStructure[EMLS] :=
  ERS["Sets",
    {SetEq, SetNotEq, SetIn, SetNotIn,
      SetEqUnion, SetEqDiff},
    {{}},
    {2,2,2,2,3,3}]

```

“Sets” in the construction may be interpreted to mean the Von Neumann universe of sets, although more restrictive definitions may be semantically adequate for this class of sentences [CFO89, §1.3.2].

The type of *EMLS*, computed in *Gedanken*, is:

```
delta[RelationalStructure[EMLS]]
```

```
ElementaryType[1, {2, 2, 2, 2, 3, 3}]
```

The total size of *EMLS* with up to n variables, computed in *Gedanken*, is:

```
Cardinality[EPCPPL[delta[RelationalStructure[EMLS]]][n]]
```

$$4 \binom{2}{1+n} + 2 \binom{3}{1+n}$$

2

Another way of computing this is to refer to the previous Theorem:

Cardinality[AdjacentLWithK[4,2,3,n]]

$$4 \binom{2}{n} + 2 \binom{3}{n}$$

2

The size of EMLS expressions with exactly n variables, computed in Gedanken, is:

```
Cardinality[
  Stratified[
    EPCPPL[delta[RelationalStructure[EMLS]]]] [n]]
```

```
Stratify[Cardinality[
  EPCPPL[
    ElementaryType[1, {2, 2, 2, 2, 3, 3}][#1]] & ] [n]
```

Example. The number of EMLS expressions in which exactly 2 variables occur is:

```
Cardinality[
  Stratified[
    EPCPPL[delta[RelationalStructure[EMLS]]]] [2]]
```

1237940039285380266309189568

And the number of EMLS sentences involving 0 variables is:

```
Cardinality[
  Stratified[
    EPCPPL[delta[RelationalStructure[EMLS]]]] [0]]
```

8.2 The Model Graph algorithm for *EMLS*

8.2.1 Overview

There is an algorithm to decide EMLS [Pol87b]. It is based on the idea of the model graph, which is a representation of a satisfying assignment for a sentence, should one exist. The algorithm constructs all possible model graphs of a sentence of degree n , and then checks them in sequence to decide if the sentence is satisfiable. If none are, then the sentence is unsatisfiable.

To proceed with the discussion of this chapter we need to construct a good upper bound SMG_n on the number of (skeletal; see below) model graphs of size n , and therefore it is necessary to know what a model graph is. A good upper bound on the number of model graphs SMG_n of degree n that must be constructed and checked will turn out to be

$$\sum_{n \leq k \leq n+4} n^3 \binom{4n^3}{k-n} = 2^{4n^3}$$

Example. An upper bound on the number of model graphs of degree 4 is

UpperBound[Cardinality[SMG[4]]]

472553303154964924989004370051186389478210715642481882577\
3288591535660703365661008446502996340542399698574313284\
33974960326174706663509667348266572071494

We will give the essential constructions necessary to derive our upper bound, without proving the correctness of the Model Graph algorithm. A sketch of the proof of correctness which is sufficient to perform a complete proof is given by Policriti [Pol87a, §2, MLS-Sat problem is in NP]. The MLS-Sat problem is reducible to the equivalent problem of “ $\forall_0^\ell$ simple prenex formulæ”, for which there is a published proof [COP90, §3.2].

The results we build on are as follows. We describe a construction $G = MG_n(a, \pi)$, which stands for the model graph of π at a , and a transformation on it, $\star(G)$, which Policriti [Pol87b], calls the “Mostowski collapse”. We give a construction SMG_n , which stands for the set of skeletal model graphs of degree n , and an extension of the Mostowski collapse $\star(H)$ to skeletal model graphs $H \in SMG_n$.

Theorem 15 If $EMLS \models_a \pi$ for some $a \in \mathbf{Sets}_n$, then $EMLS \models_{\star(G)} \pi$.

Theorem 16 If $EMLS \models_{\star(G)} \pi$, then $(\exists H \in SMG_n) EMLS \models_{\star(H)} \pi$.

Hence (this is the “Model Graph Algorithm”),

Theorem 17 If $\neg(\exists H \in SMG_n) EMLS \models_{\star(H)} \pi$, then π is unsatisfiable.

8.2.2 Set-Theoretic Preliminaries

We follow the notation of [CFO89, ch. 1–2].

Let $Un(s) = \{x : x \in y \wedge y \in s\}$. Define the predicate

$$ordinal(s) = s \supseteq Un(s) \wedge (\forall x, y \in s) x \in y \vee y \in x \vee x = y$$

Let the class of ordinal numbers be defined as $Ord = \{x : ordinal(x)\}$.

Define the predicate

$$limitord(s) \equiv s \neq \emptyset \wedge s \in Ord \wedge \neg(\exists y \in Ord) y = s \cup \{s\}$$

Let the class of limit ordinals be defined as $LimitOrd = \{x : limitord(x)\}$.

Define the Von Neumann universe $\mathcal{V}$ of sets as

$$\mathcal{V} = \bigcup_{\alpha \in Ord} V_\alpha$$

where

$$\left\{ \begin{array}{ll} V_\emptyset &= \emptyset \\ V_{\alpha+1} &= Pow(V_\alpha) \quad \text{for } \alpha \in Ord \setminus LimitOrd \\ V_\gamma &= \bigcup_{\beta \in \gamma} V_\beta \quad \text{for } \gamma \in LimitOrd \end{array} \right.$$

Define the rank $R(x)$ of a set x as the ordinal number

$$R(x) = \bigcup_{y \in x} (R(y) \cup \{R(y)\})$$

8.2.3 The Model Graph Construction

Let a be an n -vector of sets. Define the model graph

$$G = (V, E) = MG_n(a)$$

of a as the directed acyclic graph G with nodes V and edges E where

$$\begin{aligned} V &= \{(x_i, x_i^{\mathbf{EMLS}}[a]) : 1 \leq i \leq n\} \cup \\ &\quad \{(u_{1ijk}, U_{1ijk}) : 1 \leq i, j, k \leq n \wedge \\ &\quad U_{1ijk} \in (x_j^{\mathbf{EMLS}}[a] \setminus x_k^{\mathbf{EMLS}}[a]) \setminus x_i^{\mathbf{EMLS}}[a]\} \cup \\ &\quad \{(u_{2ijk}, U_{2ijk}) : 1 \leq i, j, k \leq n \wedge \\ &\quad U_{2ijk} \in x_i^{\mathbf{EMLS}}[a] \setminus (x_j^{\mathbf{EMLS}}[a] \setminus x_k^{\mathbf{EMLS}}[a])\} \cup \\ &\quad \{(u_{3ijk}, U_{3ijk}) : 1 \leq i, j, k \leq n \wedge \\ &\quad U_{3ijk} \in (x_j^{\mathbf{EMLS}}[a] \cup x_k^{\mathbf{EMLS}}[a]) \setminus x_i^{\mathbf{EMLS}}[a]\} \cup \\ &\quad \{(u_{4ijk}, U_{4ijk}) : 1 \leq i, j, k \leq n \wedge \\ &\quad U_{4ijk} \in x_i^{\mathbf{EMLS}}[a] \setminus (x_j^{\mathbf{EMLS}}[a] \cup x_k^{\mathbf{EMLS}}[a])\} \\ E &= \{((u, S), (v, T)) : (u, S), (v, T) \in V \wedge S \in T\} \end{aligned}$$

8.2.4 The Mostowski Collapse of a Model Graph

Define the Mostowski collapse of a model graph G as follows, where $G = (V, E)$.

Recall that each node in V is either of form $(x_i, x_i^{\mathbf{EMLS}}[a])$ for $1 \leq i \leq n$ or of form (u_{mijk}, U_{mijk}) . So $\star(G) = \langle MC(G)(x_i) \rangle_{1 \leq i \leq n}$ i.e. the n -tuple of sets where

$$MC(G)(x_i) = \{MC(G)(v) : (v, S) \in V \wedge ((v, S), (x_i, x_i^{\mathbf{EMLS}}[a])) \in E\}$$

and

$$MC(G)(u_{mijk}) =$$

$$\{Repr(n, U_{mijk})\} \cup \{MC(G)(v) : (v, S) \in V \wedge ((v, S), (u_{mijk}, U_{mijk})) \in E\}$$

and $Repr(n, U_{mijk})$ is a set of rank $r > \min(\max_{1 \leq i \leq n} rank(a(i)), \#V) + 1$ which represents the set U_{mijk} in the sense that $Repr(n, S) = Repr(n, T)$ iff $S = T$.

Let t , $1 \leq t \leq 4n^3$, be the least integer such that there exists t sets R such that for $1 \leq i, j \leq t$, $R_i = R_j$ iff $i = j$ and for $1 \leq i, j, k \leq n$, $1 \leq m \leq 4$, there is a $1 \leq q(m, i, j, k) \leq t$ such that $R_{q(m, i, j, k)} = U_{mijk}$. A suitable definition of r is then $\min(\max_{1 \leq i \leq n} rank(a(i)), \#V) + 2$, and of $Repr$ is then

$$Repr(n, U_{mijk}) = \{\{V_{r-2}\}, \{V_{q(m, i, j, k)}\}\}$$

8.2.5 The Skeletal Model Graph construction

A skeletal model graph H is defined as the restriction $smg(G)$ on a model graph $G = (V, E)$ in which we lose the sets associated with each node in the model graph. Let t , $q(m, i, j, k)$ be constructed for G as before. Define

$$\begin{cases} Rename(x_i) &= x_i \\ Rename(u_{mijk}) &= w_{q(m, i, j, k)} \end{cases}$$

Then,

$$\begin{aligned} smg(G) = \\ smg(V, E) = & (\{Rename(v) : (v, S) \in V\}, \\ & \{(Rename(s), Rename(t)) : ((s, S), (t, T)) \in E\}) \end{aligned}$$

The set of all skeletal model graphs of n variables is the set

$$SMG_n = \{smg(G) : (\exists a, \pi) G = MG_n(a, \pi)\}$$

We may make this definition completely constructive, as follows. Let

$$\begin{aligned} X_n &= \{x_i : 1 \leq i \leq n\} \\ W_n &= \{w_q : 1 \leq q \leq 4n^3\} \end{aligned}$$

be two sets of symbols. Then the set of all node sets of skeletal model graphs of n variables is

$$SMGVars_n = \{X_n \cup W : W \subseteq W_n\}$$

Note that $\#W_n = 4n^3$, and so $\#SMGVars_n = 2^{\#W_n}$. The distribution of sizes of the $\#SMGVars_n$ variable sets is from n to $n + 4n^3$. So the number of variable sets of size k elements, for k from n to $n + 4n^3$, written $\#SMGVars_{n,k}$, is simply $4\binom{n^3}{k-n}$. A quick reality check should show that $\#SMGVars_n = \sum k = n^{n+4n^3} \#SMGVars_{n,k}$. We can test this in Gedanken for a few numbers as follows:

```
Block[{n,k},
  Table[Cardinality[SMGVars[n]] -
    Sum[Cardinality[SMGVars[n,k]],{k,n,n+4 n^3}],
    {n,0,5}]]

{0, 0, 0, 0, 0, 0}
```

For each $V \in SMGVars_n$, the space of edge sets is

$$SMGEdges_{n,V} = Acyclic(Pow(V \times V))$$

where

$$V \times V = \{(v,u) : v \in V \wedge u \in U\}$$

is the usual Cartesian product and

$$Acyclic(S) = \{s : s \in S \wedge acyclic(s)\}$$

where

$$acyclic(s) = \neg(\exists v_1, v_2 \dots v_k)(v_1, v_2) \wedge (v_2, v_3) \dots (v_{k-1}, v_k) \in s \wedge v_1 = v_k$$

So the set of all skeletal model graphs of n variables is constructable as

$$SMG_n = \bigcup \{ \{ (V, E) : E \in SMGEdges_{n,V} : V \in SMGVars_n \}$$

The number of these is synonymous with the number of edge sets. It is harder to count the number of edge sets arising from a given n , because it is hard to count the acyclicity constraint. Let us be satisfied with a count of the number of edge sets excluding this constraint, which tells us that

$$\#SMG_n \leq \sum_{V \in SMGVars_n} \#SMGEdges_{n,V} \leq \sum_{k=n}^{n+4n^3} \#SMGVars_{n,k}^2$$

Example. Using Gedanken, we compute the following upper bound on the number of skeletal model graphs of degree 4:

UpperBound[Cardinality[SMG[4]]]

472553303154964924989004370051186389478210715642481882577\
 3288591535660703365661008446502996340542399698574313284\
 33974960326174706663509667348266572071494

As a power of 2, this is

N[Log[UpperBound[Cardinality[SMG[4]]]]/Log[2]]

507.1735474947737759

8.2.6 Mostowski Collapse of a Skeletal Model Graph

For $H = (V, E) \in SMG_n$ we define the Mostowski collapse of a skeletal model graph $*(H)$ as

$$*(H) = \langle SMC(H)(x_i) \rangle_{1 \leq i \leq n}$$

i.e. the n -tuple of sets when

$$SMC(G)(x_i) = \{ MC(G)(v) : v \in V \wedge (v, x_i) \in E \}$$

$$SMC(G)(wj) = \{SRepr(V, j) \cup SMC(G)(v) : v \in V \wedge (v, w_j) \in E\}$$

and

$$SRepr(V, j) = V_{\#V+j+1}$$

8.2.7 Some simplification rules

A. Policriti's heuristic algorithm for *EMLS* [Private communication, 1992] employs several kinds of problem space simplification prior to applying the model graph algorithm proper. The time cost of the model graph algorithm is a function of the number S of skeletal model graphs it must check. S is polynomially exponential in the degree n of the sentence π whose satisfiability is to be decided. Here are some simplification rules worked out by A. Policriti. We can distinguish two classes of rules, **variable elimination rules**, and **sentence invalidating rules**. Some examples of elimination simplifiers in *EMLS* are:

$$ES_1 : i = j \cup \emptyset \wedge \pi \rightarrow \pi[j/i]$$

$$ES_2 : i = \emptyset \cup j \wedge \pi \rightarrow \pi[j/i]$$

$$ES_3 : i = j \cup j \wedge \pi \rightarrow \pi[j/i]$$

$$ES_4 : i = \emptyset \cup j \wedge \pi \rightarrow \pi[\emptyset/i]$$

$$ES_5 : i = j \cup \emptyset \wedge \pi \rightarrow \pi[j/i]$$

$$ES_6 : i = j \cup j \wedge \pi \rightarrow \pi[\emptyset/i]$$

$$ES_7 : i = j \cup i \wedge \pi \rightarrow \pi[\emptyset/i]$$

$$ES_8 : i = j \wedge \pi \rightarrow \pi[j/i]$$

$$ES_9 : \emptyset = i \cup j \wedge \pi \rightarrow \pi[\emptyset/i][\emptyset/j]$$

$$ES_{10} : i = j \cup k \wedge k = j \cup l \wedge \pi \rightarrow \pi[\emptyset/i]$$

$$ES_{11} : i = j \cup k \wedge k = j \cup l \wedge \pi \rightarrow \pi[k/i]$$

$$ES_{12} : i = j \cup k \wedge j = k \cup l \wedge \pi \rightarrow \pi[j/i]$$

Three invalidating rules are (where we use the symbol $\perp$ to designate the invalid sentence):

$$IS_1 : i \in i \wedge \pi \rightarrow \perp$$

$$IS_2 : i \neq i \wedge \pi \rightarrow \perp$$

$$IS_3 : k \in \emptyset \wedge \pi \rightarrow \perp$$

By the “Model Graph Algorithm after Simplifiers” we will mean just applying the elimination and invalidating simplifiers until no further simplifier applies, and then applying the Model Graph Algorithm on the result if the sentence is not already invalidated.

8.3 Analysis of average case effect

Now we make some limited demonstrations of the effectiveness of these rules. Recall that the complexity of the model graph algorithm is

$$O(2^{4(n+1)^3}) = O(2^{k(n+1)^3})$$

Recall that the number of *EMLS* sentences of size n is

$$2^{4(n+1)^2+2(n+1)^3} = O(2^{k(n+1)^3})$$

So for a fixed syntactic simplifier rule set R to have an effect on the average case complexity of the model graph algorithm, as n increases, the rule set must apply to $O(2^{k(n+1)^3})$ cases. We do not know of a valid set of syntactic simplifiers with this property. Thus, we will expect to see some average case effect for the simplifiers we study, but not an effect which will affect the tractability of the algorithm. We proceed

with this analysis below. The highlights of our analysis will be that, our match counting algorithm, using the isomorphism $\text{EMLS}(n) = L_{4,2,3,n}$, reveals that for $n = 4$, with the elimination rules above, the Model Graph algorithm will run 10^{19} times faster on average, but will still take time 10^{133} cycles. With the invalidation rules, it will run only 10^4 times faster, taking 10^{148} cycles on average. The latter is a slight surprise since the invalidation rules seem to be a complete summary of what could go wrong with a sentence, pointing towards the possibility that there may be far more satisfiable EMLS sentences of size n than invalid ones.

8.3.1 Definition of average case complexity

We define average case complexity following the paradigm of Flajolet and Vitter [FV87]. We partition a set of inputs L to a decision algorithm A under size into subsets $L_n = \{q \in L : \#q = n\}$ of elements of size n . Let the counting sequence for class L be the integer sequence c_n such that $c_n = \#L_n$. For each n , we will assume that every element of L_n may occur in practice as an input to A with equal likelihood, that is we assume a uniform distribution, or in other words a symmetric probability space.

For EMLS there is an upper bound on the number of distinct conjuncts that can be expressed with exactly n variables. To be more precise,

$$c_n = \#Stratified(EPCPPL(\delta(\text{EMLS})(n))) \leq 2^{4(n+1)^2+2n^3}$$

and indeed every L_n is thus a finite set.

Now let the time cost of computing A on $q \in L_n$ be written $T_n[A](q)$. We then define the average case complexity of A over L_n to be

$$T_n[A] = \frac{1}{c_n} \sum_{k \geq 0} k \cdot \#\{q \in L_n : T_n[A](q) = k\}$$

By saying that q' is a simplification of q in L , we mean that $T_n[A](q') < T_n[A](q)$ and $q = q'$ is valid in our model of L .

8.3.2 Pragmatic considerations

What if we try to evaluate all the rules at once? First, let's compute, in *Gedanken*, the unstratified matches for all rules together for $n = 1$. First we define the pattern set:

```
pi = PatSet[Conjunct[5[1, 2, IndConst[1]]],  
  Conjunct[5[1, IndConst[1], 2]], Conjunct[5[1, 2, 2]],  
  Conjunct[6[1, IndConst[1], 2]],  
  Conjunct[6[1, 2, IndConst[1]]], Conjunct[6[1, 2, 2]],  
  Conjunct[6[1, 2, 1]], Conjunct[1[1, 2]],  
  Conjunct[5[IndConst[1], 1, 2]], Conjunct[3[1, 1]],  
  Conjunct[2[1, 1]], Conjunct[3[1, IndConst[1]]]];
```

For $n = 1$ it runs in 80 seconds:

```
Cardinality[Matches[Emls[1]][pi]]
```

```
4294966272
```

Let's look at the matches for all rules together for $n = 2$:

```
Cardinality[Matches[Emls[2]][pi]]
```

```
1237940039285380274899091456
```

This took 388.17 seconds. Let's try the matches for all rules together for $n = 10$:

```
Cardinality[Matches[Emls[10]][pi]]
```

```
$Aborted
```

We have aborted the $n = 10$ run, as debugging traces (not included here) tell us that we have to count a pattern set involving up to (there may be some duplicates, but detecting them at this stage without expanding the patterns involves solving an integer linear programming problem which may be equally as difficult) 30404 patterns, and our best counting procedure will take more than 2^{30404} time steps. We will not attempt it!

What if we try to evaluate two rule sets in sequence? We are going to compute the following conditional probabilities:

$$p_1 = \text{Prob}(\text{Invalidate})$$

$$p_2 = \text{Prob}(\text{Eliminate 1 Var} \vee \text{Invalidate})$$

Due to the overall complexity of the procedure and the modest speed of the Apple Macintosh IIci, we shall run through the whole analysis with only one single variable elimination rule, no double variable elimination rule, and one invalidation, each in a separate rule set. We are going to have to evaluate the following sequence of rule set combinations in order to obtain the data we need for an graph of the average case complexity for $0 \leq n \leq 2$:

Quantity	Computation	Probability
q_1	$\#IS$	p_1
q_2	$\#ES_1$	
q_3	$\#IS \wedge ES_1$	
q_4	$q_2 - q_3$	p_2

Then, on a per-rule set basis, we compute data for $0 \leq n \leq 1$:

```
TestRules[r_,n_] :=
  Timing[Cardinality[Matches[Stratified[Emls][n]][r]]]
```

Here is the data for q_1 :

```
Table[TestRules[IS1, i], {i, 0, 2}]

{{0.0166667 Second, 32}, {0.2 Second, 3221225440},
 {0.116667 Second, 1083197534374707734094282720}}
```

```
q1 = {32, 3221225440, 1083197534374707734094282720};
```

Here is the data for q_2 :

```
Table[TestRules[es1, i], {i, 0, 2}]
```

```
{{2.18333 Second, 32}, {2.53333 Second, 4026531808},  
 {4.75 Second, 1235522187646151008496648160}}
```

```
q2 = {32, 4026531808, 1235522187646151008496648160};
```

Here is the data for q_3 :

```
Table[TestRules[Conjoin[IS1,ES_1], i], {i, 0, 1}]
```

```
{{3.26667 Second, 16}, {63.1167 Second, 3019898864}}
```

```
q3 = {16, 3019898864};
```

Here is why we can't go as high as, say, $n = 2$. The key is that the Sieve Method will attempt to form the power set of 24 elements (we abort the calculation):

```
TestRules[Conjoin[IS1,ES_1], 2]
```

```
$Aborted
```

Here is q_4 :

```
q4 = Table[q1[[i]] + q2[[i]] - q3[[i]], {i, 1, 2}]
```

```
{48, 4227858448}
```

Here is a cross-check. The following should equal q_4 :

```
Table[TestRules[Join[IS1, ES1], i], {i, 0, 1}]
```

{0.216666666666666667 Second, 48},
 {0.05 Second, 4227858432}}

8.3.3 Analysis for the simplifier model graph algorithm

Suppose that we have a set of simplifiers S for which computation of the stratified matches is tractable enough (say, for $n \leq 5$). Suppose that the simplifiers either (a) reduce the degree of a sentence by exactly 1, or (b) invalidate a sentence. Then analysis proceeds as follows:

- (1) Let c_n be the number of inputs of degree exactly n , so

$$c_n = \#Stratified(EMLS_n)$$

- (2) Let e_n be the number of inputs of degree exactly n reduced in degree by 1 by our elimination simplifiers. Let k_n be the number of inputs of degree exactly n invalidated by invalidating simplifiers.

- (3) Let $h_n = c_n - (e_n + k_n)$ be the “hardest” inputs (those not simplified).

- (4) Then a good approximation to the “pragmatic average case complexity” of Model Graphs After Simplifiers for a few initial n for which we can compute the numbers is:

$$\begin{aligned} T_0 &= h_0 \#SMG_0 + e_0 + k_0 \\ T_n &= \frac{1}{c_n} (h_n [\#SMG_n] n + e_n T_{n-1} + k_n n) \end{aligned}$$

The recurrence is justified because we apply the algorithm recursively to reduced sentences, which are uniformly distributed in the lower-degree space, because to any sentence of degree $n - 1$ we can add atoms which cause it to match some rule.

Let's look at the average case effect of the first 3 elimination rules for $0 \leq n \leq 4$. Figure 8.1 shows the log base 2 of the absolute average speed of a version of Model Graphs after Simplifiers with three elimination rules for $0 \leq n \leq 4$. Figure 8.2 shows the log base 2 of the worst-case complexity divided by the average speed, to give us an idea of the order of magnitude of speedup we can expect from the use of simplifiers. Note that while the algorithm is a factor of 10^{19} faster with these rules than without, it still has an execution time on the order of 10^{133} for an input of degree 4.

Let's look at the average case effect of the invalidation rules for $0 \leq n \leq 4$. Figure 8.3 gives the log base 2 of the absolute average speed of a version Model Graphs after Simplifiers with three invalidation rules for $0 \leq n \leq 3$. Figure 8.4 gives the log base 2 of the worst-case complexity divided by the average speed, to give us an idea of the speedup we can expect from the use of simplifiers. Note that while the algorithm is a factor of 10^4 faster with these rules than without, it still has an execution time on the order of 10^{148} for an input of degree 3. The invalidation rules have a much smaller average case effect than the elimination rules. However, the invalidation rules have greater intuitive “semantic primitiveness” to them — one would have expected them to be more applicable.

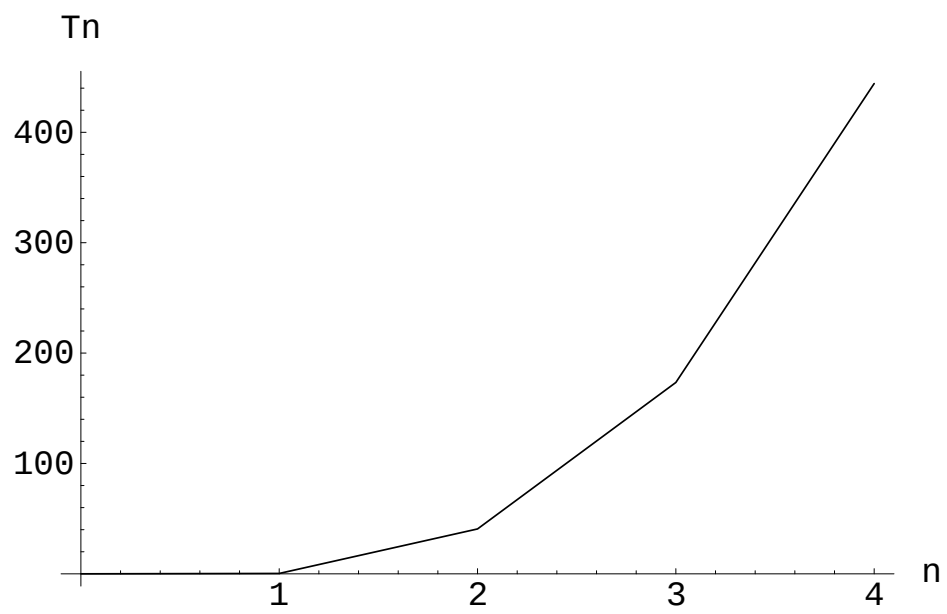


Figure 8.1: $\log_2$ speed of Model Graphs after elimination

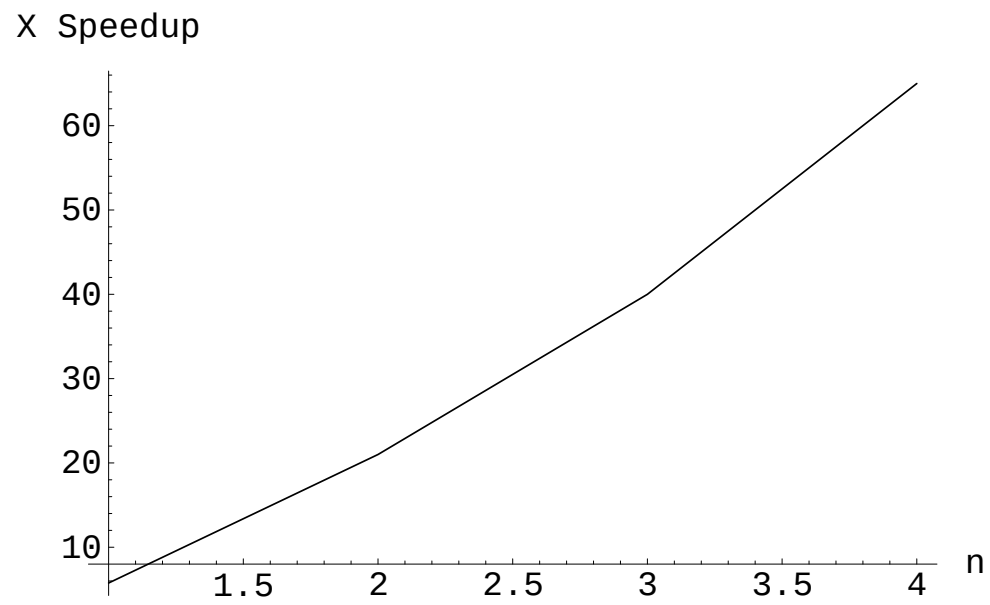


Figure 8.2: $\log_2$ speed-up of Model Graphs after elimination

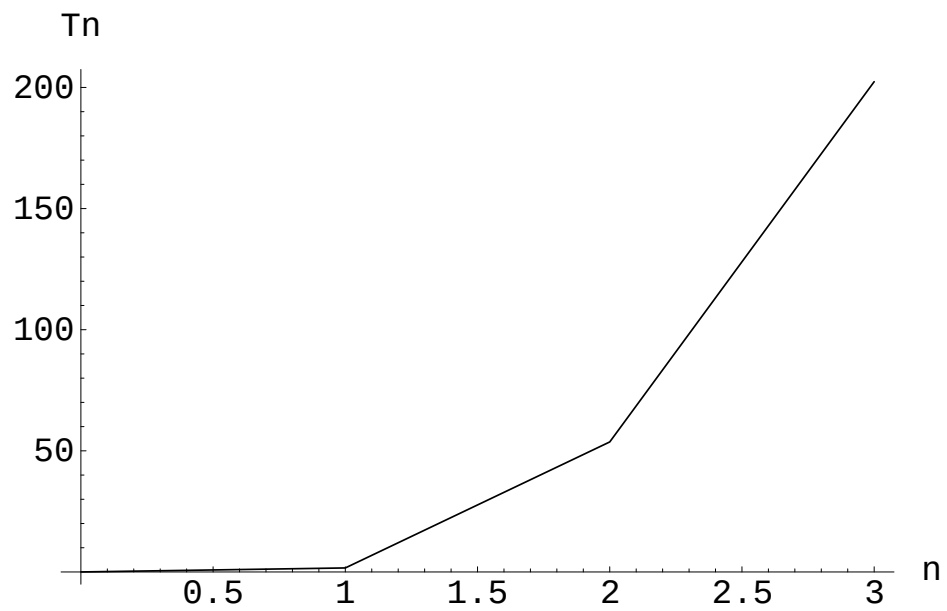


Figure 8.3: $\log_2$ speed of Model Graphs after invalidation

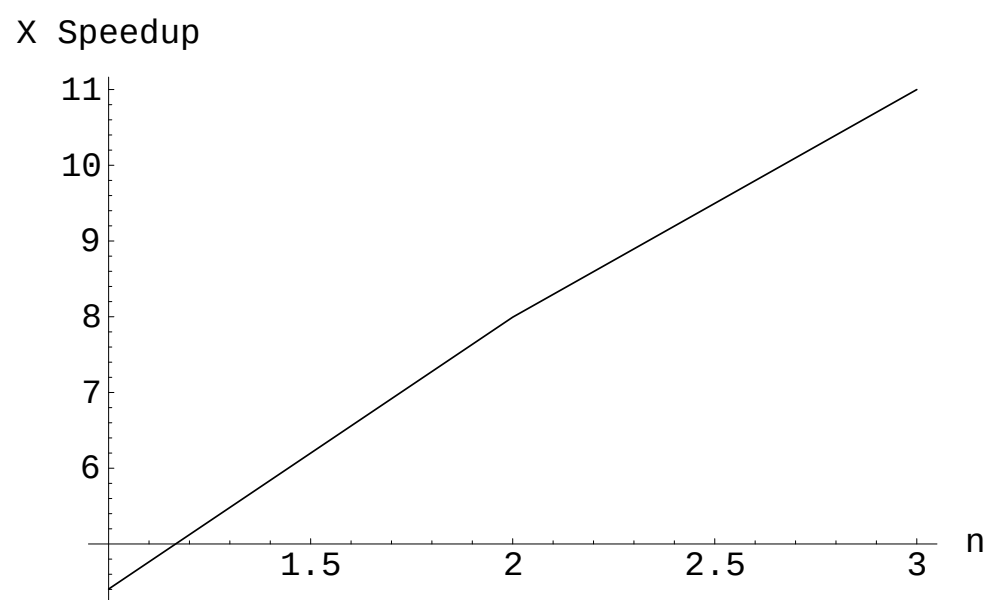


Figure 8.4: $\log_2$ speed-up of Model Graphs after invalidation

Chapter 9

Conclusions

9.1 Pragmatics

We have examined an approach to the development of program verification tools that makes a strong call to the availability of efficient decision procedures for decidable sublanguages of set theory. Several key decidable sublanguages that one may reduce to in a program verification context include *SAT*, *EMLS*, *ILP*, and Tarski sentences. The decision problems for these languages are NP-Complete or worse. The enumeration problems for the languages are #P-Complete or probably worse, which limits the ability to detect effective syntactic simplification rule sets. In addition, there are arguments against the existence of average-case effective simplifier rule sets, not to mention that *SAT* is NP-average complete, which is a strong pessimistic indication regarding the existence of average-case polynomial algorithms for these problems under any polynomially-computable distribution of input sentences.

To this extent, we may conclude that this approach to program verification is, if not in fact dead, at least highly unlikely to yield practical benefit, unless we adopt the approach of looking for a preponderance of complex special cases which detect the hard-but-solvable decision problems most likely to occur in practice, and which are solvable polynomially. However, arguing against the likely success of this approach is the possibility that most machine-generated intermediate problems arising from program verification are unlikely to fall into any easily described distribution. Under the

assumption of the uniform distribution, program verification is intractable.

9.2 Outstanding Issues

Our study progressed from actual counting of the effects of simplification rules, to a consideration of limits on the average-case effectiveness of a particular algorithm, the Model Graph Algorithm, for the problem *EMLS*, to a complexity-theory level discussion of average-case hardness. The theories of average case complexity, #P-Completeness and Kolmogorov complexity have developed as this thesis was being written, and these theories may be tied together more compactly in the future. There are numerous conjectures in this thesis to be cleaned up, e.g. the question of whether there are generating functionals for series whose members are powers of other series.

Appendix A

Gedanken

In this appendix we give the Mathematica code for the thought experiments in program verification we call **Gedanken**. The code is comprised of two major components: Correct Program Editor and Rule Set Analyser.

A.1 Correct Program Editor

The Correct Program Editor experiment is comprised of the following components:

- Target programming language and its operational semantics.
- Placed propositions
- Claims
- Praas
- Claim proofs
- Praa proofs

A.1.1 Target programming language

```
state::usage = "state[E,AS,v] represents the  
most recently evaluated expression,  
assignment stack and value.";
```

```

M::usage = "M[E][s] gives the meaning of E as a set of
state sequences.";

initial::usage = "initial = state[bot, {{}}, bot]
is the initial state";

bot::usage = "bot is the undefined expression";

e::usage = "e[x] escapes to Mathematica after evaluating
x in the current assignment stack environment";

let::usage = "let[B,E] evaluates E in the bindings B";

place::usage = "place[id,E] labels expression E with
the given id";

choose::usage = "choose[E] nondeterministically picks
one element of set-valued
expression E, or bot if E is empty";

if::usage = "if[e1,e2,e3] is the standard conditional";

while::usage = "while[e1,e2] is the standard while-loop";

call::usage = "call[f,{A}] calls f with actual parameters A";

bind::usage = "bind[x,E] binds x to E in current environment;
bind[fun[f,{A}], E] declares a function f with arguments
A and body E";

Formals::usage = "Formals[funbind] gives the formal parameters
of a function";

Body::usage = "Body[funbind] gives the body of a function";

Id::usage = "Id[funbind] gives the name of a function";

fun::usage = "fun[f,{F}] gives the name of function f with
formals F";

par::usage = "par[E1,...] evaluates E1... in parallel";

initial = state[bot, {{}}, bot];

(* E ::= e[any Mathematica expression] *)

M[z:e[x_]] [s:state[E_, AS_,v_]] :=
  {{s, state[e[x], AS, x /. AS[[1]]]}}

(* E ::= bot *)

M[z:bot] [s:state[E_,AS_,v_]] := {{s,state[z,AS,bot]}};

```

```

(* E ::= placei[E] *)

M[z:place[i_,F_]] [s:state[E_,AS_,v_]] :=
  Join[{s}, #]& /@ M[F][state[z, AS,v]];

(* E ::= if[E1, E2, E3] *)

M[z:if[E0_,E1_,E2_]] [s:state[E_,AS_,v_]] :=
  nexteval[cond[z,E1,E2]] [state[z, AS, v], E0];

cond[z_,E1_,E2_] [s:state[E_,AS_,v_]] :=
  Which[MatchQ[v,True], M[E1][s],
        MatchQ[v,False], M[E2][s],
        True, {{state[bot,AS,bot]}}];

nexteval[f_] [s_,E_] := Union @@ enchain[f] /@ M[E][s];

enqueue[f_] [{s_state}] := f[s];
enqueue[f_] [{S__state}] :=
  Join[Drop[{S},-1], #]& /@ f[Last[{S}]];

(* E ::= choose[E] *)

M[z:choose[E_]] [s_state] := nexteval[mchoose[z]] [s, E];

mchoose[z_] [state[E_,AS_,v_]] := {state[z,AS,#]}& /@ v;

(* E ::= B *)

(* B ::= bind[i,E] *)

M[z:bind[i_Symbol,E_]] [s_state] :=
  nexteval[mbind[z,i]] [s, E];

mbind[z_,i_] [state[E_,AS_,v_]] :=
  {{state[z,Join[{update[i, v, AS[[1]]}], Rest[AS]], v]}};

update[v_,e_,{ }] := {v -> e};
update[v_,e_,{m__}] :=
  If[MatchQ[v, {m}[[1,1]]],
    Join[{v->e}, Rest[{m}]],
    Join[{m}[[1]], update[v,e,Rest[{m}]]]]

(* E ::= i *)

(* i ::= any identifier but bot *)

M[z:i_Symbol] [s:state[E_,AS_,v_]] :=
  {{s, state[z, AS,lookup[i,AS[[1]]]}}};

lookup[i_,sigma_] :=
  If[!MemberQ[#[[1]]]& /@ sigma, i], bot, i/.sigma];

```

```

(* E ::= let[B,E] *)

M[z:let[B_,E_]] [s_state] :=
  Union @@
  (enchain[{{PopAS[#]}}&] /@
   nexteval[M[mlet[E]]] [DupAS[s], B]);

M[z:mlet[F_]] [s:state[E_,AS_,v_]] :=
  Join[{s}, #]& /@ M[F] [state[z, AS,v]];

PopAS[state[E_,AS_,v_]] := state[E,Rest[AS], v];

DupAS[state[E_,AS_,v_]] := state[E,Join[{AS[[1]]}, AS], v];

(* E ::= while[E1, E2] *)

M[z:while[E_,E1_]] [s_state] := nexteval[mwhile[E,E1]] [s, E];

mwhile[E_,E1_] [s:state[F_,AS_,v_]] :=
  Which[MatchQ[v,True], nexteval[M[while[E,E1]]] [s, E1],
        MatchQ[v,False], {{s}},
        True, {{state[while[E,E1], AS,bot]}}];

(* E ::= seq[E1,E2,...] *)

M[z:seq[E_]] [s_state] := M[E] [s];
M[z:seq[E_,F_]] [s_state] := nexteval[M[seq[F]]] [s, E]

(* B ::= par[E1,E2,...] *)

M[z:par[E_]] [s_] :=
  parmerge[z] /@ (CartesianProduct @@ (M[#] [s]& /@ {E}));

parmerge[z_] [A_] := Join[Join@@A, {merge[z] @@ (Last /@ A)}];

CartesianProduct[a_,b_] :=
  Join @@
  Table[{a[[i]], b[[j]]},
        {i, 1, Length[a]},
        {j, 1, Length[b]}];

CartesianProduct[a_,b_,r_] :=
  Join@@#& /@
  CartesianProduct[{#}& /@ a,
                   CartesianProduct[b,r]]

merge[z_] [s_state] := s;

merge[z_] [state[E_,AS_,v_], state[E1_,AS1_,v1_]] :=
  state[z,Join[{coalesce[AS[[1]],AS1[[1]]}], Rest[AS]], v];

merge[z_] [s_state, t_state, r_state] :=
  merge[z] [merge[z] [s,t], r]

```

```

coalesce[{}], sigma_] := sigma;

coalesce[{i_ -> v_}, sigma_] := update[i,v,sigma];

coalesce[{i_ -> v_, r_}, sigma_] :=
  coalesce[{r}, update[i,v,sigma]];

(* B ::= bind[fun[i0,{i1,i2,..}],E] *)

M[z:bind[fun[i_,A_], E_]] [s:state[F_,AS_,v_]] :=
  {{s, state[z, Join[{update[i, z, AS[[1]]}], Rest[AS]], z]}};

Formals[bind[fun[i_,A_], E_]] := A;
Body[bind[fun[i_,A_], E_]] := E;
Id[bind[fun[i_,A_], E_]] := i;

(* E ::= call[i, {E1,E2,..}] *)

M[z:call[i_, {E_}]] [s:state[F_,AS_,v_]] :=
  funccall[z, i /. AS[[1]], s, {E}];

funccall[z_, f_, s:state[F_,AS_,v_], actuals_] :=
  With[{formals = Formals[f], E = Body[f]},
    With[{bindings =
      Table[bind[formals[[i]], actuals[[i]],
        {i,1,Length[formals]}]},
      Join[{s},#]& /@
      M[let[par @@ bindings, E]] [state[z,AS,v]]]];

```

A.1.2 Placed propositions

```

(* A predicate logic/ZF sets/Float-Hoare approach *)

(* Placed Proposition's and satisfied computations *)

Proposition::usage = "Proposition[fp] yields f";

Exists::usage =
  "Exists[x,f] says there exists an x,f such that f.";

InSet::usage = "InSet[x, L] is true if x is a member of set L";

At::usage = "At[f,pi] says proposition f applies at place pi.";

GetPlace::usage = "GetPlace[E][pi] gets E'
  where place[pi,E'] is in E.
  Returns Fail if the place does not exist.";

ReplacePlace::usage =
  "ReplacePlace[E][pi,E'] replaces the statement
  immediately after place pi in program E with expression E'.";

```

```

DeletePlace::usage =
  "DeletePlace[E][pi] deletes the statement immediately
  after place pi in program E";

Occurs::usage =
  "Occurs[E] gives the free variables of object E, where
  the notion of binding is to be found in the definition of E.";

SatisfiesQ::usage =
  "SatisfiesQ[E][computation] is true if computation
  satisfies placed proposition or set of propositions E.";

VerifyQ::usage =
  "VerifyQ[f] is a general purpose theorem prover which
  returns True if there exists a proof of phi
  in first order logic plus
  Zermelo-Frankel set theory, False otherwise.";

Imply::usage =
  "Like Implies, but we don't want to reduce to not and and";

Imply[False,x_] := True;

Imply[True,x_] := x;

Format[Imply[A_,B_]] := SequenceForm["(", A, " => ", B, ")"];

Format[Exists[x_,f_]] :=
  SequenceForm["[Exists ", x, "]  (" , f, ")"];

Format[InSet[x_,L_]] := SequenceForm[x, " in ", L];

Exists[x_, f_] := f /; !MemberQ[Variables[f], x]

Unprotect[Or];
a_ || a_ := a;
Protect[Or];

Unprotect[And];
a_ && a_ := a;
Protect[And];

Unprotect[Variables];
Variables[a_ < b_] := Union[Variables[a], Variables[b]];
Variables[a_ == b_] := Union[Variables[a], Variables[b]];
Variables[a_ > b_] := Union[Variables[a], Variables[b]];
Variables[a_ <= b_] := Union[Variables[a], Variables[b]];
Variables[a_ && b_] := Union[Variables[a], Variables[b]];
Variables[a_ || b_] := Union[Variables[a], Variables[b]];
Variables[InSet[x_,L_]] := Union[{x}, Variables[L]];
Protect[Variables];

```

```

(* Placed propositions and satisfied computations *)

GetPlace[e[X_]] [p_Symbol] := Fail;
GetPlace[bot] [p_Symbol] := Fail;
GetPlace[place[i_,E_]] [p_Symbol] :=
  If[MatchQ[i,p], E, GetPlace[E] [p]];
GetPlace[choose[E_]] [p_Symbol] := GetPlace[E] [p];
GetPlace[bind[i_Symbol,E_]] [p_Symbol] := GetPlace[E] [p];
GetPlace[i_Symbol] [p_Symbol] := Fail;
GetPlace[par[E_]] [p_Symbol] := GetPlace[seq[E]] [p];
GetPlace[call[i_, {E_}]] [p_Symbol] := GetPlace[seq[E]] [p];

GetPlace[if[E_,E1_,E2_]] [p_Symbol] :=
  With[{t1 = GetPlace[E] [p]},
    If[!MatchQ[t1, Fail], t1,
      With[{t2 = GetPlace[E1] [p]},
        If[!MatchQ[t2, Fail], t2,
          GetPlace[E2] [p]]]]];

GetPlace[let[B_,E_]] [p_Symbol] :=
  With[{t1 = GetPlace[B] [p]},
    If[!MatchQ[t1, Fail], t1, GetPlace[E] [p]]];

GetPlace[while[E_,E1_]] [p_Symbol] :=
  With[{t1 = GetPlace[E] [p]},
    If[!MatchQ[t1, Fail], t1, GetPlace[E1] [p]]];

GetPlace[seq[E_]] [p_Symbol] := GetPlace[E] [p];

GetPlace[seq[E_,F_]] [p_Symbol] :=
  With[{t1 = GetPlace[E] [p]},
    If[!MatchQ[t1, Fail], t1, GetPlace[seq[F]] [p]]];

ReplacePlace[e[X_]] [p_Symbol, F_] := e[X];

ReplacePlace[bot] [p_Symbol, F_] := bot;

ReplacePlace[place[i_,E_]] [p_Symbol, F_] :=
  If[MatchQ[i,p], place[i,F], place[i, ReplacePlace[E] [p,F]]];

ReplacePlace[if[E_,E1_,E2_]] [p_Symbol, F_] :=
  if[ReplacePlace[E] [p,F],
    ReplacePlace[E1] [p,F],
    ReplacePlace[E2] [p,F]];

ReplacePlace[choose[E_]] [p_Symbol, F_] :=
  choose[ReplacePlace[E] [p,F]];

ReplacePlace[bind[i_Symbol,E_]] [p_Symbol, F_] :=
  bind[i, ReplacePlace[E] [p,F]];

ReplacePlace[i_Symbol] [p_Symbol, F_] := i;

```



```

ReplacePlace[let[B_,E_]] [p_Symbol, F_] :=
  let [ReplacePlace[B] [p,F], ReplacePlace[E] [p,F]];

ReplacePlace[while[E_,E1_]] [p_Symbol, F_] :=
  while [ReplacePlace[E] [p,F], ReplacePlace[E1] [p,F]];

ReplacePlace[seq[E_]] [p_Symbol, F_] :=
  seq @@ (ReplacePlace[#] [p,F]& /@ {E});

ReplacePlace[par[E_]] [p_Symbol, F_] :=
  par @@ (ReplacePlace[#] [p,F]& /@ {E});

ReplacePlace[call[i_, {E_}]] [p_Symbol, F_] :=
  call[i, ReplacePlace[#] [p,F]& /@ {E}];

DeletePlace[E_] [p_] := ReplacePlace[E] [p, bot];

Occurs[e[X_]] := Variables[X];
Occurs[bot] := {};
Occurs[place[i_,E_]] := Occurs[E];
Occurs[if[E_,E1_,E2_]] := Union @@ (Occurs /@ {E,E1,E2});
Occurs[choose[E_]] := Occurs[E];
Occurs[bind[i_Symbol,E_]] := Complement[Occurs[E], {i}];
Occurs[i_Symbol] := {i};
Occurs[let[B_,E_]] :=
  Complement[Union[Occurs[E], Occurs[B]], Bindings[B]];
Occurs[while[E_,E1_]] := Union[Occurs[E], Occurs[E1]];
Occurs[seq[F_]] := Union @@ (Occurs /@ {F});
Occurs[par[E_]] := Union @@ (Occurs /@ {F});
Occurs[bind[fun[i_,A_], E_]] :=
  Complement[Occurs[E], Join[{i},A]];
Occurs[call[i_, {E_}]] := Union[{i}, Join @@ (Occurs /@ {E})];

Bindings[e[X_]] := {};
Bindings[bot] := {};
Bindings[place[i_,E_]] := Bindings[E];
Bindings[if[E_,E1_,E2_]] := {};
Bindings[choose[E_]] := Bindings[E];
Bindings[bind[i_Symbol,E_]] := {i};
Bindings[i_Symbol] := {};
Bindings[let[B_,E_]] := {};
Bindings[while[E_,E1_]] := {};
Bindings[seq[F_]] := Union @@ (Bindings /@ {F});
Bindings[par[E_]] := Union @@ (Bindings /@ {F});
Bindings[bind[fun[i_,A_], E_]] := {i};
Proposition[At[f_,pi_]] := f;

Place[At[f_,pi_]] := pi;

VerifyQ := Simplify;

Every[S_] [P_] := And @@ (P /@ S);

```

```

SatisfiesQ[At[f_,pi_Symbol]][{}] := True;

SatisfiesQ[P:At[f_,pi_Symbol]][{S__state}] :=
  Every[{S}] [SatisfiesQ[P]];

SatisfiesQ[{}][S_] := True;

SatisfiesQ[{E__At}][S_] := Every[{E}] [SatisfiesQ[#][S]&];

SatisfiesQ[At[f_,pi_Symbol]][state[F_,AS_,v_]] :=
  If[MatchQ[Head[F], place] && MatchQ[pi,F[[1]]],
    VerifyQ[ReleaseHold[f /. AS[[1]]],
      True];

SatisfiesQ[pp:At[f_,fun[pi_]]][{S__state}] :=
  With[{s = {S}[[1]]},
    With[{E = s[[1]]},
      If[MatchQ[Head[E], call] && MatchQ[E[[1]], pi],
        SatisfyCall[pi, f, s, Rest[{S}]],
        True] &&
      SatisfiesQ[pp][Rest[{S}]]];

SatisfiesQ[pp:At[f_,fun[pi_]]][{}] := True;

SatisfyCall[pi_, f_, state[E_,AS_,v_], {S__state}] :=
  With[{m = Length[AS]},
    With[{sigma = firstmlet[{S}]}],
      With[{t = Select[{S}, Length#[[2]] < m&, 1][[1]]},
        With[{ret = {Ret[pi] -> t[[3]]}},
          With[{phi = f /. Join[ret,sigma]},
            VerifyQ[ReleaseHold[phi]]]]]]];

firstmlet[{S__state}] :=
  With[{s = {S}[[1]]},
    With[{E = s[[1]]},
      If[MatchQ[Head[E], mlet],
        With[{AS = s[[2]]}, AS[[1]]],
        firstmlet[Rest[{S}]]]]];

```

A.1.3 Claims

(* Claims *)

```

claim::usage =
  "claim[r,E,P,At[f,pi]] is true if [r] E |-P At[f,pi]";

Assumptions::usage = "Assumptions[[r] E |-P At[f,pi]] = E";
Assertion::usage = "Assertion[[r] E |-P At[f,pi]] = At[f,pi]";

ValidQ::usage = "ValidQ[[r] E |-P At[f,pi]] is true if the
  claim is established by constructing

```

```

    the computation sequences of P and
    then doing direct verification with SatisfiesQ.";

(* Claim proof rules *)

From::usage = "From[pi] is a propositional
  variable indicating that
  we have arrived from a function call statement at pi.";

DeductionTheorem::usage = "Claim proof rule";
ModusPonens::usage = "Claim proof rule";
Resolution::usage = "Claim proof rule";
WidenAssumptions::usage = "Claim proof rule";
Tautology::usage = "Claim proof rule";
IntroduceTheorem::usage = "Claim proof rule";
PushPlace::usage = "Claim proof rule";
AssignmentRule::usage = "Claim proof rule";
SelectionRule::usage = "Claim proof rule";
BranchNotTaken::usage = "Claim proof rule";
IfTrue::usage = "Claim proof rule";
IfFalse::usage = "Claim proof rule";
AfterIf::usage = "Claim proof rule";
ReturnRule::usage = "Claim proof rule";
AllCalls::usage = "Claim proof rule";
OneCall::usage = "Claim proof rule";
CallInduce::usage = "Claim proof rule";

(* Claims *)

Assumptions[claim[r_,E_,P_,At[f_,pi_]]] := E;

Assertion[claim[r_,E_,P_,f_]] := f;

ValidQ[claim[r_,E_,P_,At[f_,pi_]]] :=
  With[{C = M[P][initial]},
    Every[TrimUpTo[r] /@ C] [
      Implies[SatisfiesQ[E][#], SatisfiesQ[At[f,pi]][#]]&];

TrimUpTo[fun[r_]] [S_] := S;

TrimUpTo[r_Symbol] [{_}] := {};

TrimUpTo[r_Symbol] [z:{s_, S__}] :=
  With[{E = s[[1]]},
    With[{h = Head[E]},
      If[MatchQ[h, place],
        If[MatchQ[E[[1]], r], z, TrimUpTo[r] [{S}]],
        TrimUpTo[r] [{S}]]];

claim[E_,P_,f_At] := claim[init, E, place[init,P], f];

Format[claim[pi_,E_,P_,f_]] :=
  SequenceForm["[", pi, "]" " ", E,

```

```

        Subscripted[" |-"["P"]], " ",
        f];

Format[At[f_,pi_]] := HoldForm[f^pi];

(* Claim proof rules *)

(* Pure logic rules *)

DeductionTheorem[] [claim[pi1_,E_,P_,At[f1_,pi_]],
                    claim[pi1_,F_,P_,At[f2_,pi2_]]] :=
    If[MatchQ[Union[E,{At[f1,pi]}],Union[F]],
        claim[pi1,E,P,At[f2,pi2]],
        Fail];

ModusPonens[] [claim[pi1_,E_,P_,At[f1_,pi_]],
               claim[pi1_,E_,P_,At[Implied[f1_,f2_],pi_]]] :=
    claim[pi1,E,P,At[f2,pi]];

Resolution[] [claim[pi1_,E_,P_,At[Not[f1_],pi_]],
              claim[pi1_,E_,P_,At[Or[f1_,f2_],pi_]]] :=
    claim[pi1,E,P,At[f2,pi]];

WidenAssumptions[F_] [claim[pi1_,E_,P_,At[f_,pi_]]] :=
    If[MatchQ[Complement[E,F],{}], claim[pi1,F,P,At[f,pi]], Fail];

Tautology[pi1_,E_,P_,At[f_,pi_]] [] :=
    If[MemberQ[E,At[f,pi]], claim[pi1,E,P,At[f,pi]], Fail];

IntroduceTheorem[pi1_,E_,P_,At[f_,pi_]] [] :=
    If[VerifyQ[f], claim[pi1,E,P,At[f,pi]], Fail];

(* Push Place *)

PushPlace[beta_] [claim[pi_,E_,P_,At[f_,alpha_]]] :=
    If[MatchQ[FindPlaces[alpha,beta][P], Fail], Fail,
        claim[pi, E, P, At[f, beta]]];

FindPlaces[alpha_, beta_] [P_] :=
    With[{try = GetPlace[P][alpha]},
        If[MatchQ[try, Fail], Fail, seqplace[beta][try]]];

seqplace[beta_] [seqplace[beta_, e_], r_] := True;

seqplace[beta_] [x_] := Fail;

(* Assignment Rule *)

AssignmentRule[b1_] [claim[pi_,E_,P_,At[phi_,b0_]]] :=
    If[MatchQ[pi, b1], Fail,
        With[{xe = FindAssignTo[anyassignment][b0,b1][P]},
            If[MatchQ[xe, Fail], Fail,
                With[{x = xe[[1]], e = xe[[2]]},

```

```

Module[{y},
  claim[pi, E, P,
    At[Exists[y, (phi /. x -> y) &&
      x == (e /. x -> y)], b1]]]]];

FindAssignTo[what_] [b0_, b1_] [e[X_]] := Fail;

FindAssignTo[what_] [b0_, b1_] [bot] := Fail;

FindAssignTo[what_] [b0_, b1_] [if[E_, E1_, E2_]] :=
  With[{t1 = FindAssignTo[what] [b0, b1] [E]},
    If[!MatchQ[t1, Fail], t1,
      With[{t2 = FindAssignTo[what] [b0, b1] [E1]},
        If[!MatchQ[t2, Fail], t2,
          FindAssignTo[what] [b0, b1] [E2]]]]];

FindAssignTo[what_] [b0_, b1_] [place[i_, E_]] :=
  FindAssignTo[what] [b0, b1] [E];

FindAssignTo[what_] [b0_, b1_] [choose[E_]] :=
  FindAssignTo[what] [b0, b1] [E];

FindAssignTo[what_] [b0_, b1_] [bind[i_Symbol, E_]] :=
  FindAssignTo[what] [b0, b1] [E];

FindAssignTo[what_] [b0_, b1_] [i_Symbol] := Fail;

FindAssignTo[what_] [b0_, b1_] [let[B_, E_]] :=
  With[{t1 = FindAssignTo[what] [b0, b1] [B]},
    If[!MatchQ[t1, Fail], t1, FindAssignTo[what] [b0, b1] [E]]];

FindAssignTo[what_] [b0_, b1_] [while[E_, E1_]] :=
  With[{t1 = FindAssignTo[what] [b0, b1] [E]},
    If[!MatchQ[t1, Fail], t1, FindAssignTo[what] [b0, b1] [E1]]];

FindAssignTo[what_] [b0_, b1_] [call[i_, {E_}]] :=
  FindAssignTos[what] [b0, b1] [{E}];

FindAssignTo[what_] [b0_, b1_] [par[E_]] :=
  FindAssignTos[what] [b0, b1] [{E}];

FindAssignTos[what_] [b0_, b1_] [{}] := Fail;

FindAssignTos[what_] [b0_, b1_] [{E_, F_}] :=
  With[{try = FindAssignTo[what] [b0, b1] [E]},
    If[!MatchQ[try, Fail], try,
      FindAssignTos[what] [b0, b1] [{F}]]];

FindAssignTo[what_] [b0_, b1_] [seq[a_]] :=
  FindAssignTo[what] [b0, b1] [a];

FindAssignTo[what_] [b0_, b1_] [seq[a_, b_, r_]] :=
  With[{try = what[b0, b1] [a, b]},

```

```

    If[!MatchQ[try, Fail], try,
      With[{try2 = FindAssignTo[what][b0,b1][a]},
        If[!MatchQ[try2, Fail], try2,
          FindAssignTo[what][b0,b1][seq[b,r]]]]];

anyassignment[b0_,b1_] [
  place[b0_, bind[x_,E1_]],
  place[b1_, E2_] := {x, E1};

anyassignment[b0_,b1_] [a_,b_] := Fail;

(* Selection Rule *)

SelectionRule[b1_] [claim[pi_,E_,P_,At[phi_,b0_]]] :=
  If[MatchQ[pi, b1], Fail,
    With[{xe = FindAssignTo[chooseassignment][b0,b1][P]},
      If[MatchQ[xe, Fail], Fail,
        With[{x = xe[[1]], e = xe[[2]]},
          Module[{y},
            claim[pi, E, P,
              At[Exists[y, (phi /. x -> y) &&
                InSet[x, (e /. x -> y)]]],
              b1]]]]];

chooseassignment[b0_,b1_] [
  place[b0_, bind[x_,choose[E1_]]],
  place[b1_, E2_] := {x, E1};

chooseassignment[b0_,b1_] [a_,b_] := Fail;

(* If True *)

IfTrue[b1_] [claim[pi_, E_, P_, At[phi_, b0_]]] :=
  With[{C = FindIfTrue[b0,b1][P] /. e[x_] -> x},
    If[MatchQ[C,Fail] || MatchQ[pi, b1], Fail,
      claim[pi, E, P, At[phi && C, b1]]];

FindIfTrue[b0_,b1_] [e[X_]] := Fail;

FindIfTrue[b0_,b1_] [bot] := Fail;

FindIfTrue[b0_,b1_] [if[E_,E1_,E2_]] :=
  With[{t1 = FindIfTrue[b0,b1][E]},
    If[!MatchQ[t1, Fail], t1,
      With[{t2 = FindIfTrue[b0,b1][E1]},
        If[!MatchQ[t2, Fail], t2, FindIfTrue[b0,b1][E2]]]]];

FindIfTrue[b0_,b1_] [place[b0_,if[C_, place[b1_, E1_], E2_]]] :=
  C;

FindIfTrue[b0_,b1_] [place[i_,E_]] := FindIfTrue[b0,b1][E];

FindIfTrue[b0_,b1_] [choose[E_]] := FindIfTrue[b0,b1][E];

```

```

FindIfTrue[b0_,b1_][bind[i_Symbol,E_]] := FindIfTrue[b0,b1][E];

FindIfTrue[b0_,b1_][i_Symbol] := Fail;

FindIfTrue[b0_,b1_][let[B_,E_]] :=
  With[{t1 = FindIfTrue[b0,b1][B]},
    If[!MatchQ[t1, Fail], t1, FindIfTrue[b0,b1][E]]];

FindIfTrue[b0_,b1_][while[E_,E1_]] :=
  With[{t1 = FindIfTrue[b0,b1][E]},
    If[!MatchQ[t1, Fail], t1, FindIfTrue[b0,b1][E1]]];

FindIfTrue[b0_,b1_][seq[a_]] := FindIfTrue[b0,b1][a];

FindIfTrue[b0_,b1_][seq[a_,r_]] :=
  With[{try = FindIfTrue[b0,b1][a]},
    If[!MatchQ[try, Fail], try,
      FindIfTrue[b0,b1][seq[r]]];

(* If False *)

IfFalse[b3_][claim[pi_, E_, P_, At[phi_, b0_]]] :=
  With[{C = FindIfFalse[b0,b3][P] /. e[x_] -> x},
    If[MatchQ[C,Fail] || MatchQ[pi, b3], Fail,
      claim[pi, E, P, At[phi && !C, b3]]];

FindIfFalse[b0_,b1_][e[X_]] := Fail;

FindIfFalse[b0_,b1_][bot] := Fail;

FindIfFalse[b0_,b1_][if[E_,E1_,E2_]] :=
  With[{t1 = FindIfFalse[b0,b1][E]},
    If[!MatchQ[t1, Fail], t1,
      With[{t2 = FindIfFalse[b0,b1][E1]},
        If[!MatchQ[t2, Fail],t2, FindIfFalse[b0,b1][E2]]]]];

FindIfFalse[b0_,b1_][place[i_,E_]] :=
  If[MatchQ[b0, i] && MatchQ[Head[E], if] &&
    MatchQ[Head[E[[3]]], place] &&
    MatchQ[E[[3,1]], b1],
    E[[1]],
    FindIfFalse[b0,b1][E]];

FindIfFalse[b0_,b1_][choose[E_]] := FindIfFalse[b0,b1][E];

FindIfFalse[b0_,b1_][bind[i_Symbol,E_]] :=
  FindIfFalse[b0,b1][E];

FindIfFalse[b0_,b1_][i_Symbol] := Fail;

FindIfFalse[b0_,b1_][let[B_,E_]] :=
  With[{t1 = FindIfFalse[b0,b1][B]},

```

```

    If[!MatchQ[t1, Fail], t1, FindIfFalse[b0,b1][E]];

FindIfFalse[b0_,b1_][while[E_,E1_]] :=
  With[{t1 = FindIfFalse[b0,b1][E]},
    If[!MatchQ[t1, Fail], t1, FindIfFalse[b0,b1][E1]]];

FindIfFalse[b0_,b1_][seq[a_]] := FindIfFalse[b0,b1][a];

FindIfFalse[b0_,b1_][seq[a_,r_]] :=
  With[{try = FindIfFalse[b0,b1][a]},
    If[!MatchQ[try, Fail], try,
      FindIfFalse[b0,b1][seq[r]]];

(* After If *)

AfterIf[bplus_] [claim[pi_,E_,P_,At[phi1_,b2_]],
             claim[pi_,E_,P_,At[phi2_,b4_]]] :=
  If[MatchQ[pi, bplus], Fail,
    If[MatchQ[FindAfterIf[b2,b4][P], Fail], Fail,
      claim[pi, E, P, At[phi1 || phi2, bplus]]];

FindAfterIf[b0_,b1_][e[X_]] := Fail;

FindAfterIf[b0_,b1_][bot] := Fail;

FindAfterIf[b0_,b1_][if[E_,E1_,E2_]] :=
  With[{t1 = FindAfterIf[b0,b1][E]},
    If[!MatchQ[t1, Fail], t1,
      With[{t2 = FindAfterIf[b0,b1][E1]},
        If[!MatchQ[t2, Fail],t2, FindAfterIf[b0,b1][E2]]]]];

FindAfterIf[b0_,b1_][place[i_,E_]] := FindAfterIf[b0,b1][E];

FindAfterIf[b0_,b1_][choose[E_]] := FindAfterIf[b0,b1][E];

FindAfterIf[b0_,b1_][bind[i_Symbol,E_]] :=
  FindAfterIf[b0,b1][E];

FindAfterIf[b0_,b1_][i_Symbol] := Fail;

FindAfterIf[b0_,b1_][let[B_,E_]] :=
  With[{t1 = FindAfterIf[b0,b1][B]},
    If[!MatchQ[t1, Fail], t1, FindAfterIf[b0,b1][E]]];

FindAfterIf[b0_,b1_][while[E_,E1_]] :=
  With[{t1 = FindAfterIf[b0,b1][E]},
    If[!MatchQ[t1, Fail], t1, FindAfterIf[b0,b1][E1]]];

FindAfterIf[b0_,b1_][call[i_, {E_}]] :=
  FindAfterIfs[b0,b1][{E}]

FindAfterIf[b0_,b1_][par[E_]] := FindAfterIfs[b0,b1][{E}]

```



```

FindAfterIfs[b0_,b1_] [{}] := Fail;

FindAfterIfs[b0_,b1_] [{E_,F___}] :=
  With[{try = FindAfterIf[b0,b1][E]},
    If[!MatchQ[try, Fail], try,
      FindAfterIfs[b0,b1] [{F}]]];

FindAfterIf[b0_,b1_][seq[a_]] := Fail;

FindAfterIf[b0_,b1_][seq[a_,b_,r___]] :=
  With[{try = afterif[b0,b1][a,b]},
    If[!MatchQ[try, Fail], try,
      FindAfterIf[b0,b1][seq[b,r]]];

afterif[b0_,b1_][
  place[_ ,
    if[E0_,
      place[_ , seq[_ , place[b0_, _]]],
      place[_ , seq[_ , place[b1_, _]]]],
  place[bplus_, E3_]] := True;

afterif[b0_,b1_][
  if[E0_,
    place[_ , seq[_ , place[b0_, _]]],
    place[_ , seq[_ , place[b1_, _]]],
  place[bplus_, E3_]] := True;

afterif[b0_,b1_][
  if[E0_,
    seq[E11_, place[b0_, E12_]],
    seq[E21_, place[b1_, E22_]],
  place[bplus_, E3_]] := True;

afterif[b0_,b1_][a_,b_] := Fail;

(* Call Rule *)

CallRule[a_] [claim[pi1_,E_,P_,At[phi1_,b]],
  claim[pi1_,E_,P_,At[phi2_,fun[f_]]]] :=
  With[{xepi = FindCall[b,f][P]},
    If[MatchQ[xepi, Fail], Fail,
      With[{x = xepi[[1]], e = xepi[[2]], pi = xepi[[3]]},
        If[MatchQ[pi, pi1], Fail,
          With[{m = Length[a]},
            Module[{y},
              claim[pi1, E, P,
                At[Exists[y,
                  (phi1 /. x -> y) &&
                  (phi2 /.
                    Join[{y -> x},
                      (a[[#]] ->
                        (e[[#]] /. y -> x))&
                        /@ Range[m]])]],

```

```

pi]]]]]]]]];

FindCall[b_,f_][e[X_]] := Fail;

FindCall[b_,f_][bot] := Fail;

FindCall[b_,f_][if[E_,E1_,E2_]] :=
  With[{t1 = FindCall[b,f][E]},
    If[!MatchQ[t1, Fail], t1,
      With[{t2 = FindCall[b,f][E1]},
        If[!MatchQ[t2, Fail], t2, FindCall[b,f][E2]]]]];

FindCall[b_,f_][place[i_,E_]] := FindCall[b,f][E];

FindCall[b_,f_][choose[E_]] := FindCall[b,f][E];

FindCall[b_,f_][bind[i_Symbol,E_]] := FindCall[b,f][E];

FindCall[b_,f_][i_Symbol] := Fail;

FindCall[b_,f_][let[B_,E_]] :=
  With[{t1 = FindCall[b,f][B]},
    If[!MatchQ[t1, Fail], t1, FindCall[b,f][E]]];

FindCall[b_,f_][while[E_,E1_]] :=
  With[{t1 = FindCall[b,f][E]},
    If[!MatchQ[t1, Fail], t1, FindCall[b,f][E1]]];

FindCall[b_,f_][call[i_, {E__}]] := FindCalls[b,f][{E}]

FindCall[b_,f_][par[E__]] := FindCalls[b,f][{E}]

FindCalls[b_,f_][{}] := Fail;

FindCalls[b_,f_][{E_,F__}] :=
  With[{try = FindCall[b,f][E]},
    If[!MatchQ[try, Fail], try,
      FindCalls[b,f][{F}]]];

FindCall[b_,f_][seq[a_]] := Fail;

FindCall[beta_,f_][seq[a_,b_,r__]] :=
  With[{try = findcall[beta,f][a,b]},
    If[!MatchQ[try, Fail], try,
      FindCall[beta,f][seq[b,r]]];

findcall[b_,f_][
  place[b_,bind[x_,call[f_,E_]]],
  place[pi_,E0_] := {x,E,pi};

findcall[b_,f_][x_,y_] := Fail;

```

A.1.4 Praas

```
(* Programs with assumptions and assertions *)

Praa::usage = "Praa[P,E,F], where P
  is a program and E and F are lists
  of propositions, respectively the
  assumptions and assertions of P, is
  a program with assumptions and assertions or Praa.";

Program::usage = "Program[R] gives the program of Praa R.";
Assumptions::usage =
  "Assumptions[R] gives the assumptions of Praa R.";
Assertions::usage =
  "Assertions[R] gives the assertions of Praa R.";
Assumption::usage =
  "Assumption[i][R] gives the ith assumption of R.";

Assertion::usage = "An assertion is a proposition at a place.
  Assertion[i][R] gives the ith assertions of R.
  Assertion[c] gives the assertion of a claim.";

Claim::usage =
  "Claim[i][R] gives the claim of the ith assertion of R";

(* Praa transformation rules *)

VacuousPraa::usage =
  "VacuousPraa[P,E] is (P,E,{}), the vacuously correct Praa.";

VacuousRule::usage =
  "VacuousRule[(P,E,{})] = ((P,E,{}), {})"

Fail::usage = "Indicates that a proof rule doesn't apply";

EqReduceLeft::usage = "Praa transformation rule";
EqReduceRight::usage = "Praa transformation rule";
Choice::usage = "Praa transformation rule";
Subset::usage = "Praa transformation rule";
DeleteDead::usage = "Praa transformation rule";
ReplaceDead::usage = "Praa transformation rule";

(* Programs with assumptions and assertions *)

Program[Praa[P_,{E___At},{F___At}]] := P;
Assumptions[Praa[P_,{E___At},{F___At}]] := {E};
Assertions[Praa[P_,{E___At},{F___At}]] := {F};
Assumption[i_Integer][Praa[P_,{E___At},{F___At}]] := {E}[[i]];
Assertion[i_Integer][Praa[P_,{E___At},{F___At}]] := {F}[[i]];
Claim[i_Integer][Praa[P_,E_,F_]] := claim[E, P, F[[i]]];

VacuousPraa[P_,{E___At}] := Praa[P,{E},{}];
```

```

Format[Praa[P_,{E___At},{F___At}]] :=
  ColumnForm[{"Praa", "====",
    SequenceForm["program: ", P], "",
    SequenceForm["assumptions: ", {E}], "",
    SequenceForm["assertions: ", {F}]]];

EqReduceLeft[F_,e1_,e2_,pi_] [Praa[P_,{E___At},{G___At}]] :=
  If[!MatchQ[Union[{G}], Union[F, {At[e1 == e2, pi]}]],
    Fail,
    With[{expr = GetPlace[P][pi]},
      If[!MatchQ[expr,e1], Fail,
        Praa[P /. e1 -> e2, {E}, {G}]]];

EqReduceRight[{F___At},e1_,e2_,pi_] [Praa[P_,{E___At},{G___At}]] :=
  If[!MatchQ[Union[{G}], Union[F, {At[e1 == e2, pi]}]], Fail,
    With[{expr = GetPlace[P][pi]},
      If[!MatchQ[expr,e2], Fail,
        Praa[P /. e2 -> e1, {E}, {G}]]];

Choice[{F___At},e1_,e2_,pi_] [Praa[P_,{E___At},{G___At}]] :=
  If[!MatchQ[Union[{G}], Union[{F},
    {At[InSet[e1,e2], pi]}]], Fail,
    With[{expr = GetPlace[P][pi]},
      If[!MatchQ[expr, choose[e2]], Fail,
        Praa[ReplacePlace[P][pi,e1], {E}, {G}]]];

Subset[{F___At},e1_,e2_,pi_] [Praa[P_,{E___At},{G___At}]] :=
  If[!MatchQ[Union[{G}], Union[{F},
    {At[SubSetEq[e1,e2], pi]}]], Fail,
    With[{expr = GetPlace[P][pi]},
      If[!MatchQ[expr, choose[e2]], Fail,
        Praa[ReplacePlace[P][pi,choose[e1]], {E}, {G}]]];

DeleteDead[{F___At},e1_,e2_,pi_] [Praa[P_,{E___At},{G___At}]] :=
  If[!MatchQ[Union[{G}], Union[{F}, {At[False, pi]}]], Fail,
    Praa[DeletePlace[P][pi], {E}, {F}]];

ReplaceDead[{F___At},e1_,e2_,pi_,s_] [Praa[P_,{E___At},{G___At}]] :=
  If[!MatchQ[Union[{G}], Union[{F}, {At[False, pi]}]], Fail,
    Praa[ReplacePlace[P][pi,s], {E}, {G}]];

AddAssumption[f_At] [Praa[P_,E_,F_]] := Praa[P, Join[E,{f}], F];

```

A.1.5 Claim proofs

(* Proof of a claim *)

```

cproof::usage =
  "cproof[s1,...,si] is a sequence of claim proof steps";

```

```

cproofStep::usage = "cproofStep[c,r,A]
  establishes claim c by applying

```

```

    rule r to arguments A.";

Attempt::usage = "Attempt[X_, R_, A_] applies R to X.
  If R returns Fail, so does the Attempt,
  otherwise the Attempt extends X.";

ValidRule::usage =
  "ValidRule[X][c] is just if ValidQ[c] is True then
  [(c, ValidRule, [c])] else [].
  ValidRule[PI][i] extends the proof of fi by one step";

Claim::usage = "Claim[X][i] gives the ith claim of proof X";

Try::usage = "Try[v,f] looks at v; if v == Fail,
  then Fail, otherwise f[v]";

(* Proof of a claim *)

Try[v_,f_] := If[MatchQ[v,Fail], Fail, f[v]];

Assertion[cproofStep[c_, R_, A_]] := Assertion[c];

Claim[cproofStep[c_,R_,A_]] := c;

Format[cproofStep[c_, R_, A_]] :=
  SequenceForm[c, "  (", Head[R], ")"];

Claim[X_cproof][i_Integer] := Claim[X[[i]]];

Format[cproof[s___cproofStep]] :=
  ColumnForm[Join[{"proof of claim"},
    stepnumber[#, {s}][[#]]& /@
    Range[Length[{s}]]]];

Format[stepnumber[i_,c_]] := SequenceForm[i, ". ", c];

Attempt[X_cproof, R_, A_] :=
  Try[R @@ (Claim[X] /@ A),
    Join[X, cproof[cproofStep[#, R, A]]&];

```

A.1.6 Praa proofs

```

(* Proof of a Praa *)

Ppc::usage = "Ppc[R,Xe,Xf] is a Praa proof of correctness
  of Praa R with auxiliary proofs of assumptions Xe and proofs
  of assertions Xf";

PraaOf::usage = "Praa proof accessor";
AssumptionProofs::usage = "Praa proof accessor";
AssertionProofs::usage = "Praa proof accessor";
Program::usage = "Praa proof accessor";

```

```

Assumptions::usage = "Praa proof accessor";
Assertions::usage = "Praa proof accessor";
Assumption::usage = "Praa proof accessor";
Assertion::usage = "Praa proof accessor";
Claim::usage = "Praa proof accessor";
Claim::usage = "Praa proof accessor";

VacuousRule::usage = "Praa proof rule";
Attempt::usage = "Praa proof rule";
Promote::usage = "Praa proof rule";

(* Proof of a Praa *)

Format[Ppc[R_Praa,{XE___cproof}, {XF___cproof}]] :=
  ColumnForm[{"Praa proof of correctness", R,
    SequenceForm["Assumptions: ",
      ColumnForm[{"Claim proofs: ", XE}]],
    SequenceForm["Assertions: ",
      ColumnForm[{"Claim proofs: ", XF}]]}];

PraaOf[Ppc[R_Praa, _, _]] := R;

AssumptionProofs[Ppc[_Praa, X_, _]] := X;

AssertionProofs[Ppc[_Praa, _, X_]] := X;

Program[Ppc[R_Praa, _, _]] := Program[R];

Assumptions[Ppc[R_Praa, _, _]] := Assumptions[R];

Assertions[Ppc[R_Praa, _, _]] := Assertions[R];

Assumption[i_Integer][Ppc[R_Praa, _, _]] := Assumption[i][R];

Assertion[i_Integer][Ppc[R_Praa, _, _]] := Assertion[i][R];

Claim[i_Integer][Ppc[R_, XE_, XF_]] := Claim[Last[XF[[i]]]];

Claim[i_Integer, j_Integer][Ppc[R_, XE_, XF_]] :=
  Claim[j][XE[[i]]];

VacuousRule[P_, {E___At}] [] :=
  Ppc[VacuousPraa[P, {E}], cproof[] & /@ {E}, {}];

Attempt[D_, A_, i_Integer][Pi:Ppc[R:Praa[P_, E_, F_], XE_, XF_]] :=
  Try[Attempt[XE[[i]], D, A],
    Ppc[R, Join[Take[XE, i-1], {#}, Drop[XE, i]], XF]&];

Promote[i_Integer][Ppc[Praa[P_, E_, F_], {XE___cproof},
  {XF___cproof}]] :=
  Module[{X, Y},
    X = {XE}[[i]];
    Y = E[[i]];

```

```

    If[MatchQ[Y,Assertion[Last[X]]],
      Ppc[Praa[P,Join[Take[E,i-1], Drop[E,i]], Join[F, {Y}]],
        Join[Take[{XE},i-1], Drop[{XE},i]],
        Join[{XF}, {X}]],
      Fail]];

```

A.2 Rule Set Analyser

(* Operation or relation on sets *)

```

Exists::usage =
  "Exists[S,p] is true if there is one s in S such that p[S]";

```

```

IntersectQ::usage =
  "IntersectQ[A,B] is true if there is an s in A and B";

```

```

NoDuplicates::usage =
  "NoDuplicates[L] removes duplicates from L with
  changing the original order of elements in L";

```

```

Split::usage =
  "Split[S,P] breaks S into A and B
  such that P is true on A and false on B";

```

```

NonEmptyRankedPowerSet::usage =
  "NonEmptyRankedPowerSet[n] constructs the
  nonempty elements of the power set of {1..n},
  returning the elements grouped by size";

```

(* Mathematical structure descriptions *)

```

CartesianProduct::usage =
  "CartesianProduct[{S1,...,Sn}] is the set of
  n-vectors of elements where component i is an element of Si.
  CartesianProduct[S_,k_Integer] is the set of k-vectors where
  the components are elements of S";

```

```

Nat::usage = "Nat[n_Integer] is the set of integers from 0 to n";

```

```

Pos::usage = "Pos[n_Integer] is the set of integers from 1 to n";

```

```

PowerSet::usage =
  "PowerSet[S_] is the power set of elements from set S";

```

```

Vector::usage =
  "Vector[m_Integer, k_Ineger] is the m-vector of k's";

```

(* Operations on described mathematical structures *)

```

Cardinality::usage =
  "Cardinality[descr] computes the number of elements in
  the described mathematical structure";

```

```

Construct::usage =
  "Construct[descr] creates a mathematical structure according
  to its description";

(* Computational elementary model theory *)

(* Elementary relational structures *)

ERS::usage =
  "Elementary relational structure
  ERS[U,relations,constants,beta]
  where U is the name of a domain, relations
  is the sequence of operations,
  constants is the sequence of constants,
  and beta is the arity
  of the relations";

Domain::usage = "Domain[ERS[U,relations,constants,beta]] = U";

Relns::usage =
  "Relns[ERS[U,relations,constants,beta]] = relations";

Consts::usage =
  "Consts[ERS[U,relations,constants,beta]] = constants";

Arity::usage = "Arity[ERS[U,relations,constants,beta]] = beta";

ElementaryType::usage = "ElementaryType[m,beta] gives the
  type of an ERS, where m is the number of constants and beta
  the arities of the relations";

delta::usage =
  "delta[ERS[U,relations,constants,beta]] =
  ElementaryType[Length[constants], beta]";

(* Elementary positive conjunct parametrised
  propositional language *)

(* Abstract syntax *)

(* Syntactic categories *)

EPCPPLVars::usage =
  "EPCPPLVars[delta][n] describes the variables of
  an elementary positive conjunct
  parametrised propositional language
  of elementary type delta and n variables";

EPCPPLConsts::usage =
  "EPCPPLConsts[delta][n] describes the constants of
  an elementary positive conjunct
  parametrised propositional language

```



```

    of elementary type delta and n variables";

EPCPPLPredLets::usage =
    "EPCPPLPredLets[delta][n] describes the predicate letters of
    an elementary positive conjunct
    parametrised propositional language
    of elementary type delta and n variables";

EPCPPLTerms::usage =
    "EPCPPLTerms[delta][n] describes the terms of
    an elementary positive conjunct
    parametrised propositional language
    of elementary type delta and n variables";

EPCPPLAtoms::usage =
    "EPCPPLAtoms[delta][n] describes the atoms of
    an elementary positive conjunct
    parametrised propositional language
    of elementary type delta and n variables";

EPCPPL::usage =
    "EPCPPL[delta][n] describes an elementary positive conjunct
    parametrised propositional
    language of elementary type delta and n variables";

EPCPPLTermSubsts::usage =
    "EPCPPLTermSubsts[delta][n] describes the
    term substitutions of an elementary
    positive conjunct parametrised
    propositional language of elementary type
    delta and n variables";

(* Constructors of elements in syntactic categories *)

IndConst::usage =
    "IndConst[i] makes the ith individual constant";

IndVar::usage = "IndVar[i] makes the ith individual variable";

PredicateLetter::usage =
    "PredicateLetter[i] makes the ith predicate letter";

Term::usage =
    "Term[x] makes variable or constant x into a term";

Conjunct::usage =
    "Conjunct[x__] makes a conjunct of atoms x__ (the empty
    conjunct is allowed)";

(* Operations on abstract syntax categories *)

Stratified::usage =
    "Stratified[L][k] are the sentences

```

```

    of L with exactly k variables";

Stratify::usage =
  "Stratify[F][n] gets the stratified value of F at n";

(* Operations on conjuncts *)

Canonical::usage = "Canonical[p] gives the canonical form of p";

Deg::usage =
  "Deg[p] gives the number of distinct variables in p";

Occurs::usage = "Occurs[p] gives the distinct variables in p";

Precedes::usage =
  "Precedes[x,y] is true if x precedes y in lexicographic order";

TermSubstitute::usage =
  "TermSubstitute[s][p] applies substitution s to p";

(* Realization, Denotation and Model *)

Denotation::usage =
  "Denotation[ERS[A,R,C,beta],a][t]
   gives the denotation of term t in the
   given elementary relational structure";

Satisfies::usage =
  "Satisfies[U,a][p] is true if p in realization U at term
   substitution a evaluates to true";

(* Patterns *)

(* Abstract syntax *)

IsMoreGeneral::usage =
  "IsMoreGeneral[EPCPPL[delta][n]][p,q]
   is true if p subsumes q";

Matches::usage =
  "Matches[EPCPPL[delta][n]][P] describes
   the sentences of the EPCPPL
   of type delta over n variables which match pattern set P";

PatSet::usage = "PatSet[P] is a set of patterns P";

(* Operations on abstract syntax categories *)

EPCPPLPatSets::usage =
  "EPCPPLPatSets[delta][n] is the power set of EPCPPL[delta][n]";

(* Operations on patterns and pattern sets *)

```

```

Conjoin::usage =
  "Conjoin[A,B] describes the conjunction of two pattern sets";

Disjoin::usage =
  "Disjoin[A,B] describes the conjunction of two pattern sets";

MatchesQ::usage =
  "MatchesQ[P,pi] is true if pattern set P matches sentences w";

MostGeneral::usage =
  "MostGeneral[P] constructs the most general
    form of pattern set P";

(* Combinatorial aspects of Gedanken *)

(* Counting pattern set matches in Lkmn *)

(* Lkmn *)

L::usage =
  "L[k,m,n] is the language of k m-ary relations in n variables";

PC::usage =
  "PC[k] is the language of k propositional constants";

(* Match count in Lkmn by projection to Lkn,m-1,n *)

Elide::usage =
  "Elide[x,L] is list L with occurrences of x deleted";

ExpandVars::usage = "ExpandVars[n,w,P_PatSet]";

RetractRelations::usage =
  "RetractRelations[P] renumbers relations in P so that
    the minimum number of relation names is used";

(* Match counting in a mixed arity language with a constant *)

AdjacentLWithK::usage =
  "AdjacentLWithK[j,k,m,n] describes the language
    of j m-1 ary relations and k m-ary relations in n variables";

Emls::usage =
  "Emls[n] is elementary multilevel syllogistic over n variables";

(* The Model Graph algorithm for deciding EMLS *)

SMGVars::usage =
  "SMGVars[n] is the number of nodes of
    a model graph over n variables";

SMG::usage =
  "SMG[n] is the number of distinct model

```

```

graphs over n variables";

UpperBound::usage =
  "UpperBound[descr] computes an upper bound on the value of an
  integer quantity described by descr";

Log2::usage = "Log2[x] gives the numerical log base 2 of x";

(* Operation or relation on sets *)

(* Exists, IntersectQ, NoDuplicates,
  Split, NonEmptyRankedPowerSet *)

Exists[S_,p_] := Not[MatchQ[Select[S,p,1], {}]];

IntersectQ[A_,B_] := Not[MatchQ[Intersection[A,B], {}]];

Contains[S_,T_] := MatchQ[Complement[T,S], {}];

NoDuplicates[{}] := {};

NoDuplicates[{x_}] := {x};

NoDuplicates[{x_,r__}] :=
  Join[{x}, NoDuplicates[If[MemberQ[{r},x],
    Elide[x, {r}], {r}]]];

Split[{}, P_] := {{},{}};

Split[{a_}, P_] := If[P[a], {{a},{}}, {{},{a}}];

Split[{r__}, P_] :=
  Block[{S, fr}, S = Split[Rest[{r}], P]; fr = First[{r}];
  If[P[fr],
    Join[{Join[{fr}, S[[1]]], {S[[2]]}],
    Join[{S[[1]]}, {Join[{fr}, S[[2]]}]]];

GroupByRank[n_,C_,{}] := {C};

GroupByRank[n_,C_,{s__}] :=
  Block[{h,t,l},
    h = First[{s}]; t = Rest[{s}];
    If[Length[h] == n,
      GroupByRank[n,Join[C,{h}],t],
      Join[{C}, GroupByRank[n+1,{},s]]];

NonEmptyRankedPowerSet[0] := {};

NonEmptyRankedPowerSet[n_] :=
  GroupByRank[1,{},Rest[Construct[PowerSet[Pos[n]]]]];

(* Mathematical structure descriptions *)

```

```

(* CartesianProduct, Nat,
   Pos, PowerSet, Vector *)

(* Operations on described mathematical structures *)

(* Cardinality, Construct *)

Cardinality[x_List] := Length[x];

Cardinality[Nat[n_]] := n+1;

Cardinality[Pos[n_]] := n;

Cardinality[Vector[m_,k_]] := m;

Cardinality[PowerSet[S_]] := 2^Cardinality[S];

Cardinality[CartesianProduct[x_List]] :=
  Times @@ (Cardinality /@ x);

Cardinality[CartesianProduct[S_,k_Integer]] :=
  Cardinality[S]^k;

Construct[x_List] := x;

Construct[Nat[n_]] := Table[i, {i,0,n}];

Construct[Pos[n_]] := Table[i, {i,1,n}];

Construct[Vector[m_,k_]] := Table[k, {i,1,m}];

Construct[PowerSet[{}]] := {};

Construct[PowerSet[{x_}]] := {{}, {x}};

Construct[PowerSet[{x_,r_}]] :=
  Block[{P},
    P = Construct[PowerSet[{r}]];
    Union[P, Union[#, {x}]& /@ P]];

Construct[PowerSet[S_]] :=
  Construct[PowerSet[Construct[S]]] /; Not[ListQ[S]]

Construct[CartesianProduct[{}]] := {};

Construct[CartesianProduct[{x_}]] := {#}& /@ Construct[x];

Construct[CartesianProduct[{x_,r_}]] :=
  (Join @@
    (Join @@
      (Function[pi, {Join[{#}, pi]}] /@
        Construct[CartesianProduct[{r}]])& /@
      Construct[x]))

```

```

Construct[CartesianProduct[S_,k_Integer]] :=
  Block[{CS}, CS = Construct[S];
    Construct[CartesianProduct[Table[CS, {i,1,k}]]]]

(* Computational elementary model theory *)

(* Elementary relational structures *)

(* ERS, Arity, Consts, Domain, Relns, ElementaryType, delta *)

Domain[ERS[U_,relations_,constants_,beta_]] := U;

Relns[ERS[U_,relations_,constants_,beta_]] := relations;

Consts[ERS[U_,relations_,constants_,beta_]] := constants;

Arity[ERS[U_,relations_,constants_,beta_]] := beta

delta[ERS[U_,relations_,constants_,beta_]] :=
  ElementaryType[Length[constants], beta]

(* Elementary positive conjunct parametrised
   propositional language *)

(* Abstract syntax *)

(* Syntactic categories *)

(* EPCPPL, EPCPPLAtoms, EPCPPLConsts, EPCPPLPredLets,
   EPCPPLTerms, EPCPPLTermSubsts, EPCPPLVars *)

Construct[EPCPPLVars[delta_][n_]] :=
  Table[IndVar[i], {i, 1, n}]

Construct[EPCPPLConsts[ElementaryType[m_,beta_]][n_]] :=
  Table[IndConst[i], {i, 1, m}]

Construct[EPCPPLTerms[delta_][n_]] :=
  Join[Construct[EPCPPLConsts[delta][n]],
    Construct[EPCPPLVars[delta][n]]]

Construct[EPCPPLTermSubsts[delta_][n_]] :=
  Construct[CartesianProduct[EPCPPLTerms[delta][n],n]]

Construct[EPCPPLPredLets[ElementaryType[m_,beta_]]] :=
  Table[PredicateLetter[i], {i, 1, Cardinality[beta]}]

Construct[EPCPPLAtoms[delta:ElementaryType[m_,beta_]][n_]] :=
  Block[{R, T, J, b},
    R = Construct[EPCPPLPredLets[delta]];
    T = EPCPPLTerms[delta][n];
    J = Table[i, {i, 1, Cardinality[beta]}];

```

```

    b = Construct[beta];
    Join @@
    (Function[{i},
      R[[i]]@#@#& /@
      If[b[[i]] == 0, {}},
      Construct[CartesianProduct[T,b[[i]]]]]]
    /@ J)]

Construct[EPCPPL[delta_][n_]] :=
  Conjunct@#@#& /@ Construct[PowerSet[EPCPPLAtoms[delta][n]]]

Construct[Stratified[EPCPPL[delta_][k_]]] :=
  Select[Construct[EPCPPL[delta][k]], Deg[#]==k&]

Construct[Matches[EPCPPL[delta_][n_][P_]]] :=
  Select[Construct[EPCPPL[delta][n]],
    IsMoreGeneral[EPCPPL[delta][n][P,#]&];

Construct[Matches[L[k_,0,n_][r_PatSet]]] :=
  Select[Construct[L[k,0,n]], MatchesQ[r,#]&]

Construct[EPCPPLPatSets[delta_][n_]] :=
  (PatSet@#@#)& /@ Construct[PowerSet[EPCPPL[delta][n]]]

Cardinality[EPCPPLPatSets[delta_][n_]] :=
  Cardinality[Construct[EPCPPLPatSets[delta][n]]]

Cardinality[EPCPPLVars[delta_][n_]] := n

Cardinality[EPCPPLConsts[ElementaryType[m_,beta_][n_]]] := m

Cardinality[EPCPPLTerms[ElementaryType[m_,beta_][n_]]] := n+m

Cardinality[EPCPPLTermSubsts[delta_][n_]] :=
  Cardinality[EPCPPLTerms[delta][n]]^n

Cardinality[EPCPPLPredLets[ElementaryType[m_,beta_]]] :=
  Cardinality[beta]

Cardinality[EPCPPLAtoms[ElementaryType[m_,beta_List][n_]]] :=
  Plus @@ ((n+m)^#& /@ beta)

Cardinality[EPCPPLAtoms[ElementaryType[m_,Vector[L_,k_]]][n_]] :=
  L*(n+m)^k

Cardinality[EPCPPL[delta_][n_]] :=
  2^(Cardinality[EPCPPLAtoms[delta][n]])

(* Constructors of elements in syntactic categories *)

(* IndConst, IndVar, PredicateLetter, Term, Conjunct *)

IndVar[j_] := j

```

```

Term[x_] := x

PredicateLetter[i_] := i

(* Operations on abstract syntax categories *)

(* Stratified, Stratify *)

Stratify[F_][0] := F[0];

Stratify[F_][n_] :=
  F[n] - Sum[Binomial[n,i] F[n-i], {i,1,n}] /; n > 0

Cardinality[Stratified[L_][k_]] :=
  Stratify[Cardinality[L[#]]&][k]

(* Operations on conjuncts *)

(* Canonical, Deg, Occurs, Precedes, TermSubstitute *)

Canonical[p_Conjunct] :=
  Block[{V,W,i,j,n},
    V = Occurs[p];
    n = Length[V];
    Conjunct @@
      (Sort[Union[List @@
        If[n>0,
          W = Table[1, {i, 1, Max @@ V}];
          For[j=1,j<=n,j++,W[[V[[j]]]] = j];
          TermSubstitute[W][p],
          p]],
        Precedes]])

Deg[pi_] := Length[Occurs[pi]]

Occurs[x_Integer] := {x}

Occurs[x_IndConst] := {}

Occurs[reln_] := {};

Occurs[reln_[args_]] := Union @@ (Occurs /@ {args})

Precedes[{}], x_List] := True;

Precedes[{x_}, {}] := False;

Precedes[{x_,r_}, {}] := False;

Precedes[{p_}, {r_}] := p < First[{r}]

Precedes[{p_,q_}, {r_}] :=

```



```

(p < First[{r}]) || (p == First[{r}] &&
  Precedes[{q}, Rest[{r}]]);

Precedes[x1_Integer[], x2_Integer[]] := x1 < x2;

Precedes[x1_Integer[], x2_Integer[a2_Integer]] := x1 <= x2;

Precedes[x1_Integer[a1_Integer], x2_Integer[]] := x1 < x2;

Precedes[x1_Integer[a1_Integer], x2_Integer[a2_Integer]] :=
  (x1 < x2) || ((x1 == x2) && Precedes[{a1}, {a2}])

Precedes[Conjunct[], x_Conjunct] := True;

Precedes[Conjunct[x_], Conjunct[]] := False;

Precedes[Conjunct[x_, r_], Conjunct[]] := False;

Precedes[Conjunct[p_], Conjunct[r_]] :=
  Precedes[p, First[{r}]];

Precedes[Conjunct[p_, q_], Conjunct[r_]] :=
  (Precedes[p, First[{r}]] ||
   MatchQ[p, First[{r}]] && Precedes[Conjunct[q],
    Conjunct @@ Rest[{r}]])

TermSubstitute[s_][x_Integer] := s[[x]]

TermSubstitute[s_][x_IndConst] := x

TermSubstitute[s_][reln_] := reln[];

TermSubstitute[s_][reln_[args_]] :=
  reln@@(TermSubstitute[s]/@{args})

TermSubstitute[s_][atoms_Conjunct] :=
  TermSubstitute[s] /@ atoms

(* Realization, Denotation and Model *)

(* Denotation, Satisfies *)

Denotation[U_, a_][x_Integer] := a[[x]];

Denotation[ERS[A_, R_, C_, beta_], a_][IndConst[x_]] := C[[x]]

Satisfies[U_, a_][J_Conjunct] :=
  And @@ (Satisfies[U, a][#] & /@ J);

Satisfies[U:ERS[A_, R_, C_, beta_], a_][p_[ti_]] :=
  R[[p]] @@ (Denotation[U, a][#] & /@ {ti})

(* Patterns *)

```

```

(* Abstract syntax *)

(* IsMoreGeneral, Matches, PatSet *)

IsMoreGeneral[EPCPPL[delta_][0]][p_Conjunct,q_Conjunct] :=
  Contains[List @@ q, List @@ p];

IsMoreGeneral[EPCPPL[delta_][n_]][p_Conjunct,q_Conjunct] :=
  Exists[Construct[EPCPPLTermSubsts[delta][n]],
    Contains[List @@ q,
      List @@ TermSubstitute[#][p]]&] /; n > 0

IsMoreGeneral[EPCPPL[delta_][n_]][P_PatSet,q_Conjunct] :=
  Exists[List @@ P,IsMoreGeneral[EPCPPL[delta][n]][#,q]&];

Cardinality[Matches[EPCPPL[delta_][n_]][P_]] :=
  Cardinality[Construct[Matches[EPCPPL[delta][n]][P]]];

Cardinality[Matches[Stratified[L_][n_]][S_]] :=
  Stratify[Cardinality[Matches[L[#]][S]]&][n]

(* Operations on abstract syntax categories *)

(* EPCPPLPatSets *)

(* Operations on patterns and pattern sets *)

(* Canonical, Conjoin, Disjoin, MatchesQ,
  MostGeneral, TermSubstitute *)

Canonical[PatSet[]] := PatSet[];

Canonical[PatSet[p_]] :=
  PatSet @@ Sort[Union[Canonical/@{p}],Precedes]

Disjoin[A_PatSet,B_PatSet] := Union[A,B]

MatchesQ[PatSet[r_],pi_] :=
  Exists[{r}, Contains[List @@ pi, List @@ #]&]

MostGeneral[PatSet[]] := PatSet[];

MostGeneral[PatSet[x_]] := PatSet[x];

MostGeneral[PatSet[x_,r_]] :=
  Block[{s}, s = MostGeneral[PatSet[r]];
  If[MatchesQ[PatSet[r], x], s,
    PatSet @@ Join[{x}, List @@ s]]]

TermSubstitute[s_][p_PatSet] := TermSubstitute[s] /@ p;

(* Combinatorial aspects of Gedanken *)

```

```

(* Counting pattern set matches in Lkmn *)

(* Lkmn *)

(* L *)

L[k_,m_,n_] := EPCPPL[ElementaryType[0,Vector[k,m]]][n]

PC[k_] := L[k,0,0]

(* AdjacentLWithK, Emls *)

Emls[n_] := AdjacentLWithK[4,2,3,n];

(* Counting pattern set matches in the language LkOn *)

ConnectedComponents[P_PatSet] :=
Block[{PV,V,n,E,c,C},
  PV = {Relations[#],#}& /@ (List @@ P);
  V = #[[1]]& /@ PV;
  n = Max[Join @@ V];
  E = Table[i, {i, 1, n}];
  Block[{m},
    m = Min[(E[[#]])& /@ #];
    (E[[#]] = m)& /@ #& /@ V;
    c = Sum[If[E[[i]] == i,1,0], {i,1,n}];
    C = Table[{} , {i,1,n}];
    Block[{p}, p=E[[#[[1,1]]]];
    C[[p]]=Join[C[[p]],{#[[2]]}]]& /@ PV;
    PatSet @@ # & /@ Union /@ Select[C,Not[MatchQ[{} ,#]]&]]

Cardinality[Matches[L[k_,0,n_]] [P_PatSet]] :=
Block[{rho,cc,m},
  Print["Pi6 = ", P, " in PC[", k,""]];
  cc = ConnectedComponents[
    RetractRelations[MostGeneral[
      Canonical[RetractRelations[P]]]]];
  If[Length[cc] > 1,
    Cardinality[Matches[L[k,0,n]] [Components[cc]]],
    Cardinality[Matches[L[k,0,n]] [
      SieveMethod[cc[[1]]]]]]

Cardinality[Matches[L[k_,m_,n_]] [Components[{}]]] := 0

Cardinality[Matches[L[k_,0,n_]] [Components[{c1_}]]] :=
Cardinality[Matches[L[k,0,n]] [
  SieveMethod[RetractRelations[c1]]]];

Cardinality[Matches[L[k_,0,n_]] [Components[{c1_,r_}]]] :=
Block[{Single,c},
  c = {c1,r};
  Single = Split[c, Length[#] == 1&];

```

```

If[MatchQ[Single[[1]],{ }],
  Cardinality[Matches[L[k,0,n]][Theorem1[c]]],
  Cardinality[Matches[L[k,0,n]][
    Theorem2[Single[[1]][[1]],
    Join[Rest[Single[[1]]],Single[[2]]]]]]]]

CountNs[k_,P_PatSet,S_] := Plus @@ (CountSievePlace[k,P] /@ S);

CountPatternIntersection[k_,P_PatSet,s_] :=
Block[{U},
  U = Table[0, {i,1,k}];
  ((U[[#]] = Max[U[[#]],1])& /@
    (Head /@ #))& /@ (P[[#]]& /@ s);
  Plus @@ U]

CountSievePlace[k_,P_PatSet][s_] :=
  2^(k-CountPatternIntersection[k,P,s]);

Cardinality[Matches[L[k_,0,n_]] [SieveMethod[P_PatSet]]] :=
Block[{p,U,r,N, v},
  p = Length[P];
  U = NonEmptyRankedPowerSet[p];
  r = Table[i, {i,1,p}];
  N = CountNs[k,P,U[[#]]]& /@ r;
  Sum[(-1)^(i-1) N[[i]], {i,1,p}]]

Cardinality[Matches[L[k_,0,n_]] [Theorem1[c_]]] :=
Block[{r1, r2, MP1r1, MP2r2, t1},
  r1 = Length[Relations[c[[1]]]];
  r2 = Length[Relations[Join @@ (Rest[c])]];
  MP1r1=Cardinality[Matches[L[r1,0,n]] [Components[{c[[1]]}]]];
  MP2r2=Cardinality[Matches[L[r2,0,n]] [Components[Rest[c]]]];
  2^(k - r1 - r2)(2^r2 MP1r1 + 2^r1 MP2r2 - MP1r1 MP2r2)]

Cardinality[Matches[L[k_,0,n_]] [Theorem2[c1_,c_]]] :=
Block[{r1, MP2kr1, c2},
  r1 = Length[Relations[c1]];
  MP2kr1 = Cardinality[Matches[L[k-r1,0,n]] [Components[c]]];
  2^(k - r1) + (2^r1 - 1) MP2kr1]

Cardinality[Matches[L[k_,0,n_]] [Conjoin[A_PatSet, B_PatSet]]] :=
Cardinality[Matches[L[k,0,n]] [PatSet @@
  (Union@@#& /@
    Construct[CartesianProduct[{List@@A,List@@B}]]]]]

(* Match count in Lkmn by projection to Lkn,m-1,n *)

(* Elide, ExpandVars, PatOccurs, Relations,
  ReInHeadSubst, ReInSubstitute, RetractRelations *)

Elide[x_,r_] := Join @@ (If[MatchQ[x,#], { }, {#}]& /@ r)

BuildSpace[n_][-Infinity] := {{}};

```

```

BuildSpace[n_][w_Integer] :=
  Construct[CartesianProduct[Table[Pos[n],{i,1,w}]]]

GetVars[P_Conjunct] :=
  Union @@ (GetVars[#]& /@ (Head /@ (List @@ P)));

GetVars[i_Integer] := {};

GetVars[a_ + b_] := Join[GetVars[a], GetVars[b]];

GetVars[a_ * b_] := Join[GetVars[a], GetVars[b]];

GetVars[V_Var] := {V}

ExpandVars[n_,w_,P_PatSet] :=
  PatSet@@
  Union[Union /@
    (Join@@ (List@@#& /@
      (RelnSubstitute[#][P]& /@ BuildSpace[n][w])))]

PatOccurs[Conjunct[p_]] :=
  NoDuplicates[Flatten[List@@#& /@ {p}]];

PatOccurs[P_PatSet] := NoDuplicates[Join @@ (PatOccurs /@ P)]

Relations[Conjunct[p_]] := Union[Head /@ {p}];

Relations[PatSet[p_]] := Union @@ (Relations /@ {p})

RelnSubstitute[v_][P_PatSet] := RelnSubstitute[v] /@ P;

RelnSubstitute[v_][P_Conjunct] := RelnSubstitute[v] /@ P;

RelnSubstitute[v_][e_[]] := RelnHeadSubst[v][e][];

RelnSubstitute[v_][e_[f_]] := RelnHeadSubst[v][e][f];

RelnHeadSubst[v_][a_ + b_] :=
  RelnHeadSubst[v][a] + RelnHeadSubst[v][b];

RelnHeadSubst[v_][a_ * b_] :=
  RelnHeadSubst[v][a] RelnHeadSubst[v][b];

RelnHeadSubst[v_][x_Integer] := x;

RelnHeadSubst[v_][Var[i_]] := v[[i]]

RetractRelations[P_PatSet] :=
  Block[{r, sigma, i, j, n},
    r = Relations[P];
    n = Length[r];
    sigma = Table[1, {i, 1, Max[r]}];

```

```

For[j = 1, j <= n, j++, sigma[[r[[j]]]] = j];
((sigma[[Head[#]]] @@ #) & /@ #)& /@ P

ConsecutiveVars[{}] := {};

ConsecutiveVars[{r_}] :=
Block[{c},
c = 1; Table[If[MemberQ[{r},i], Var[c++], i],{i,1,Max[r]}]]

ProjectDown[L[k_,m_,1]] [P_PatSet] :=
Transfer[L[k,m,1]] /@ RetractRelations[Canonical[P]] /; m > 0

ProjectDown[L[k_,m_,n_]] [P_PatSet] :=
Block[{T,w,U,i,sigma},
S = RetractRelations[Canonical[P]];
w = Max @@ PatOccurs[S];
T = If[w == -Infinity, {S},
sigma = If[n==1,
Table[1,{i,1,w}],Table[Var[i], {i,1,w}]];
TermSubstitute[sigma][S]];
HoldVars[w][Transfer[L[k,m,n]] /@ T]] /; m > 0 && n > 1

ProjectDown[AdjacentLWithK[j_,k_,m_,n_]] [P_PatSet] :=
Block[{w,Q,R,V,sigma},
Q = ((Block[{h},h=Head[#];If[h>j,h-j,h]] @@ #)& /@ #)& /@ P;
V = Occurs[Q];
w = Max[0,Length[V]];
sigma = ConsecutiveVars[V];
R = TermSubstitute[sigma][Q];
Print["Pi2 = ", R];
R = R /. IndConst[1] -> n+1;
Print["Pi3 = ", R];
HoldVars[w][Transfer[AdjacentLWithK[j,k,m,n]] /@ R]] /; m>0

Transfer[L_] [p_PatSet] := Transfer[L] /@ p;

Transfer[L_] [p_Conjunct] := Transfer[L] /@ p;

Transfer[L[k_,m_,n_]] [e_] := e;

Transfer[L[k_,m_,n_]] [e_[f_]] :=
(Expand[(e - 1) n + First[{f}]] @@ Rest[{f}])

Transfer[AdjacentLWithK[j_,k_,m_,n_]] [e_Integer[]] :=
(e + k*(n + 1))[];

Transfer[AdjacentLWithK[j_,k_,m_,n_]] [e_Integer[f_]] :=
If[Length[{f}] == m,
((e - 1) (n + 1) + First[{f}]) @@ (Rest[{f}]),
(e + k*(n + 1)) @@ {f}]

Cardinality[Matches[L[k_,m_,n_]] [P_PatSet]] :=
Cardinality[Matches[L[k,m,n]] [Theorem3[P]]] /; m > 0

```

```

Cardinality[Matches[L[k_,m_,n_]] [Theorem3[P_PatSet]]] :=
  Cardinality[Matches[L[k n, m-1, n]] [
    ProjectDown[L[k,m,n]] [P]]] /; m > 0

Cardinality[Matches[L[k_,0,n_]] [HoldVars[w_] [P_PatSet]]] :=
  Cardinality[Matches[L[k,0,n]] [ExpandVars[n,w,P]]]

Cardinality[Matches[L[k_,m_,n_]] [HoldVars[w_] [P_PatSet]]] :=
  Module[{R},
    R = HoldVars[w] [Transfer[L[k,m,n]] [P]];
    Print["Pi5 = ", R, " in L[" , k n, ",", m-1, ",", n, ""];
    Cardinality[Matches[L[k n, m-1, n]] [R]]] /; m > 0

Cardinality[Matches[L[k_,m_,n_]] [P_PatSet]] := (*remember*)
( (* Cardinality[Matches[L[k,m,n]] [P]] = *)
  Cardinality[Matches[L[k,m,n]] [Theorem4[P]]]) /; m > 0

Cardinality[Matches[L[k_,m_,n_]] [Theorem4[r_PatSet]]] :=
  Block[{rho,cc,cnt},
    rho = MostGeneral[Canonical[RetractRelations[r]]];
    cc = ConnectedComponents[rho];
    If[Length[cc] > 1,
      Cardinality[Matches[L[k,m,n]] [Components[cc]]],
      Cardinality[Matches[L[k,m,n]] [Theorem3[rho]]]] /; m > 0

Cardinality[Matches[L[k_,m_,n_]] [Components[{c1_}]]] :=
  Cardinality[Matches[L[k,m,n]] [Theorem3[c1]]] /; m > 0

Cardinality[Matches[L[k_,m_,n_]] [Components[{c1_,r_}]]] :=
  Cardinality[Matches[L[k,m,n]] [
    Theorem4[Components[{c1,r}]]]] /; m > 0

Cardinality[Matches[L[k_,m_,n_]] [Theorem4[Components[c_]]]] :=
  Block[{r1, r2, d, MP1r1, MP2r2, t1},
    d = n^m;
    r1 = Length[Relations[c[[1]]]];
    r2 = Length[Relations[Join @@ (Rest[c])]];
    MP1r1 = Cardinality[Matches[L[r1,m,n]] [
      Components[{c[[1]]}]]];
    MP2r2 = Cardinality[Matches[L[r2,m,n]] [
      Components[Rest[c]]]];
    2^((k - r1 - r2) d) (2^(r2 d) MP1r1 +
    2^(r1 d) MP2r2 - MP1r1 MP2r2)]

Cardinality[Matches[L[k_,m_,n_]] [
  Conjoin[A_PatSet, B_PatSet]]] :=
  Cardinality[Matches[L[k n, m-1, n]] [
    Conjoin[ProjectDown[L[k,m,n]] [A],
      ProjectDown[L[k,m,n]] [B]]]] /; m > 0

Cardinality[Matches[L[k_,0,n_]] [
  Conjoin[HoldVars[w1_] [A_], HoldVars[w2_] [B_]]]] :=

```

```

Cardinality[Matches[L[k,0,n]] [
  Conjoin[ExpandVars[n,w1,A],ExpandVars[n,w2,B]]]]

Cardinality[Matches[L[k_,m_,n_]] [
  Conjoin[HoldVars[w1_] [A_],HoldVars[w2_] [B_]]]] :=
Cardinality[Matches[L[k n, m-1, n]] [
  Conjoin[HoldVars[w1] [Transfer[L[k,m,n]] [A]],
    HoldVars[w2] [Transfer[L[k,m,n]] [B]]]]] /; m > 0

(* Match counting in a mixed arity language with a constant *)

Cardinality[AdjacentLWithK[j_,k_,m_,n_]] :=
Cardinality[L[j,m-1,n+1]] Cardinality[L[k,m,n+1]];

Cardinality[Matches[AdjacentLWithK[j_,k_,m_,n_]] [P_PatSet]] :=
(*remember
Cardinality[Matches[AdjacentLWithK[j,k,m,n]] [P]] =*)
Module[{R},
  R = ProjectDown[AdjacentLWithK[j,k,m,n]] [P];
  Print["Pi4 = ", R, " in L[", k*(n+1)+j, ",", m-1, ",", n+1,""];
  Cardinality[Matches[L[k*(n+1)+j, m-1, n+1]] [R]]];

Cardinality[Matches[AdjacentLWithK[j_,k_,m_,n_]] [
  Conjoin[A_PatSet, B_PatSet]]] :=
Cardinality[Matches[L[k*(n+1)+j, m-1, n+1]] [
  Conjoin @@ (ProjectDown[AdjacentLWithK[j,k,m,n]]
    /@ {A,B})]]];

(* The Model Graph algorithm for deciding EMLS *)

(* SMGVars, SMG, UpperBound *)

Cardinality[SMGVars[n_]] := 2^(4 n^3)

Cardinality[SMGVars[n_,k_]] := Binomial[4 n^3, k-n]

UpperBound[Cardinality[SMG[n_]]] :=
Block[{k},Sum[Cardinality[SMGVars[n,k]]^2,{k,n,n+4 n^3}]]

Log2[x_] := N[Log[x]/Log[2]];

```


Bibliography

- [AM90] Andrew W. Appel and David B. MacQueen. Standard ML Reference Manual. Princeton University, 4 May 1990.
- [BDCGL89] Shai Ben-David, Benny Chor, Oded Goldreich, and Michael Luby. On the theory of average case complexity. In Proceedings of the Twenty First Annual ACM Symposium on Theory of Computing, pages 204–216. ACM, 1989.
- [BG91] Andreas Blass and Yuri Gurevich. Randomizing reductions of search problems. In Foundations of Software Technology and Theoretical Computer Science, pages 10–24. Springer-Verlag Lecture Notes in Computer Science #560, 1991.
- [Bla84] Andreas Blass. The interaction between category theory and set theory. Contemporary Mathematics, 30:5–29, 1984.
- [BR83] Rodney M. Burstall and D.E. Rydeheard. Computational category theory. Department of Computer Science, University of Edinburgh, Scotland, March 1983.
- [Bri77] Jane Bridge. Beginning Model Theory. Oxford University Press, 1977.
- [BW91] Edward A. Bender and S. Gill Williamson. Foundations of Applied Combinatorics. Addison-Wesley, 1991.
- [CFO89] Domenico Cantone, Alfredo Ferro, and Eugenio G. Omodeo. Computable Set Theory, Volume 1. Oxford University Press, 1989.
- [CFOS87] Domenico Cantone, Alfredo Ferro, Eugenio G. Omodeo, and J.T. Schwartz. Decision algorithms for some fragments of analysis and related areas. Communications on Pure and Applied Mathematics, XL:281–300, 1987.
- [CH85] Thierry Coquand and Gerard Huet. Constructions: A higher order proof system for mechanizing mathematics. In EUROCAL '85. Springer-Verlag Lectures Notes in CS 203, 1985.
- [CH91] George E. Collins and Hoon Hong. Partial cylindrical algebraic decomposition for quantifier elimination. Journal of Symbolic Computation, 12(3):299–328, 1991.

- [Coo71] D.C. Cooper. Programs for mechanical program verification. In [MM71], pages 43–59, 1971.
- [Coo72] D.C. Cooper. Theorem proving in arithmetic without multiplication. In [MM72], pages 91–99, 1972.
- [COP90] Domenico Cantone, Eugenio Omodeo, and Alberto Policriti. The automation of syllogistic. II. Optimization and complexity issues. Journal of Automated Reasoning, 6(2):173–188, June 1990.
- [Coq86] Thierry Coquand. On the analogy between propositions and types. Translation by Walt Hill, HP Labs, of: Sur l’analogie entre les propositions et les types, in: **Combinators and Functional Programming Languages**, edited by Guy Cousineau, Pierre-Louis Curien and B. Robinet, Springer-Verlag Lecture Notes in Computer Science 242, 1986.
- [CP89] Jiazhen Cai and Robert Paige. Program derivation by fixed point computation. Science of Computer Programming, 11:197–261, 1988-1989.
- [CRea89] William Clinger, Jonathan Rees, and et al. Revised^{3.99} report on the algorithmic language Scheme. MIT Artificial Intelligence Laboratory, 31 August 1989.
- [CS87] Vasek Chvatal and Endre Szemerédi. Many hard examples for resolution. CS Dept, Rutgers University, April 1987.
- [CZ93] Edmund Clarke and Xudong Zhao. Analytica: A theorem prover for mathematics. The Mathematica Journal, 3(1):56–71, Winter 1993.
- [Dav57] Martin D. Davis. A program for Presburger’s algorithm. In Summer Institute for Symbolic Logic, pages 215–233, Cornell University, Ithaca, NY, 1957.
- [dB80] Jaco de Bakker. Mathematical Theory of Program Correctness. Prentice-Hall International Series in Computer Science, 1980.
- [Dea77] Edith Gail Deak. Derivation of a related group of searching praas. Technical Report Courant Computer Science Report 12, NYU CS Dept, 1977.
- [Dea80] Edith Gail Deak. A Transformational Approach to the Development and Verification of Programs in a Very High Level Language. PhD thesis, Computer Science Dept, NYU, June 1980.
- [Dea84] James H. Davenport and et al. Scratchpad ii programming language manual. Mathematical Sciences Dept, IBM Thomas J. Watson Research Center, Yorktown Heights, NY 10598, 15 October 1984.
- [DML92] B. Selman D. Mitchell and H. Levesque. Hard and easy distributions of SAT problems. In Proceedings of the 10th National Conference on Artificial Intelligence, pages 459–465, 1992.

- [DP60] Martin D. Davis and Hilary Putnam. A computing procedure for quantification theory. Journal of the Association for Computing Machinery, 7(3):201–215, 1960.
- [DS77] Martin Davis and Jacob Theodore Schwartz. Metamathematical extensibility for theorem verifiers. Technical Report Courant Computer Science Report 12. Reprinted as [DS79]., NYU CS Dept, 1977.
- [DS79] Martin Davis and Jacob Theodore Schwartz. Metamathematical extensibility for theorem verifiers. Computer and Mathematics with Applications, 5:217–230, 1979.
- [Dyb87] Hans Dybkjær. Category theory and some applications in computer science. Unpublished, 16 December 1987.
- [ea85] Bruce W. Char et al, editor. Maple User’s Guide. WATCOM Publications Limited, Waterloo, Ontario, 1985.
- [EHMLR66] S. Eilenberg, D.K. Harrison, S. Mac Lane, and H. Röhr, editors. Proceedings of the Conference on Categorical Algebra (La Jolla, 1965). Springer-Verlag, 1966.
- [Eri91] Lars Ericson. Making mutually referential sibling classes compile and run without coredumping in SETL2 version 2.2. Author’s address: 749 Washington Street, NY, NY, 10014, March 1991.
- [FK69] Solomon Feferman and G Kreisel. Set-theoretical foundations of category theory. In [Lan69], pages 201–247, 1969.
- [Flo67] R. Floyd. Assigning meanings to programs. Proc. Symp. Appl. Math., 19, 1967.
- [FOS80] Alfredo Ferro, Eugenio G. Omodeo, and Jacob T. Schwartz. Decision procedures for elementary sublanguages of set theory, I. Multi-level syllogistic and some extensions. Communications on Pure and Applied Mathematics, XXXIII:599–608, 1980.
- [FR74] M. Fischer and M.O. Rabin. Super exponential complexity of presburger arithmetic. Technical Report Project MAC Technical Memo 43, Massachusetts Institute of Technology, 1974.
- [FR75] Jeanne Ferrante and Charles Rackoff. A decision procedure for the first order theory of real addition with order. SIAM Journal of Computing, 4(1):69–76, March 1975.
- [Fra86] John Franco. On the probabilistic performance of algorithms for the satisfiability problem. Information Processing Letters, 23:103–106, 1986.
- [Fra91] John Franco. Elimination of infrequent variables improves average case performance of satisfiability algorithms. SIAM Journal of Computing, 20(6):1119–1127, December 1991.

- [FV87] Philippe Flajolet and J.S. Vitter. Average-case analysis of algorithms and data structures. Technical report, Institut National de Recherche en Informatique et en Automatique, B.P. 105-78153 Le Chesnay Cedex France, August 1987.
- [GJ79] Michael R. Garey and Davis S. Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman and Co., 1979.
- [Gol79] Allen T. Goldberg. On the complexity of the satisfiability problem. PhD thesis, CS Dept, New York University, October 1979. Courant Institute of Mathematical Sciences Report No. NSO-16.
- [Gor79] Michael J.C. Gordon. The denotational description of programming languages. Springer-Verlag, 1979.
- [Gur91] Yuri Gurevich. Average case completeness. Journal of Computer and System Sciences, 42:346–398, 1991.
- [HB68] D. Hilbert and P. Bernays. Grundlagen der Mathematik I. Springer-Verlag, Berlin, 1968.
- [Hoa69] C.A.R. Hoare. An axiomatic basis for computer programming. Communications of the ACM, 12:576–583, 1969.
- [Hon93] Hoon Hong. Risc-clp (real): Logic programming with non-linear constraints over the reals. In In [Benham93, pp. 133-159], 1993.
- [Hue80] Gérard Huet. Confluent reductions: Abstract properties and applications to term rewriting systems. Journal of the Association for Computing Machinery, 27(4):797–821, 1980.
- [Hue87] Gérard Huet. Initiation à la théorie des catégories. INRIA, B.P. 105-78153, Le Chesnay Cedex, France, 20 December 1987.
- [Hue88a] Gérard Huet. The constructive engine, version 4.5. Projet FORMEL, INRIA, B.P. 105-78153, Le Chesnay Cedex, France, 1988.
- [Hue88b] Gérard Huet. Induction principles formalized in the calculus of constructions. In Programming of Future Generation Computers, pages 205–215. North-Holland, 1988.
- [Hue89] Gérard Huet. The Calculus of Constructions: Documentation and user's guide version 4.10, July 1989.
- [JS92] Richard Jenks and R. Sutor. Axiom. Springer-Verlag, 1992.
- [Lam92] Larry Lambe. Next generation computer algebra systems and the scratch-pad concept: applications to research in algebra. In 21st Nordic Congress of Mathematicians, Luleå, Sweden, June 1992.
- [Lan69] Saunders Mac Lane, editor. Reports of the Midwest Category Seminar III, volume 106 of Lecture Notes in Mathematics. Springer-Verlag, 1969.

- [Law66] F. William Lawvere. The category of categories as a foundation for mathematics. In [EHMLR66], pages 1–20, 1966.
- [Lev86] L. Levin. Average case complete problems. SIAM J. Comput., 15:285–286, 1986.
- [Liu68] C.L. Liu. Introduction to Combinatorial Mathematics. McGraw-Hill, 1968.
- [LV88] Ming Li and Paul M.B. Vitanyi. Two decades of applied kolmogorov complexity. In Third Annual Conference on Structure in Complexity. Washington, D.C., June 1988.
- [Man74] Zohar Manna. Mathematical theory of computation. Addison-Wesley, 1974.
- [Mis93] Bhubaneswar Mishra. Algorithmic Algebra. Springer-Verlag, 1993.
- [MM71] B. Meltzer and D. Michie, editors. Machine Intelligence, volume 6. American Elsevier, 1971.
- [MM72] B. Meltzer and D. Michie, editors. Machine Intelligence, volume 7. American Elsevier, 1972.
- [Moh86] Christine Mohring. Algorithm development in the calculus of constructions. In Symposium on Logic in Computer Science. IEEE Computer Society Press, Cambridge, Massachusetts, 1986.
- [MY89] Bud Mishra and Chee Yap. Notes on gröbner bases. Information Sciences, 48:219–252, 1989.
- [Opp75] D. Oppen. A 2^{2^n} upper bound on the complexity of Presburger arithmetic. PhD thesis, U. of Toronto, Ontario, 1975.
- [Pet90] Marko Petkovsek. Finding Closed-Form Solutions of Difference Equations by Symbolic Methods. PhD thesis, School of Computer Science, Carnegie-Mellon University, 1990. Available as CMU CSD Tech Report CMU-CS-91-103.
- [Pie90] Benjamin C. Pierce. A taste of category theory for computer scientists. Technical Report CMU-CS-90-113, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA 15213, 6 March 1990.
- [PM88] Christine Paulin-Mohring. Extracting f_ω ’s programs from proofs in the calculus of constructions. Laboratoire d’Informatique, Ecole Normale Supérieure, 45 Rue d’Ulm, 75230 Paris Cedex 05, France, April 28 1988.
- [Pol87a] Alberto Policriti. The NP-Completeness of MLS. Computer Science Department, New York University, November 1987.
- [Pol87b] Alberto Policriti. Survey on results in the decidability of sets. Computer Science Department, New York University, 1987.

- [Pra77] V.R. Pratt. Two easy theories whose combination is hard. Massachusetts Institute of Technology, 1977.
- [PS92] Alberto Policriti and Jacob Theodore Schwartz. T — theorem proving. Technical Report Technical Report 8/92, University of Udine, July 1992.
- [PT92] Alberto Policriti and Presad Tetali. On the satisfiability problem for the ground case of first order theories. Technical Report Technical Report DIMACS-92-38, Computer Science, Rutgers University, 1992.
- [RI82] Jonathan Rees and Norman I. Adams IV. T: A dialect of lisp or lambda: The ultimate software tool. In 1982 ACM Symposium on LISP and Functional Programming. Association for Computing Machinery, Order Number 552820, August 1982.
- [RIM84] Jonathan Rees, Norman I. Adams IV, and James R. Meehan. The t manual, fourth edition. Yale University Computer Science Department, 10 January 1984.
- [Rob65] J.A. Robinson. A machine-oriented logic based on the resolution principle. Journal of the Association for Computing Machinery, 12:23–41, January 1965.
- [RS93] Rüdiger Reischuk and Christian Schindelhauer. Precise average case complexity. In 10th Annual Symposium on Theoretical Aspects of Computer Science, pages 650–661, 1993.
- [Sch77] Jacob Theodore Schwartz. On correct-program technology. Technical Report Courant Computer Science Report 12, NYU CS Dept, 1977.
- [Sch78] Jacob Theodore Schwartz. Program proof technology. Technical Report Technical Survey No. 1, NYU CS Dept, 1978.
- [Sch92] Jacob Theodore Schwartz. Survey of computational logic. Course notes, NYU Courant Institute, Fall, 1991-1992.
- [Sho77] Robert E. Shostak. On the SUP-INF method for proving Presburger formulas. Journal of the Association for Computing Machinery, 24(4):529–543, October 1977.
- [Sho81] R. Shostak. Deciding linear inequalities by computing loop residues. Journal of the Association for Computing Machinery, 28(4):769–779, 1981.
- [Sim75] J. Simon. On Some Central Problems in Computational Complexity. PhD thesis, Cornell University Computer Science Department, 1975.
- [Sny90a] W. Kirk Snyder. The SETL2 programming language. Department of Computer Science, New York University, September 1990.
- [Sny90b] W. Kirk Snyder. The SETL2 programming language: Update on current developments. Department of Computer Science, New York University, September 1990.

- [SSS81] Ed Schonberg, Jacob Theodore Schwartz, and Micha Sharir. An automatic technique for selection of data representations in setl programs. ACM TOPLAS, 3(2):126–143, 1981.
- [SY92] Rainer Schuler and Tomoyuki Yamakami. Structural average case complexity. In Foundations of Software Technology and Theoretical Computer Science, pages 128–139. Springer-Verlag Lecture Notes in Computer Science #652, 1992.
- [VR92] R. Venkatesan and S. Rajagopalan. Average case intractability of matrix and Diophantine problems. In 24th Annual ACM STOC, pages 632–642, May 1992.
- [WAL⁺87] Pierre Weis, María-Virginia Aponte, Alain Laville, Michel Mauny, and Ascánder Suárez. The CAML Reference Manual, Version 2.5. Projet FORMEL, INRIA, B.P. 105-78153, Le Chesnay Cedex, France, 1 December 1987.
- [Wil90] Herbert S. Wilf. generatingfunctionology. Academic Press, 1990.
- [Wol91] Stephen Wolfram. Mathematica: A System for Doing Mathematics by Computer. Addison-Wesley, 1991.